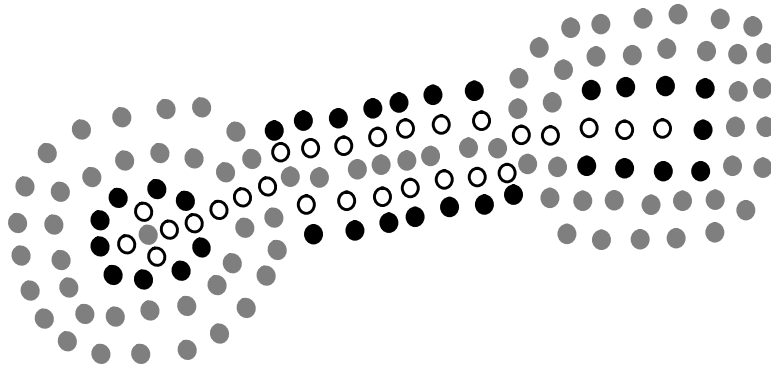


Chapter

6

Stacks, Queues, and Deques



Contents

6.1	Stacks	229
6.1.1	The Stack Abstract Data Type	230
6.1.2	Simple Array-Based Stack Implementation	231
6.1.3	Reversing Data Using a Stack	235
6.1.4	Matching Parentheses and HTML Tags	236
6.2	Queues	239
6.2.1	The Queue Abstract Data Type	240
6.2.2	Array-Based Queue Implementation	241
6.3	Double-Ended Queues	247
6.3.1	The Deque Abstract Data Type	247
6.3.2	Implementing a Deque with a Circular Array	248
6.3.3	Deques in the Python Collections Module	249
6.4	Exercises	250

6.1 Stacks

A *stack* is a collection of objects that are inserted and removed according to the *last-in, first-out (LIFO)* principle. A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called “top” of the stack). The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser. In this case, the fundamental operations involve the “pushing” and “popping” of plates on the stack. When we need a new plate from the dispenser, we “pop” the top plate off the stack, and when we add a plate, we “push” it down on the stack to become the new top plate. Perhaps an even more amusing example is a PEZ[®] candy dispenser, which stores mint candies in a spring-loaded container that “pops” out the topmost candy in the stack when the top of the dispenser is lifted (see Figure 6.1). Stacks are a fundamental data structure. They are used in many applications, including the following.

Example 6.1: Internet Web browsers store the addresses of recently visited sites in a stack. Each time a user visits a new site, that site’s address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.

Example 6.2: Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.

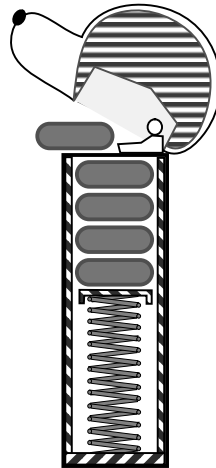


Figure 6.1: A schematic drawing of a PEZ[®] dispenser; a physical implementation of the stack ADT. (PEZ[®] is a registered trademark of PEZ Candy, Inc.)

6.1.1 The Stack Abstract Data Type

Stacks are the simplest of all data structures, yet they are also among the most important. They are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms. Formally, a stack is an abstract data type (ADT) such that an instance *S* supports the following two methods:

S.push(e): Add element *e* to the top of stack *S*.

S.pop(): Remove and return the top element from the stack *S*; an error occurs if the stack is empty.

Additionally, let us define the following accessor methods for convenience:

S.top(): Return a reference to the top element of stack *S*, without removing it; an error occurs if the stack is empty.

S.is_empty(): Return True if stack *S* does not contain any elements.

len(S): Return the number of elements in stack *S*; in Python, we implement this with the special method `--len--`.

By convention, we assume that a newly created stack is empty, and that there is no a priori bound on the capacity of the stack. Elements added to the stack can have arbitrary type.

Example 6.3: The following table shows a series of stack operations and their effects on an initially empty stack *S* of integers.

Operation	Return Value	Stack Contents
S.push(5)	—	[5]
S.push(3)	—	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	—	[7]
S.push(9)	—	[7, 9]
S.top()	9	[7, 9]
S.push(4)	—	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	—	[7, 9, 6]
S.push(8)	—	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

6.1.2 Simple Array-Based Stack Implementation

We can implement a stack quite easily by storing its elements in a Python list. The list class already supports adding an element to the end with the `append` method, and removing the last element with the `pop` method, so it is natural to align the top of the stack at the end of the list, as shown in Figure 6.2.

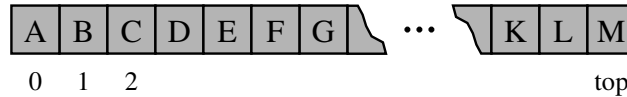


Figure 6.2: Implementing a stack with a Python list, storing the top element in the rightmost cell.

Although a programmer could directly use the list class in place of a formal stack class, lists also include behaviors (e.g., adding or removing elements from arbitrary positions) that would break the abstraction that the stack ADT represents. Also, the terminology used by the list class does not precisely align with traditional nomenclature for a stack ADT, in particular the distinction between `append` and `push`. Instead, we demonstrate how to use a list for internal storage while providing a public interface consistent with a stack.

The Adapter Pattern

The *adapter* design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface. One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable. By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way. In the context of the stack ADT, we can adapt Python’s list class using the correspondences shown in Table 6.1.

<i>Stack Method</i>	<i>Realization with Python list</i>
<code>S.push(e)</code>	<code>L.append(e)</code>
<code>S.pop()</code>	<code>L.pop()</code>
<code>S.top()</code>	<code>L[-1]</code>
<code>S.is_empty()</code>	<code>len(L) == 0</code>
<code>len(S)</code>	<code>len(L)</code>

Table 6.1: Realization of a stack *S* as an adaptation of a Python list *L*.

Implementing a Stack Using a Python List

We use the adapter design pattern to define an `ArrayStack` class that uses an underlying Python list for storage. (We choose the name `ArrayStack` to emphasize that the underlying storage is inherently array based.) One question that remains is what our code should do if a user calls `pop` or `top` when the stack is empty. Our ADT suggests that an error occurs, but we must decide what type of error. When `pop` is called on an empty Python list, it formally raises an `IndexError`, as lists are index-based sequences. That choice does not seem appropriate for a stack, since there is no assumption of indices. Instead, we can define a new exception class that is more appropriate. Code Fragment 6.1 defines such an `Empty` class as a trivial subclass of the Python Exception class.

```
class Empty(Exception):
    """Error attempting to access an element from an empty container."""
    pass
```

Code Fragment 6.1: Definition for an `Empty` exception class.

The formal definition for our `ArrayStack` class is given in Code Fragment 6.2. The constructor establishes the member `self._data` as an initially empty Python list, for internal storage. The rest of the public stack behaviors are implemented, using the corresponding adaptation that was outlined in Table 6.1.

Example Usage

Below, we present an example of the use of our `ArrayStack` class, mirroring the operations at the beginning of Example 6.3 on page 230.

<code>S = ArrayStack()</code>	<code># contents: []</code>	
<code>S.push(5)</code>	<code># contents: [5]</code>	
<code>S.push(3)</code>	<code># contents: [5, 3]</code>	
<code>print(len(S))</code>	<code># contents: [5, 3];</code>	outputs 2
<code>print(S.pop())</code>	<code># contents: [5];</code>	outputs 3
<code>print(S.is_empty())</code>	<code># contents: [5];</code>	outputs False
<code>print(S.pop())</code>	<code># contents: [];</code>	outputs 5
<code>print(S.is_empty())</code>	<code># contents: [];</code>	outputs True
<code>S.push(7)</code>	<code># contents: [7]</code>	
<code>S.push(9)</code>	<code># contents: [7, 9]</code>	
<code>print(S.top())</code>	<code># contents: [7, 9];</code>	outputs 9
<code>S.push(4)</code>	<code># contents: [7, 9, 4]</code>	
<code>print(len(S))</code>	<code># contents: [7, 9, 4];</code>	outputs 3
<code>print(S.pop())</code>	<code># contents: [7, 9];</code>	outputs 4
<code>S.push(6)</code>	<code># contents: [7, 9, 6]</code>	

```

1  class ArrayStack:
2      """ LIFO Stack implementation using a Python list as underlying storage."""
3
4      def __init__(self):
5          """ Create an empty stack."""
6          self._data = [ ]                # nonpublic list instance
7
8      def __len__(self):
9          """ Return the number of elements in the stack."""
10         return len(self._data)
11
12     def is_empty(self):
13         """ Return True if the stack is empty."""
14         return len(self._data) == 0
15
16     def push(self, e):
17         """ Add element e to the top of the stack."""
18         self._data.append(e)            # new item stored at end of list
19
20     def top(self):
21         """ Return (but do not remove) the element at the top of the stack.
22
23         Raise Empty exception if the stack is empty.
24         """
25         if self.is_empty():
26             raise Empty('Stack is empty')
27         return self._data[-1]           # the last item in the list
28
29     def pop(self):
30         """ Remove and return the element from the top of the stack (i.e., LIFO).
31
32         Raise Empty exception if the stack is empty.
33         """
34         if self.is_empty():
35             raise Empty('Stack is empty')
36         return self._data.pop( )        # remove last item from list

```

Code Fragment 6.2: Implementing a stack using a Python list as storage.

Analyzing the Array-Based Stack Implementation

Table 6.2 shows the running times for our `ArrayStack` methods. The analysis directly mirrors the analysis of the list class given in Section 5.3. The implementations for `top`, `is_empty`, and `len` use constant time in the worst case. The $O(1)$ time for `push` and `pop` are *amortized* bounds (see Section 5.3.2); a typical call to either of these methods uses constant time, but there is occasionally an $O(n)$ -time worst case, where n is the current number of elements in the stack, when an operation causes the list to resize its internal array. The space usage for a stack is $O(n)$.

Operation	Running Time
<code>S.push(e)</code>	$O(1)^*$
<code>S.pop()</code>	$O(1)^*$
<code>S.top()</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$
<code>len(S)</code>	$O(1)$

*amortized

Table 6.2: Performance of our array-based stack implementation. The bounds for `push` and `pop` are amortized due to similar bounds for the list class. The space usage is $O(n)$, where n is the current number of elements in the stack.

Avoiding Amortization by Reserving Capacity

In some contexts, there may be additional knowledge that suggests a maximum size that a stack will reach. Our implementation of `ArrayStack` from Code Fragment 6.2 begins with an empty list and expands as needed. In the analysis of lists from Section 5.4.1, we emphasized that it is more efficient in practice to construct a list with initial length n than it is to start with an empty list and append n items (even though both approaches run in $O(n)$ time).

As an alternate model for a stack, we might wish for the constructor to accept a parameter specifying the maximum capacity of a stack and to initialize the `_data` member to a list of that length. Implementing such a model requires significant changes relative to Code Fragment 6.2. The size of the stack would no longer be synonymous with the length of the list, and pushes and pops of the stack would not require changing the length of the list. Instead, we suggest maintaining a separate integer as an instance variable that denotes the current number of elements in the stack. Details of such an implementation are left as Exercise C-6.17.

6.1.3 Reversing Data Using a Stack

As a consequence of the LIFO protocol, a stack can be used as a general tool to reverse a data sequence. For example, if the values 1, 2, and 3 are pushed onto a stack in that order, they will be popped from the stack in the order 3, 2, and then 1.

This idea can be applied in a variety of settings. For example, we might wish to print lines of a file in reverse order in order to display a data set in decreasing order rather than increasing order. This can be accomplished by reading each line and pushing it onto a stack, and then writing the lines in the order they are popped. An implementation of such a process is given in Code Fragment 6.3.

```
1 def reverse_file(filename):
2     """ Overwrite given file with its contents line-by-line reversed."""
3     S = ArrayStack()
4     original = open(filename)
5     for line in original:
6         S.push(line.rstrip('\n'))      # we will re-insert newlines when writing
7     original.close()
8
9     # now we overwrite with contents in LIFO order
10    output = open(filename, 'w')      # reopening file overwrites original
11    while not S.is_empty():
12        output.write(S.pop() + '\n')  # re-insert newline characters
13    output.close()
```

Code Fragment 6.3: A function that reverses the order of lines in a file.

One technical detail worth noting is that we intentionally strip trailing newlines from lines as they are read, and then re-insert newlines after each line when writing the resulting file. Our reason for doing this is to handle a special case in which the original file does not have a trailing newline for the final line. If we exactly echoed the lines read from the file in reverse order, then the original last line would be followed (without newline) by the original second-to-last line. In our implementation, we ensure that there will be a separating newline in the result.

The idea of using a stack to reverse a data set can be applied to other types of sequences. For example, Exercise R-6.5 explores the use of a stack to provide yet another solution for reversing the contents of a Python list (a recursive solution for this goal was discussed in Section 4.4.1). A more challenging task is to reverse the order in which elements are stored within a stack. If we were to move them from one stack to another, they would be reversed, but if we were to then replace them into the original stack, they would be reversed again, thereby reverting to their original order. Exercise C-6.18 explores a solution for this task.

6.1.4 Matching Parentheses and HTML Tags

In this subsection, we explore two related applications of stacks, both of which involve testing for pairs of matching delimiters. In our first application, we consider arithmetic expressions that may contain various pairs of grouping symbols, such as

- Parentheses: “(” and “)”
- Braces: “{” and “}”
- Brackets: “[” and “]”

Each opening symbol must match its corresponding closing symbol. For example, a left bracket, “[,” must match a corresponding right bracket, “],” as in the expression $[(5+x)-(y+z)]$. The following examples further illustrate this concept:

- Correct: $()(())\{([()])\}$
- Correct: $((())(())\{([()])\})$
- Incorrect: $)()(\{([()])\}$
- Incorrect: $\{([[]]\}$
- Incorrect: $($

We leave the precise definition of a matching group of symbols to Exercise R-6.6.

An Algorithm for Matching Delimiters

An important task when processing arithmetic expressions is to make sure their delimiting symbols match up correctly. Code Fragment 6.4 presents a Python implementation of such an algorithm. A discussion of the code follows.

```

1 def is_matched(expr):
2     """Return True if all delimiters are properly match; False otherwise."""
3     lefty = '([{' # opening delimiters
4     righty = ')]}' # respective closing delims
5     S = ArrayStack()
6     for c in expr:
7         if c in lefty:
8             S.push(c) # push left delimiter on stack
9         elif c in righty:
10            if S.is_empty():
11                return False # nothing to match with
12            if righty.index(c) != lefty.index(S.pop()):
13                return False # mismatched
14    return S.is_empty() # were all symbols matched?
```

Code Fragment 6.4: Function for matching delimiters in an arithmetic expression.

We assume the input is a sequence of characters, such as `'[(5+x)-(y+z)]'`. We perform a left-to-right scan of the original sequence, using a stack S to facilitate the matching of grouping symbols. Each time we encounter an opening symbol, we push that symbol onto S , and each time we encounter a closing symbol, we pop a symbol from the stack S (assuming S is not empty), and check that these two symbols form a valid pair. If we reach the end of the expression and the stack is empty, then the original expression was properly matched. Otherwise, there must be an opening delimiter on the stack without a matching symbol.

If the length of the original expression is n , the algorithm will make at most n calls to push and n calls to pop. Those calls run in a total of $O(n)$ time, even considering the amortized nature of the $O(1)$ time bound for those methods. Given that our selection of possible delimiters, $\{[, [$, has constant size, auxiliary tests such as `c in lefty` and `righty.index(c)` each run in $O(1)$ time. Combining these operations, the matching algorithm on a sequence of length n runs in $O(n)$ time.

Matching Tags in a Markup Language

Another application of matching delimiters is in the validation of markup languages such as HTML or XML. HTML is the standard format for hyperlinked documents on the Internet and XML is an extensible markup language used for a variety of structured data sets. We show a sample HTML document and a possible rendering in Figure 6.3.

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 6.3: Illustrating HTML tags. (a) An HTML document; (b) its rendering.

In an HTML document, portions of text are delimited by *HTML tags*. A simple opening HTML tag has the form “<name>” and the corresponding closing tag has the form “</name>”. For example, we see the <body> tag on the first line of Figure 6.3(a), and the matching </body> tag at the close of that document. Other commonly used HTML tags that are used in this example include:

- body: document body
- h1: section header
- center: center justify
- p: paragraph
- ol: numbered (ordered) list
- li: list item

Ideally, an HTML document should have matching tags, although most browsers tolerate a certain number of mismatching tags. In Code Fragment 6.5, we give a Python function that matches tags in a string representing an HTML document. We make a left-to-right pass through the raw string, using index *j* to track our progress and the find method of the str class to locate the '<' and '>' characters that define the tags. Opening tags are pushed onto the stack, and matched against closing tags as they are popped from the stack, just as we did when matching delimiters in Code Fragment 6.4. By similar analysis, this algorithm runs in $O(n)$ time, where *n* is the number of characters in the raw HTML source.

```

1 def is_matched_html(raw):
2     """Return True if all HTML tags are properly match; False otherwise."""
3     S = ArrayStack()
4     j = raw.find('<')                # find first '<' character (if any)
5     while j != -1:
6         k = raw.find('>', j+1)        # find next '>' character
7         if k == -1:
8             return False            # invalid tag
9         tag = raw[j+1:k]             # strip away < >
10        if not tag.startswith('/'):  # this is opening tag
11            S.push(tag)
12        else:                         # this is closing tag
13            if S.is_empty():
14                return False         # nothing to match with
15            if tag[1:] != S.pop():
16                return False         # mismatched delimiter
17            j = raw.find('<', k+1)      # find next '<' character (if any)
18    return S.is_empty()              # were all opening tags matched?

```

Code Fragment 6.5: Function for testing if an HTML document has matching tags.

6.2 Queues

Another fundamental data structure is the *queue*. It is a close “cousin” of the stack, as a queue is a collection of objects that are inserted and removed according to the *first-in, first-out (FIFO)* principle. That is, elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.

We usually say that elements enter a queue at the back and are removed from the front. A metaphor for this terminology is a line of people waiting to get on an amusement park ride. People waiting for such a ride enter at the back of the line and get on the ride from the front of the line. There are many other applications of queues (see Figure 6.4). Stores, theaters, reservation centers, and other similar services typically process customer requests according to the FIFO principle. A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant. FIFO queues are also used by many computing devices, such as a networked printer, or a Web server responding to requests.

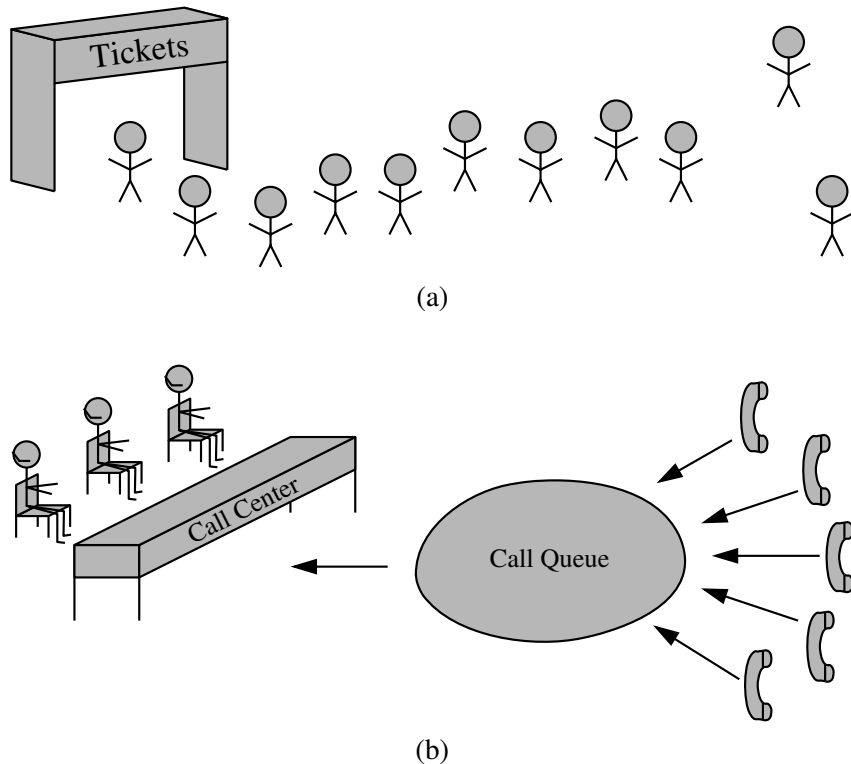


Figure 6.4: Real-world examples of a first-in, first-out queue. (a) People waiting in line to purchase tickets; (b) phone calls being routed to a customer service center.

6.2.1 The Queue Abstract Data Type

Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the *first* element in the queue, and element insertion is restricted to the back of the sequence. This restriction enforces the rule that items are inserted and deleted in a queue according to the first-in, first-out (FIFO) principle. The *queue* abstract data type (ADT) supports the following two fundamental methods for a queue *Q*:

Q.enqueue(*e*): Add element *e* to the back of queue *Q*.

Q.dequeue(): Remove and return the first element from queue *Q*;
an error occurs if the queue is empty.

The queue ADT also includes the following supporting methods (with first being analogous to the stack's top method):

Q.first(): Return a reference to the element at the front of queue *Q*,
without removing it; an error occurs if the queue is empty.

Q.is_empty(): Return True if queue *Q* does not contain any elements.

len(*Q*): Return the number of elements in queue *Q*; in Python,
we implement this with the special method `__len__`.

By convention, we assume that a newly created queue is empty, and that there is no a priori bound on the capacity of the queue. Elements added to the queue can have arbitrary type.

Example 6.4: The following table shows a series of queue operations and their effects on an initially empty queue *Q* of integers.

Operation	Return Value	first $\leftarrow Q \leftarrow$ last
Q.enqueue(5)	—	[5]
Q.enqueue(3)	—	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue()	5	[3]
Q.is_empty()	False	[3]
Q.dequeue()	3	[]
Q.is_empty()	True	[]
Q.dequeue()	“error”	[]
Q.enqueue(7)	—	[7]
Q.enqueue(9)	—	[7, 9]
Q.first()	7	[7, 9]
Q.enqueue(4)	—	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue()	7	[9, 4]

6.2.2 Array-Based Queue Implementation

For the stack ADT, we created a very simple adapter class that used a Python list as the underlying storage. It may be very tempting to use a similar approach for supporting the queue ADT. We could enqueue element e by calling `append(e)` to add it to the end of the list. We could use the syntax `pop(0)`, as opposed to `pop()`, to intentionally remove the *first* element from the list when dequeuing.

As easy as this would be to implement, it is tragically inefficient. As we discussed in Section 5.4.1, when `pop` is called on a list with a non-default index, a loop is executed to shift all elements beyond the specified index to the left, so as to fill the hole in the sequence caused by the `pop`. Therefore, a call to `pop(0)` always causes the worst-case behavior of $\Theta(n)$ time.

We can improve on the above strategy by avoiding the call to `pop(0)` entirely. We can replace the dequeued entry in the array with a reference to `None`, and maintain an explicit variable f to store the index of the element that is currently at the front of the queue. Such an algorithm for dequeue would run in $O(1)$ time. After several dequeue operations, this approach might lead to the configuration portrayed in Figure 6.5.

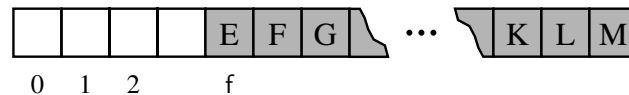


Figure 6.5: Allowing the front of the queue to drift away from index 0.

Unfortunately, there remains a drawback to the revised approach. In the case of a stack, the length of the list was precisely equal to the size of the stack (even if the underlying array for the list was slightly larger). With the queue design that we are considering, the situation is worse. We can build a queue that has relatively few elements, yet which are stored in an arbitrarily large list. This occurs, for example, if we repeatedly enqueue a new element and then dequeue another (allowing the front to drift rightward). Over time, the size of the underlying list would grow to $O(m)$ where m is the *total* number of enqueue operations since the creation of the queue, rather than the current number of elements in the queue.

This design would have detrimental consequences in applications in which queues have relatively modest size, but which are used for long periods of time. For example, the wait-list for a restaurant might never have more than 30 entries at one time, but over the course of a day (or a week), the overall number of entries would be significantly larger.

Using an Array Circularly

In developing a more robust queue implementation, we allow the front of the queue to drift rightward, and we allow the contents of the queue to “wrap around” the end of an underlying array. We assume that our underlying array has fixed length N that is greater than the actual number of elements in the queue. New elements are enqueued toward the “end” of the current queue, progressing from the front to index $N - 1$ and continuing at index 0, then 1. Figure 6.6 illustrates such a queue with first element E and last element M.



Figure 6.6: Modeling a queue with a circular array that wraps around the end.

Implementing this circular view is not difficult. When we dequeue an element and want to “advance” the front index, we use the arithmetic $f = (f + 1) \% N$. Recall that the `%` operator in Python denotes the *modulo* operator, which is computed by taking the remainder after an integral division. For example, 14 divided by 3 has a quotient of 4 with remainder 2, that is, $\frac{14}{3} = 4\frac{2}{3}$. So in Python, `14 // 3` evaluates to the quotient 4, while `14 % 3` evaluates to the remainder 2. The modulo operator is ideal for treating an array circularly. As a concrete example, if we have a list of length 10, and a front index 7, we can advance the front by formally computing $(7+1) \% 10$, which is simply 8, as 8 divided by 10 is 0 with a remainder of 8. Similarly, advancing index 8 results in index 9. But when we advance from index 9 (the last one in the array), we compute $(9+1) \% 10$, which evaluates to index 0 (as 10 divided by 10 has a remainder of zero).

A Python Queue Implementation

A complete implementation of a queue ADT using a Python list in circular fashion is presented in Code Fragments 6.6 and 6.7. Internally, the queue class maintains the following three instance variables:

- `_data`:** is a reference to a list instance with a fixed capacity.
- `_size`:** is an integer representing the current number of elements stored in the queue (as opposed to the length of the `_data` list).
- `_front`:** is an integer that represents the index within `_data` of the first element of the queue (assuming the queue is not empty).

We initially reserve a list of moderate size for storing data, although the queue formally has size zero. As a technicality, we initialize the `_front` index to zero.

When `front` or `dequeue` are called with no elements in the queue, we raise an instance of the `Empty` exception, defined in Code Fragment 6.1 for our stack.

```

1  class ArrayQueue:
2      """ FIFO queue implementation using a Python list as underlying storage."""
3      DEFAULT_CAPACITY = 10          # moderate capacity for all new queues
4
5      def __init__(self):
6          """ Create an empty queue."""
7          self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
8          self._size = 0
9          self._front = 0
10
11     def __len__(self):
12         """ Return the number of elements in the queue."""
13         return self._size
14
15     def is_empty(self):
16         """ Return True if the queue is empty."""
17         return self._size == 0
18
19     def first(self):
20         """ Return (but do not remove) the element at the front of the queue.
21
22         Raise Empty exception if the queue is empty.
23         """
24         if self.is_empty():
25             raise Empty('Queue is empty')
26         return self._data[self._front]
27
28     def dequeue(self):
29         """ Remove and return the first element of the queue (i.e., FIFO).
30
31         Raise Empty exception if the queue is empty.
32         """
33         if self.is_empty():
34             raise Empty('Queue is empty')
35         answer = self._data[self._front]
36         self._data[self._front] = None          # help garbage collection
37         self._front = (self._front + 1) % len(self._data)
38         self._size -= 1
39         return answer

```

Code Fragment 6.6: Array-based implementation of a queue (continued in Code Fragment 6.7).

```

40 def enqueue(self, e):
41     """Add an element to the back of queue."""
42     if self._size == len(self._data):
43         self._resize(2 * len(self._data))    # double the array size
44     avail = (self._front + self._size) % len(self._data)
45     self._data[avail] = e
46     self._size += 1
47
48 def _resize(self, cap):                      # we assume cap >= len(self)
49     """Resize to a new list of capacity >= len(self)."""
50     old = self._data                        # keep track of existing list
51     self._data = [None] * cap              # allocate list with new capacity
52     walk = self._front
53     for k in range(self._size):             # only consider existing elements
54         self._data[k] = old[walk]          # intentionally shift indices
55         walk = (1 + walk) % len(old)        # use old size as modulus
56     self._front = 0                         # front has been realigned

```

Code Fragment 6.7: Array-based implementation of a queue (continued from Code Fragment 6.6).

The implementation of `__len__` and `is_empty` are trivial, given knowledge of the size. The implementation of the `front` method is also simple, as the `_front` index tells us precisely where the desired element is located within the `_data` list, assuming that list is not empty.

Adding and Removing Elements

The goal of the `enqueue` method is to add a new element to the back of the queue. We need to determine the proper index at which to place the new element. Although we do not explicitly maintain an instance variable for the back of the queue, we compute the location of the next opening based on the formula:

$$\text{avail} = (\text{self._front} + \text{self._size}) \% \text{len}(\text{self._data})$$

Note that we are using the size of the queue as it exists *prior* to the addition of the new element. For example, consider a queue with capacity 10, current size 3, and first element at index 5. The three elements of such a queue are stored at indices 5, 6, and 7. The new element should be placed at index $(\text{front} + \text{size}) = 8$. In a case with wrap-around, the use of the modular arithmetic achieves the desired circular semantics. For example, if our hypothetical queue had 3 elements with the first at index 8, our computation of $(8+3) \% 10$ evaluates to 1, which is perfect since the three existing elements occupy indices 8, 9, and 0.

When the `dequeue` method is called, the current value of `self._front` designates the index of the value that is to be removed and returned. We keep a local reference to the element that will be returned, setting `answer = self._data[self._front]` just prior to removing the reference to that object from the list, with the assignment `self._data[self._front] = None`. Our reason for the assignment to `None` relates to Python's mechanism for reclaiming unused space. Internally, Python maintains a count of the number of references that exist to each object. If that count reaches zero, the object is effectively inaccessible, thus the system may reclaim that memory for future use. (For more details, see Section 15.1.2.) Since we are no longer responsible for storing a dequeued element, we remove the reference to it from our list so as to reduce that element's reference count.

The second significant responsibility of the `dequeue` method is to update the value of `_front` to reflect the removal of the element, and the presumed promotion of the second element to become the new first. In most cases, we simply want to increment the index by one, but because of the possibility of a wrap-around configuration, we rely on modular arithmetic as originally described on page 242.

Resizing the Queue

When `enqueue` is called at a time when the size of the queue equals the size of the underlying list, we rely on a standard technique of doubling the storage capacity of the underlying list. In this way, our approach is similar to the one used when we implemented a `DynamicArray` in Section 5.3.1.

However, more care is needed in the queue's `_resize` utility than was needed in the corresponding method of the `DynamicArray` class. After creating a temporary reference to the old list of values, we allocate a new list that is twice the size and copy references from the old list to the new list. While transferring the contents, we intentionally realign the front of the queue with index 0 in the new array, as shown in Figure 6.7. This realignment is not purely cosmetic. Since the modular arithmetic depends on the size of the array, our state would be flawed had we transferred each element to its same index in the new array.

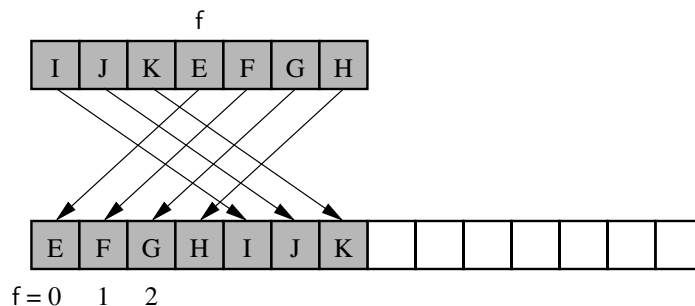


Figure 6.7: Resizing the queue, while realigning the front element with index 0.

Shrinking the Underlying Array

A desirable property of a queue implementation is to have its space usage be $\Theta(n)$ where n is the current number of elements in the queue. Our `ArrayQueue` implementation, as given in Code Fragments 6.6 and 6.7, does not have this property. It expands the underlying array when `enqueue` is called with the queue at full capacity, but the `dequeue` implementation never shrinks the underlying array. As a consequence, the capacity of the underlying array is proportional to the maximum number of elements that have ever been stored in the queue, not the current number of elements.

We discussed this very issue on page 200, in the context of dynamic arrays, and in subsequent Exercises C-5.16 through C-5.20 of that chapter. A robust approach is to reduce the array to half of its current size, whenever the number of elements stored in it falls below *one fourth* of its capacity. We can implement this strategy by adding the following two lines of code in our `dequeue` method, just after reducing `self._size` at line 38 of Code Fragment 6.6, to reflect the loss of an element.

```
if 0 < self._size < len(self._data) // 4:
    self._resize(len(self._data) // 2)
```

Analyzing the Array-Based Queue Implementation

Table 6.3 describes the performance of our array-based implementation of the queue ADT, assuming the improvement described above for occasionally shrinking the size of the array. With the exception of the `_resize` utility, all of the methods rely on a constant number of statements involving arithmetic operations, comparisons, and assignments. Therefore, each method runs in worst-case $O(1)$ time, except for `enqueue` and `dequeue`, which have *amortized* bounds of $O(1)$ time, for reasons similar to those given in Section 5.3.

Operation	Running Time
<code>Q.enqueue(e)</code>	$O(1)^*$
<code>Q.dequeue()</code>	$O(1)^*$
<code>Q.first()</code>	$O(1)$
<code>Q.is_empty()</code>	$O(1)$
<code>len(Q)</code>	$O(1)$

*amortized

Table 6.3: Performance of an array-based implementation of a queue. The bounds for `enqueue` and `dequeue` are amortized due to the resizing of the array. The space usage is $O(n)$, where n is the current number of elements in the queue.

6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a ***double-ended queue***, or ***deque***, which is usually pronounced “deck” to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation “D.Q.”

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occasionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will re-insert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

To provide a symmetrical abstraction, the deque ADT is defined so that deque D supports the following methods:

D.add_first(e): Add element e to the front of deque D.

D.add_last(e): Add element e to the back of deque D.

D.delete_first(): Remove and return the first element from deque D; an error occurs if the deque is empty.

D.delete_last(): Remove and return the last element from deque D; an error occurs if the deque is empty.

Additionally, the deque ADT will include the following accessors:

D.first(): Return (but do not remove) the first element of deque D; an error occurs if the deque is empty.

D.last(): Return (but do not remove) the last element of deque D; an error occurs if the deque is empty.

D.is_empty(): Return True if deque D does not contain any elements.

len(D): Return the number of elements in deque D; in Python, we implement this with the special method `__len__`.

Example 6.5: The following table shows a series of operations and their effects on an initially empty deque *D* of integers.

Operation	Return Value	Deque
D.add_last(5)	–	[5]
D.add_first(3)	–	[3, 5]
D.add_first(7)	–	[7, 3, 5]
D.first()	7	[7, 3, 5]
D.delete_last()	5	[7, 3]
len(D)	2	[7, 3]
D.delete_last()	3	[7]
D.delete_last()	7	[]
D.add_first(6)	–	[6]
D.last()	6	[6]
D.add_first(8)	–	[8, 6]
D.is_empty()	False	[8, 6]
D.last()	6	[8, 6]

6.3.2 Implementing a Deque with a Circular Array

We can implement the deque ADT in much the same way as the `ArrayQueue` class provided in Code Fragments 6.6 and 6.7 of Section 6.2.2 (so much so that we leave the details of an `ArrayDeque` implementation to Exercise P-6.32). We recommend maintaining the same three instance variables: `_data`, `_size`, and `_front`. Whenever we need to know the index of the back of the deque, or the first available slot beyond the back of the deque, we use modular arithmetic for the computation. For example, our implementation of the `last()` method uses the index

```
back = (self._front + self._size - 1) % len(self._data)
```

Our implementation of the `ArrayDeque.add_last` method is essentially the same as that for `ArrayQueue.enqueue`, including the reliance on a `_resize` utility. Likewise, the implementation of the `ArrayDeque.delete_first` method is the same as `ArrayQueue.dequeue`. Implementations of `add_first` and `delete_last` use similar techniques. One subtlety is that a call to `add_first` may need to wrap around the beginning of the array, so we rely on modular arithmetic to circularly *decrement* the index, as

```
self._front = (self._front - 1) % len(self._data)           # cyclic shift
```

The efficiency of an `ArrayDeque` is similar to that of an `ArrayQueue`, with all operations having $O(1)$ running time, but with that bound being amortized for operations that may change the size of the underlying list.

6.3.3 Deques in the Python Collections Module

An implementation of a deque class is available in Python's standard collections module. A summary of the most commonly used behaviors of the collections.deque class is given in Table 6.4. It uses more asymmetric nomenclature than our ADT.

Our Deque ADT	collections.deque	Description
len(D)	len(D)	number of elements
D.add_first()	D.appendleft()	add to beginning
D.add_last()	D.append()	add to end
D.delete_first()	D.popleft()	remove from beginning
D.delete_last()	D.pop()	remove from end
D.first()	D[0]	access first element
D.last()	D[-1]	access last element
	D[j]	access arbitrary entry by index
	D[j] = val	modify arbitrary entry by index
	D.clear()	clear all contents
	D.rotate(k)	circularly shift rightward k steps
	D.remove(e)	remove first matching element
	D.count(e)	count number of matches for e

Table 6.4: Comparison of our deque ADT and the collections.deque class.

The collections.deque interface was chosen to be consistent with established naming conventions of Python's list class, for which append and pop are presumed to act at the end of the list. Therefore, appendleft and popleft designate an operation at the beginning of the list. The library deque also mimics a list in that it is an indexed sequence, allowing arbitrary access or modification using the D[j] syntax.

The library deque constructor also supports an optional maxlen parameter to force a fixed-length deque. However, if a call to append at either end is invoked when the deque is full, it does not throw an error; instead, it causes one element to be dropped from the opposite side. That is, calling appendleft when the deque is full causes an implicit pop from the right side to make room for the new element.

The current Python distribution implements collections.deque with a hybrid approach that uses aspects of circular arrays, but organized into blocks that are themselves organized in a doubly linked list (a data structure that we will introduce in the next chapter). The deque class is formally documented to guarantee $O(1)$ -time operations at either end, but $O(n)$ -time worst-case operations when using index notation near the middle of the deque.

6.4 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

- R-6.1** What values are returned during the following series of stack operations, if executed upon an initially empty stack? `push(5)`, `push(3)`, `pop()`, `push(2)`, `push(8)`, `pop()`, `pop()`, `push(9)`, `push(1)`, `pop()`, `push(7)`, `push(6)`, `pop()`, `pop()`, `push(4)`, `pop()`, `pop()`.
- R-6.2** Suppose an initially empty stack *S* has executed a total of 25 push operations, 12 top operations, and 10 pop operations, 3 of which raised Empty errors that were caught and ignored. What is the current size of *S*?
- R-6.3** Implement a function with signature `transfer(S, T)` that transfers all elements from stack *S* onto stack *T*, so that the element that starts at the top of *S* is the first to be inserted onto *T*, and the element at the bottom of *S* ends up at the top of *T*.
- R-6.4** Give a recursive method for removing all the elements from a stack.
- R-6.5** Implement a function that reverses a list of elements by pushing them onto a stack in one order, and writing them back to the list in reversed order.
- R-6.6** Give a precise and complete definition of the concept of matching for grouping symbols in an arithmetic expression. Your definition may be recursive.
- R-6.7** What values are returned during the following sequence of queue operations, if executed on an initially empty queue? `enqueue(5)`, `enqueue(3)`, `dequeue()`, `enqueue(2)`, `enqueue(8)`, `dequeue()`, `dequeue()`, `enqueue(9)`, `enqueue(1)`, `dequeue()`, `enqueue(7)`, `enqueue(6)`, `dequeue()`, `dequeue()`, `enqueue(4)`, `dequeue()`, `dequeue()`.
- R-6.8** Suppose an initially empty queue *Q* has executed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which raised Empty errors that were caught and ignored. What is the current size of *Q*?
- R-6.9** Had the queue of the previous problem been an instance of `ArrayQueue` that used an initial array of capacity 30, and had its size never been greater than 30, what would be the final value of the `_front` instance variable?
- R-6.10** Consider what happens if the loop in the `ArrayQueue._resize` method at lines 53–55 of Code Fragment 6.7 had been implemented as:

```
for k in range(self._size):
    self._data[k] = old[k]                # rather than old[walk]
```

Give a clear explanation of what could go wrong.

- R-6.11** Give a simple adapter that implements our queue ADT while using a `collections.deque` instance for storage.
- R-6.12** What values are returned during the following sequence of deque ADT operations, on initially empty deque? `add_first(4)`, `add_last(8)`, `add_last(9)`, `add_first(5)`, `back()`, `delete_first()`, `delete_last()`, `add_last(7)`, `first()`, `last()`, `add_last(6)`, `delete_first()`, `delete_first()`.
- R-6.13** Suppose you have a deque D containing the numbers $(1, 2, 3, 4, 5, 6, 7, 8)$, in this order. Suppose further that you have an initially empty queue Q . Give a code fragment that uses only D and Q (and no other variables) and results in D storing the elements in the order $(1, 2, 3, 5, 4, 6, 7, 8)$.
- R-6.14** Repeat the previous problem using the deque D and an initially empty stack S .

Creativity

- C-6.15** Suppose Alice has picked three distinct integers and placed them into a stack S in random order. Write a short, straight-line piece of pseudo-code (with no loops or recursion) that uses only one comparison and only one variable x , yet that results in variable x storing the largest of Alice's three integers with probability $2/3$. Argue why your method is correct.
- C-6.16** Modify the `ArrayStack` implementation so that the stack's capacity is limited to `maxlen` elements, where `maxlen` is an optional parameter to the constructor (that defaults to `None`). If `push` is called when the stack is at full capacity, throw a `Full` exception (defined similarly to `Empty`).
- C-6.17** In the previous exercise, we assume that the underlying list is initially empty. Redo that exercise, this time preallocating an underlying list with length equal to the stack's maximum capacity.
- C-6.18** Show how to use the transfer function, described in Exercise R-6.3, and two temporary stacks, to replace the contents of a given stack S with those same elements, but in reversed order.
- C-6.19** In Code Fragment 6.5 we assume that opening tags in HTML have form `<name>`, as with `<li>`. More generally, HTML allows optional attributes to be expressed as part of an opening tag. The general form used is `<name attribute1="value1" attribute2="value2">`; for example, a table can be given a border and additional padding by using an opening tag of `<table border="3" cellpadding="5">`. Modify Code Fragment 6.5 so that it can properly match tags, even when an opening tag may include one or more such attributes.
- C-6.20** Describe a nonrecursive algorithm for enumerating all permutations of the numbers $\{1, 2, \dots, n\}$ using an explicit stack.

3.3 Asymptotic Analysis

In algorithm analysis, we focus on the growth rate of the running time as a function of the input size n , taking a “big-picture” approach. For example, it is often enough just to know that the running time of an algorithm *grows proportionally to n* .

We analyze algorithms using a mathematical notation for functions that disregards constant factors. Namely, we characterize the running times of algorithms by using functions that map the size of the input, n , to values that correspond to the main factor that determines the growth rate in terms of n . This approach reflects that each basic step in a pseudo-code description or a high-level language implementation may correspond to a small number of primitive operations. Thus, we can perform an analysis of an algorithm by estimating the number of primitive operations executed up to a constant factor, rather than getting bogged down in language-specific or hardware-specific analysis of the exact number of operations that execute on the computer.

As a tangible example, we revisit the goal of finding the largest element of a Python list; we first used this example when introducing for loops on page 21 of Section 1.4.2. Code Fragment 3.1 presents a function named `find_max` for this task.

```

1 def find_max(data):
2     """Return the maximum element from a nonempty Python list."""
3     biggest = data[0]           # The initial value to beat
4     for val in data:           # For each value:
5         if val > biggest       # if it is greater than the best so far,
6             biggest = val      # we have found a new best (so far)
7     return biggest             # When loop ends, biggest is the max

```

Code Fragment 3.1: A function that returns the maximum value of a Python list.

This is a classic example of an algorithm with a running time that grows proportional to n , as the loop executes once for each data element, with some fixed number of primitive operations executing for each pass. In the remainder of this section, we provide a framework to formalize this claim.

3.3.1 The “Big-Oh” Notation

Let $f(n)$ and $g(n)$ be functions mapping positive integers to positive real numbers. We say that $f(n)$ is $O(g(n))$ if there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

$$f(n) \leq cg(n), \quad \text{for } n \geq n_0.$$

This definition is often referred to as the “big-Oh” notation, for it is sometimes pronounced as “ $f(n)$ is *big-Oh* of $g(n)$.” Figure 3.5 illustrates the general definition.

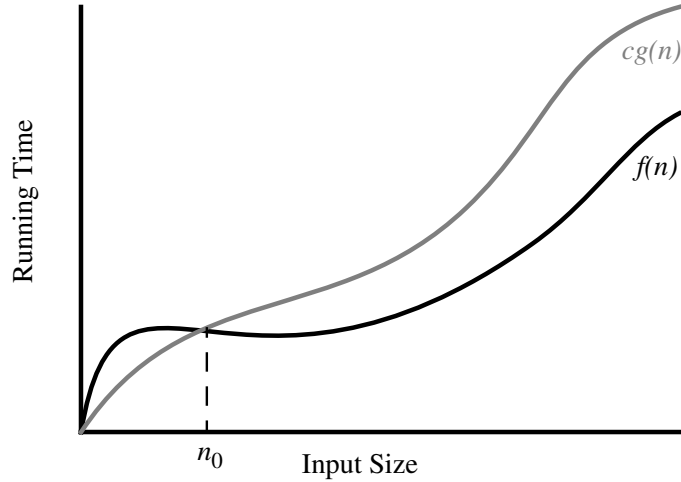


Figure 3.5: Illustrating the “big-Oh” notation. The function $f(n)$ is $O(g(n))$, since $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

Example 3.6: The function $8n + 5$ is $O(n)$.

Justification: By the big-Oh definition, we need to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $8n + 5 \leq cn$ for every integer $n \geq n_0$. It is easy to see that a possible choice is $c = 9$ and $n_0 = 5$. Indeed, this is one of infinitely many choices available because there is a trade-off between c and n_0 . For example, we could rely on constants $c = 13$ and $n_0 = 1$. ■

The big-Oh notation allows us to say that a function $f(n)$ is “less than or equal to” another function $g(n)$ up to a constant factor and in the *asymptotic* sense as n grows toward infinity. This ability comes from the fact that the definition uses “ $\leq$ ” to compare $f(n)$ to a $g(n)$ times a constant, c , for the asymptotic cases when $n \geq n_0$. However, it is considered poor taste to say “ $f(n) \leq O(g(n))$,” since the big-Oh already denotes the “less-than-or-equal-to” concept. Likewise, although common, it is not fully correct to say “ $f(n) = O(g(n))$,” with the usual understanding of the “ $=$ ” relation, because there is no way to make sense of the symmetric statement, “ $O(g(n)) = f(n)$.” It is best to say,

“ $f(n)$ is $O(g(n))$.”

Alternatively, we can say “ $f(n)$ is *order of* $g(n)$.” For the more mathematically inclined, it is also correct to say, “ $f(n) \in O(g(n))$,” for the big-Oh notation, technically speaking, denotes a whole collection of functions. In this book, we will stick to presenting big-Oh statements as “ $f(n)$ is $O(g(n))$.” Even with this interpretation, there is considerable freedom in how we can use arithmetic operations with the big-Oh notation, and with this freedom comes a certain amount of responsibility.

Characterizing Running Times Using the Big-Oh Notation

The big-Oh notation is used widely to characterize running times and space bounds in terms of some parameter n , which varies from problem to problem, but is always defined as a chosen measure of the “size” of the problem. For example, if we are interested in finding the largest element in a sequence, as with the `find_max` algorithm, we should let n denote the number of elements in that collection. Using the big-Oh notation, we can write the following mathematically precise statement on the running time of algorithm `find_max` (Code Fragment 3.1) for *any* computer.

Proposition 3.7: *The algorithm, `find_max`, for computing the maximum element of a list of n numbers, runs in $O(n)$ time.*

Justification: The initialization before the loop begins requires only a constant number of primitive operations. Each iteration of the loop also requires only a constant number of primitive operations, and the loop executes n times. Therefore, we account for the number of primitive operations being $c' + c'' \cdot n$ for appropriate constants c' and c'' that reflect, respectively, the work performed during initialization and the loop body. Because each primitive operation runs in constant time, we have that the running time of algorithm `find_max` on an input of size n is at most a constant times n ; that is, we conclude that the running time of algorithm `find_max` is $O(n)$. ■

Some Properties of the Big-Oh Notation

The big-Oh notation allows us to ignore constant factors and lower-order terms and focus on the main components of a function that affect its growth.

Example 3.8: $5n^4 + 3n^3 + 2n^2 + 4n + 1$ is $O(n^4)$.

Justification: Note that $5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq (5 + 3 + 2 + 4 + 1)n^4 = cn^4$, for $c = 15$, when $n \geq n_0 = 1$. ■

In fact, we can characterize the growth rate of any polynomial function.

Proposition 3.9: *If $f(n)$ is a polynomial of degree d , that is,*

$$f(n) = a_0 + a_1n + \cdots + a_dn^d,$$

and $a_d > 0$, then $f(n)$ is $O(n^d)$.

Justification: Note that, for $n \geq 1$, we have $1 \leq n \leq n^2 \leq \cdots \leq n^d$; hence,

$$a_0 + a_1n + a_2n^2 + \cdots + a_dn^d \leq (|a_0| + |a_1| + |a_2| + \cdots + |a_d|)n^d.$$

We show that $f(n)$ is $O(n^d)$ by defining $c = |a_0| + |a_1| + \cdots + |a_d|$ and $n_0 = 1$. ■

Thus, the highest-degree term in a polynomial is the term that determines the asymptotic growth rate of that polynomial. We consider some additional properties of the big-Oh notation in the exercises. Let us consider some further examples here, focusing on combinations of the seven fundamental functions used in algorithm design. We rely on the mathematical fact that $\log n \leq n$ for $n \geq 1$.

Example 3.10: $5n^2 + 3n \log n + 2n + 5$ is $O(n^2)$.

Justification: $5n^2 + 3n \log n + 2n + 5 \leq (5 + 3 + 2 + 5)n^2 = cn^2$, for $c = 15$, when $n \geq n_0 = 1$. ■

Example 3.11: $20n^3 + 10n \log n + 5$ is $O(n^3)$.

Justification: $20n^3 + 10n \log n + 5 \leq 35n^3$, for $n \geq 1$. ■

Example 3.12: $3 \log n + 2$ is $O(\log n)$.

Justification: $3 \log n + 2 \leq 5 \log n$, for $n \geq 2$. Note that $\log n$ is zero for $n = 1$. That is why we use $n \geq n_0 = 2$ in this case. ■

Example 3.13: 2^{n+2} is $O(2^n)$.

Justification: $2^{n+2} = 2^n \cdot 2^2 = 4 \cdot 2^n$; hence, we can take $c = 4$ and $n_0 = 1$ in this case. ■

Example 3.14: $2n + 100 \log n$ is $O(n)$.

Justification: $2n + 100 \log n \leq 102n$, for $n \geq n_0 = 1$; hence, we can take $c = 102$ in this case. ■

Characterizing Functions in Simplest Terms

In general, we should use the big-Oh notation to characterize a function as closely as possible. While it is true that the function $f(n) = 4n^3 + 3n^2$ is $O(n^5)$ or even $O(n^4)$, it is more accurate to say that $f(n)$ is $O(n^3)$. Consider, by way of analogy, a scenario where a hungry traveler driving along a long country road happens upon a local farmer walking home from a market. If the traveler asks the farmer how much longer he must drive before he can find some food, it may be truthful for the farmer to say, “certainly no longer than 12 hours,” but it is much more accurate (and helpful) for him to say, “you can find a market just a few minutes drive up this road.” Thus, even with the big-Oh notation, we should strive as much as possible to tell the whole truth.

It is also considered poor taste to include constant factors and lower-order terms in the big-Oh notation. For example, it is not fashionable to say that the function $2n^2$ is $O(4n^2 + 6n \log n)$, although this is completely correct. We should strive instead to describe the function in the big-Oh in *simplest terms*.

The seven functions listed in Section 3.2 are the most common functions used in conjunction with the big-Oh notation to characterize the running times and space usage of algorithms. Indeed, we typically use the names of these functions to refer to the running times of the algorithms they characterize. So, for example, we would say that an algorithm that runs in worst-case time $4n^2 + n \log n$ is a **quadratic-time** algorithm, since it runs in $O(n^2)$ time. Likewise, an algorithm running in time at most $5n + 20 \log n + 4$ would be called a **linear-time** algorithm.

Big-Omega

Just as the big-Oh notation provides an asymptotic way of saying that a function is “less than or equal to” another function, the following notations provide an asymptotic way of saying that a function grows at a rate that is “greater than or equal to” that of another.

Let $f(n)$ and $g(n)$ be functions mapping positive integers to positive real numbers. We say that $f(n)$ is $\Omega(g(n))$, pronounced “ $f(n)$ is big-Omega of $g(n)$,” if $g(n)$ is $O(f(n))$, that is, there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

$$f(n) \geq cg(n), \quad \text{for } n \geq n_0.$$

This definition allows us to say asymptotically that one function is greater than or equal to another, up to a constant factor.

Example 3.15: $3n \log n - 2n$ is $\Omega(n \log n)$.

Justification: $3n \log n - 2n = n \log n + 2n(\log n - 1) \geq n \log n$ for $n \geq 2$; hence, we can take $c = 1$ and $n_0 = 2$ in this case. ■

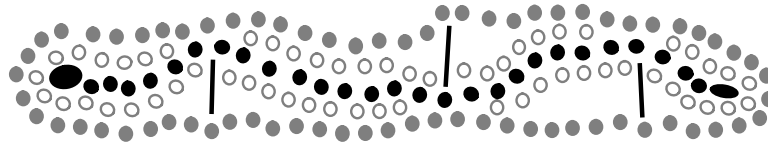
Big-Theta

In addition, there is a notation that allows us to say that two functions grow at the same rate, up to constant factors. We say that $f(n)$ is $\Theta(g(n))$, pronounced “ $f(n)$ is big-Theta of $g(n)$,” if $f(n)$ is $O(g(n))$ and $f(n)$ is $\Omega(g(n))$, that is, there are real constants $c' > 0$ and $c'' > 0$, and an integer constant $n_0 \geq 1$ such that

$$c'g(n) \leq f(n) \leq c''g(n), \quad \text{for } n \geq n_0.$$

Example 3.16: $3n \log n + 4n + 5 \log n$ is $\Theta(n \log n)$.

Justification: $3n \log n \leq 3n \log n + 4n + 5 \log n \leq (3 + 4 + 5)n \log n$ for $n \geq 2$. ■



Contents

7.1	Singly Linked Lists	256
7.1.1	Implementing a Stack with a Singly Linked List	261
7.1.2	Implementing a Queue with a Singly Linked List	264
7.2	Circularly Linked Lists	266
7.2.1	Round-Robin Schedulers	267
7.2.2	Implementing a Queue with a Circularly Linked List	268
7.3	Doubly Linked Lists	270
7.3.1	Basic Implementation of a Doubly Linked List	273
7.3.2	Implementing a Deque with a Doubly Linked List	275
7.4	The Positional List ADT	277
7.4.1	The Positional List Abstract Data Type	279
7.4.2	Doubly Linked List Implementation	281
7.5	Sorting a Positional List	285
7.6	Case Study: Maintaining Access Frequencies	286
7.6.1	Using a Sorted List	286
7.6.2	Using a List with the Move-to-Front Heuristic	289
7.7	Link-Based vs. Array-Based Sequences	292
7.8	Exercises	294

In Chapter 5 we carefully examined Python’s array-based list class, and in Chapter 6 we demonstrated use of that class in implementing the classic stack, queue, and dequeue ADTs. Python’s list class is highly optimized, and often a great choice for storage. With that said, there are some notable disadvantages:

1. The length of a dynamic array might be longer than the actual number of elements that it stores.
2. Amortized bounds for operations may be unacceptable in real-time systems.
3. Insertions and deletions at interior positions of an array are expensive.

In this chapter, we introduce a data structure known as a **linked list**, which provides an alternative to an array-based sequence (such as a Python list). Both array-based sequences and linked lists keep elements in a certain order, but using a very different style. An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements. A linked list, in contrast, relies on a more distributed representation in which a lightweight object, known as a **node**, is allocated for each element. Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

We will demonstrate a trade-off of advantages and disadvantages when contrasting array-based sequences and linked lists. Elements of a linked list cannot be efficiently accessed by a numeric index k , and we cannot tell just by examining a node if it is the second, fifth, or twentieth node in the list. However, linked lists avoid the three disadvantages noted above for array-based sequences.

7.1 Singly Linked Lists

A **singly linked list**, in its simplest form, is a collection of **nodes** that collectively form a linear sequence. Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figures 7.1 and 7.2).

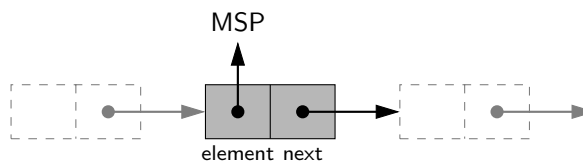


Figure 7.1: Example of a node instance that forms part of a singly linked list. The node’s element member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the next member references the subsequent node of the linked list (or None if there is no further node).

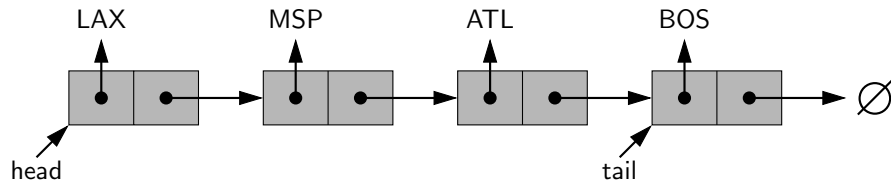


Figure 7.2: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named *head* that identifies the first node of the list, and in some applications another member named *tail* that identifies the last node of the list. The **None** object is denoted as $\emptyset$.

The first and last node of a linked list are known as the *head* and *tail* of the list, respectively. By starting at the head, and moving from one node to another by following each node’s next reference, we can reach the tail of the list. We can identify the tail as the node having None as its next reference. This process is commonly known as *traversing* the linked list. Because the next reference of a node can be viewed as a *link* or *pointer* to another node, the process of traversing a list is also known as *link hopping* or *pointer hopping*.

A linked list’s representation in memory relies on the collaboration of many objects. Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None). Another object represents the linked list as a whole. Minimally, the linked list instance must keep a reference to the head of the list. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list. However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal. In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the *size* of the list), to avoid the need to traverse the list to count the nodes.

For the remainder of this chapter, we continue to illustrate nodes as objects, and each node’s “next” reference as a pointer. However, for the sake of simplicity, we illustrate a node’s element embedded directly within the node structure, even though the element is, in fact, an independent object. For example, Figure 7.3 is a more compact illustration of the linked list from Figure 7.2.

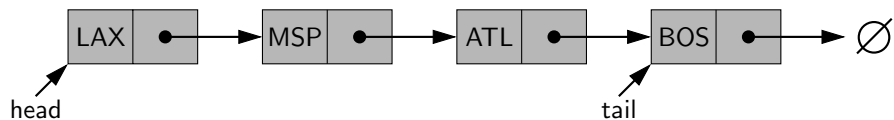


Figure 7.3: A compact illustration of a singly linked list, with elements embedded in the nodes (rather than more accurately drawn as references to external objects).

Inserting an Element at the Head of a Singly Linked List

An important property of a linked list is that it does not have a predetermined fixed size; it uses space proportionally to its current number of elements. When using a singly linked list, we can easily insert an element at the head of the list, as shown in Figure 7.4, and described with pseudo-code in Code Fragment 7.1. The main idea is that we create a new node, set its element to the new element, set its next link to refer to the current head, and then set the list's head to point to the new node.

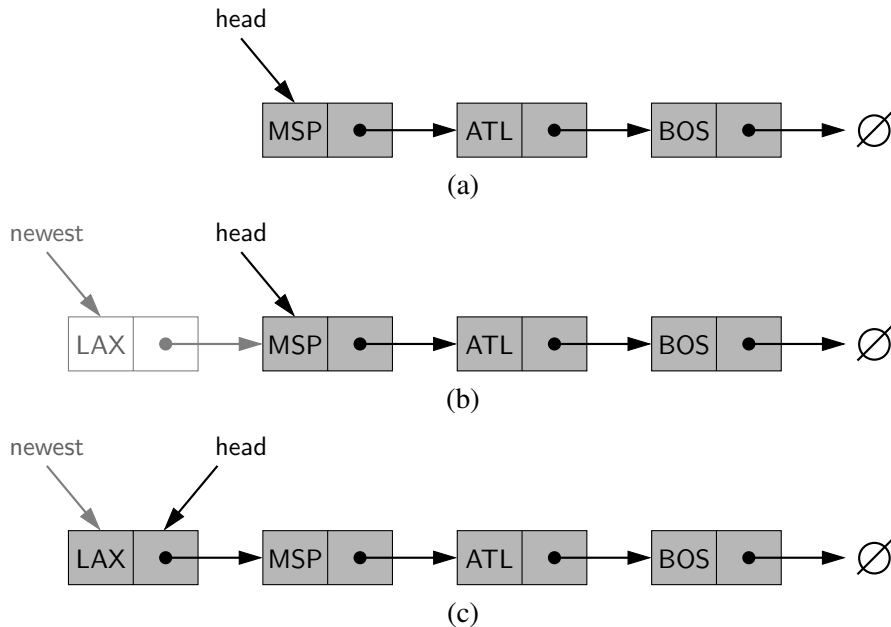


Figure 7.4: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the head reference.

Algorithm `add_first(L,e)`:

```

newest = Node(e)  {create new node instance storing reference to element e}
newest.next = L.head  {set new node's next to reference the old head node}
L.head = newest      {set variable head to reference the new node}
L.size = L.size + 1 {increment the node count}

```

Code Fragment 7.1: Inserting a new element at the beginning of a singly linked list *L*. Note that we set the next pointer of the new node *before* we reassign variable *L.head* to it. If the list were initially empty (i.e., *L.head* is *None*), then a natural consequence is that the new node has its next reference set to *None*.

Inserting an Element at the Tail of a Singly Linked List

We can also easily insert an element at the tail of the list, provided we keep a reference to the tail node, as shown in Figure 7.5. In this case, we create a new node, assign its next reference to None, set the next reference of the tail to point to this new node, and then update the tail reference itself to this new node. We give the details in Code Fragment 7.2.

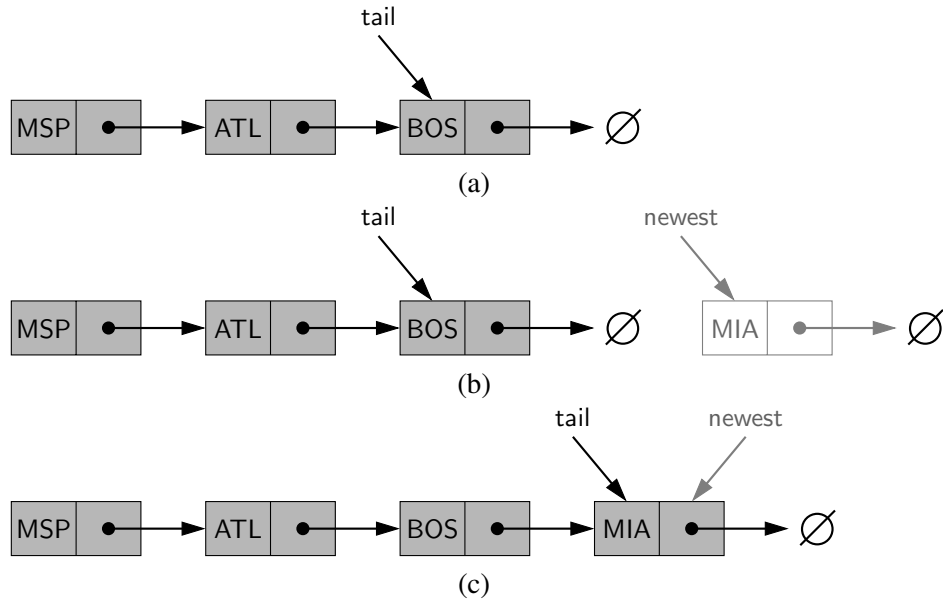


Figure 7.5: Insertion at the tail of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the tail reference. Note that we must set the next link of the tail in (b) before we assign the tail variable to point to the new node in (c).

Algorithm add_last(L, e):

```

newest = Node(e) {create new node instance storing reference to element e}
newest.next = None {set new node's next to reference the None object}
L.tail.next = newest {make old tail node point to new node}
L.tail = newest {set variable tail to reference the new node}
L.size = L.size + 1 {increment the node count}

```

Code Fragment 7.2: Inserting a new node at the end of a singly linked list. Note that we set the next pointer for the old tail node *before* we make variable tail point to the new node. This code would need to be adjusted for inserting onto an empty list, since there would not be an existing tail node.

Removing an Element from a Singly Linked List

Removing an element from the *head* of a singly linked list is essentially the reverse operation of inserting a new element at the head. This operation is illustrated in Figure 7.6 and given in detail in Code Fragment 7.3.

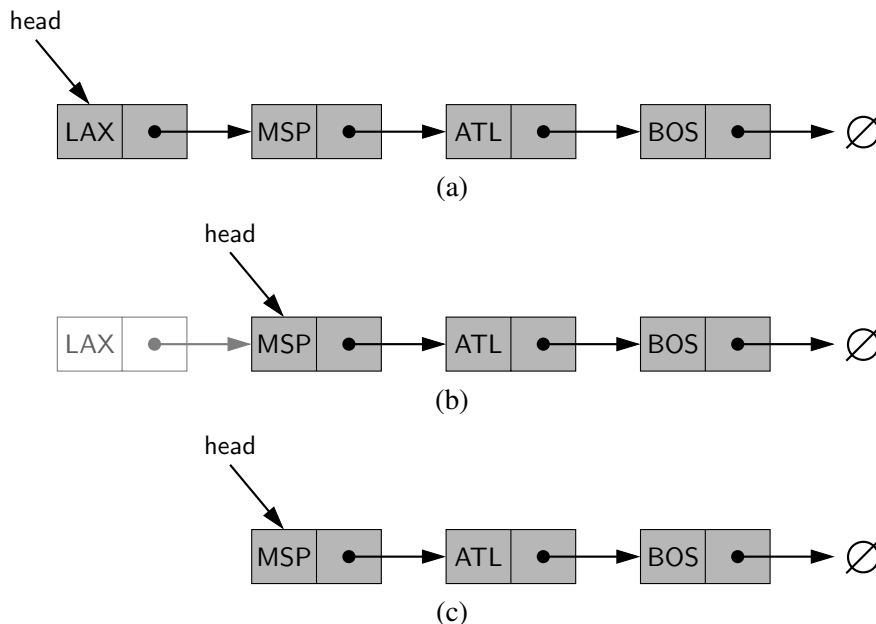


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after “linking out” the old head; (c) final configuration.

Algorithm `remove_first(L)`:

if `L.head` is `None` **then**

 Indicate an error: the list is empty.

`L.head = L.head.next` {make head point to next node (or None)}

`L.size = L.size - 1` {decrement the node count}

Code Fragment 7.3: Removing the node at the beginning of a singly linked list.

Unfortunately, we cannot easily delete the last node of a singly linked list. Even if we maintain a tail reference directly to the last node of the list, we must be able to access the node *before* the last node in order to remove the last node. But we cannot reach the node before the tail by following next links from the tail. The only way to access this node is to start from the head of the list and search all the way through the list. But such a sequence of link-hopping operations could take a long time. If we want to support such an operation efficiently, we will need to make our list *doubly linked* (as we do in Section 7.3).

7.1.1 Implementing a Stack with a Singly Linked List

In this section, we demonstrate use of a singly linked list by providing a complete Python implementation of the stack ADT (see Section 6.1). In designing such an implementation, we need to decide whether to model the top of the stack at the head or at the tail of the list. There is clearly a best choice here; we can efficiently insert and delete elements in constant time only at the head. Since all stack operations affect the top, we orient the top of the stack at the head of our list.

To represent individual nodes of the list, we develop a lightweight `_Node` class. This class will never be directly exposed to the user of our stack class, so we will formally define it as a nonpublic, nested class of our eventual `LinkedStack` class (see Section 2.5.1 for discussion of nested classes). The definition of the `_Node` class is shown in Code Fragment 7.4.

```
class _Node:
    """Lightweight, nonpublic class for storing a singly linked node."""
    __slots__ = '_element', '_next'           # streamline memory usage

    def __init__(self, element, next):        # initialize node's fields
        self._element = element              # reference to user's element
        self._next = next                    # reference to next node
```

Code Fragment 7.4: A lightweight `_Node` class for a singly linked list.

A node has only two instance variables: `_element` and `_next`. We intentionally define `__slots__` to streamline the memory usage (see page 99 of Section 2.5.1 for discussion), because there may potentially be many node instances in a single list. The constructor of the `_Node` class is designed for our convenience, allowing us to specify initial values for both fields of a newly created node.

A complete implementation of our `LinkedStack` class is given in Code Fragments 7.5 and 7.6. Each stack instance maintains two variables. The `_head` member is a reference to the node at the head of the list (or `None`, if the stack is empty). We keep track of the current number of elements with the `_size` instance variable, for otherwise we would be forced to traverse the entire list to count the number of elements when reporting the size of the stack.

The implementation of `push` essentially mirrors the pseudo-code for insertion at the head of a singly linked list as outlined in Code Fragment 7.1. When we push a new element `e` onto the stack, we accomplish the necessary changes to the linked structure by invoking the constructor of the `_Node` class as follows:

```
self._head = self._Node(e, self._head)    # create and link a new node
```

Note that the `_next` field of the new node is set to the *existing* top node, and then `self._head` is reassigned to the new node.

```

1  class LinkedStack:
2      """LIFO Stack implementation using a singly linked list for storage."""
3
4      #----- nested _Node class -----
5      class _Node:
6          """Lightweight, nonpublic class for storing a singly linked node."""
7          __slots__ = '_element', '_next'      # streamline memory usage
8
9          def __init__(self, element, next):    # initialize node's fields
10             self._element = element          # reference to user's element
11             self._next = next                 # reference to next node
12
13     #----- stack methods -----
14     def __init__(self):
15         """Create an empty stack."""
16         self._head = None                    # reference to the head node
17         self._size = 0                       # number of stack elements
18
19     def __len__(self):
20         """Return the number of elements in the stack."""
21         return self._size
22
23     def is_empty(self):
24         """Return True if the stack is empty."""
25         return self._size == 0
26
27     def push(self, e):
28         """Add element e to the top of the stack."""
29         self._head = self._Node(e, self._head)    # create and link a new node
30         self._size += 1
31
32     def top(self):
33         """Return (but do not remove) the element at the top of the stack.
34
35         Raise Empty exception if the stack is empty.
36         """
37         if self.is_empty():
38             raise Empty('Stack is empty')
39         return self._head._element                # top of stack is at head of list

```

Code Fragment 7.5: Implementation of a stack ADT using a singly linked list (continued in Code Fragment 7.6).

```

40  def pop(self):
41      """ Remove and return the element from the top of the stack (i.e., LIFO).
42
43      Raise Empty exception if the stack is empty.
44      """
45      if self.is_empty():
46          raise Empty('Stack is empty')
47      answer = self._head._element
48      self._head = self._head._next          # bypass the former top node
49      self._size -= 1
50      return answer

```

Code Fragment 7.6: Implementation of a stack ADT using a singly linked list (continued from Code Fragment 7.5).

When implementing the `top` method, the goal is to return the *element* that is at the top of the stack. When the stack is empty, we raise an `Empty` exception, as originally defined in Code Fragment 6.1 of Chapter 6. When the stack is nonempty, `self._head` is a reference to the first *node* of the linked list. The top element can be identified as `self._head._element`.

Our implementation of `pop` essentially mirrors the pseudo-code given in Code Fragment 7.3, except that we maintain a local reference to the element that is stored at the node that is being removed, and we return that element to the caller of `pop`.

The analysis of our `LinkedList` operations is given in Table 7.1. We see that all of the methods complete in *worst-case* constant time. This is in contrast to the amortized bounds for the `ArrayStack` that were given in Table 6.2.

Operation	Running Time
<code>S.push(e)</code>	$O(1)$
<code>S.pop()</code>	$O(1)$
<code>S.top()</code>	$O(1)$
<code>len(S)</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$

Table 7.1: Performance of our `LinkedList` implementation. All bounds are worst-case and our space usage is $O(n)$, where n is the current number of elements in the stack.

7.1.2 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can use a singly linked list to implement the queue ADT while supporting worst-case $O(1)$ -time for all operations. Because we need to perform operations on both ends of the queue, we will explicitly maintain both a `_head` reference and a `_tail` reference as instance variables for each queue. The natural orientation for a queue is to align the front of the queue with the head of the list, and the back of the queue with the tail of the list, because we must be able to enqueue elements at the back, and dequeue them from the front. (Recall from the introduction of Section 7.1 that we are unable to efficiently remove elements from the tail of a singly linked list.) Our implementation of a `LinkedList` class is given in Code Fragments 7.7 and 7.8.

```

1  class LinkedList:
2      """FIFO queue implementation using a singly linked list for storage."""
3
4      class _Node:
5          """Lightweight, nonpublic class for storing a singly linked node."""
6          (omitted here; identical to that of LinkedList._Node)
7
8      def __init__(self):
9          """Create an empty queue."""
10         self._head = None
11         self._tail = None
12         self._size = 0                # number of queue elements
13
14     def __len__(self):
15         """Return the number of elements in the queue."""
16         return self._size
17
18     def is_empty(self):
19         """Return True if the queue is empty."""
20         return self._size == 0
21
22     def first(self):
23         """Return (but do not remove) the element at the front of the queue."""
24         if self.is_empty():
25             raise Empty('Queue is empty')
26         return self._head._element    # front aligned with head of list

```

Code Fragment 7.7: Implementation of a queue ADT using a singly linked list (continued in Code Fragment 7.8).

```

27  def dequeue(self):
28      """ Remove and return the first element of the queue (i.e., FIFO).
29
30      Raise Empty exception if the queue is empty.
31      """
32      if self.is_empty():
33          raise Empty('Queue is empty')
34      answer = self._head._element
35      self._head = self._head._next
36      self._size -= 1
37      if self.is_empty():           # special case as queue is empty
38          self._tail = None         # removed head had been the tail
39      return answer
40
41  def enqueue(self, e):
42      """ Add an element to the back of queue. """
43      newest = self._Node(e, None)   # node will be new tail node
44      if self.is_empty():
45          self._head = newest        # special case: previously empty
46      else:
47          self._tail._next = newest
48      self._tail = newest            # update reference to tail node
49      self._size += 1

```

Code Fragment 7.8: Implementation of a queue ADT using a singly linked list (continued from Code Fragment 7.7).

Many aspects of our implementation are similar to that of the `LinkedStack` class, such as the definition of the nested `_Node` class. Our implementation of `dequeue` for `LinkedQueue` is similar to that of `pop` for `LinkedStack`, as both remove the head of the linked list. However, there is a subtle difference because our queue must accurately maintain the `_tail` reference (no such variable was maintained for our stack). In general, an operation at the head has no effect on the tail, but when `dequeue` is invoked on a queue with one element, we are simultaneously removing the tail of the list. We therefore set `self._tail` to `None` for consistency.

There is a similar complication in our implementation of `enqueue`. The newest node always becomes the new tail. Yet a distinction is made depending on whether that new node is the only node in the list. In that case, it also becomes the new head; otherwise the new node must be linked immediately after the existing tail node.

In terms of performance, the `LinkedQueue` is similar to the `LinkedStack` in that all operations run in worst-case constant time, and the space usage is linear in the current number of elements.

7.2 Circularly Linked Lists

In Section 6.2.2, we introduced the notion of a “circular” array and demonstrated its use in implementing the queue ADT. In reality, the notion of a circular array was artificial, in that there was nothing about the representation of the array itself that was circular in structure. It was our use of modular arithmetic when “advancing” an index from the last slot to the first slot that provided such an abstraction.

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown in Figure 7.7. We call such a structure a *circularly linked list*.

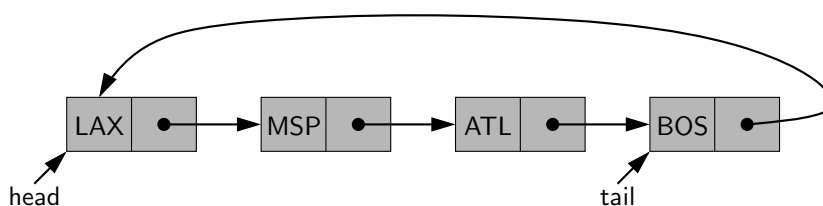


Figure 7.7: Example of a singly linked list with circular structure.

A circularly linked list provides a more general model than a standard linked list for data sets that are cyclic, that is, which do not have any particular notion of a beginning and end. Figure 7.8 provides a more symmetric illustration of the same circular list structure as Figure 7.7.

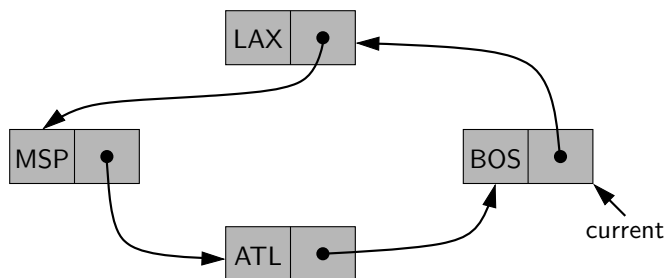


Figure 7.8: Example of a circular linked list, with *current* denoting a reference to a select node.

A circular view similar to Figure 7.8 could be used, for example, to describe the order of train stops in the Chicago loop, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list. We use the identifier *current* to describe such a designated node. By setting `current = current.next`, we can effectively advance through the nodes of the list.

7.2.1 Round-Robin Schedulers

To motivate the use of a circularly linked list, we consider a **round-robin** scheduler, which iterates through a collection of elements in a circular fashion and “services” each element by performing a given action on it. Such a scheduler is used, for example, to fairly allocate a resource that must be shared by a collection of clients. For instance, round-robin scheduling is often used to allocate slices of CPU time to various applications running concurrently on a computer.

A round-robin scheduler could be implemented with the general queue ADT, by repeatedly performing the following steps on queue Q (see Figure 7.9):

1. $e = Q.dequeue()$
2. Service element e
3. $Q.enqueue(e)$

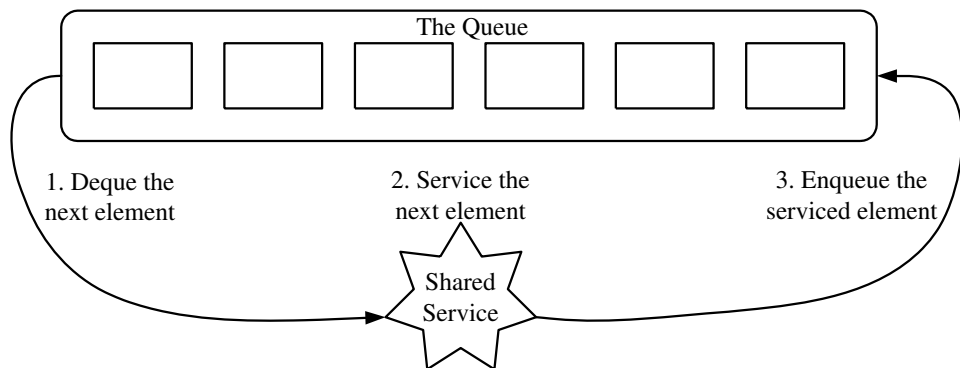


Figure 7.9: The three iterative steps for round-robin scheduling using a queue.

If we use of the `LinkedList` class of Section 7.1.2 for such an application, there is unnecessary effort in the combination of a dequeue operation followed soon after by an enqueue of the same element. One node is removed from the list, with appropriate adjustments to the head of the list and the size decremented, and then a new node is created to reinsert at the tail of the list and the size is incremented.

If using a circularly linked list, the effective transfer of an item from the “head” of the list to the “tail” of the list can be accomplished by advancing a reference that marks the boundary of the queue. We will next provide an implementation of a `CircularQueue` class that supports the entire queue ADT, together with an additional method, `rotate()`, that moves the first element of the queue to the back. (A similar method is supported by the `deque` class of Python’s `collections` module; see Table 6.4.) With this operation, a round-robin schedule can more efficiently be implemented by repeatedly performing the following steps:

1. Service element $Q.front()$
2. $Q.rotate()$

7.2.2 Implementing a Queue with a Circularly Linked List

To implement the queue ADT using a circularly linked list, we rely on the intuition of Figure 7.7, in which the queue has a head and a tail, but with the next reference of the tail linked to the head. Given such a model, there is no need for us to explicitly store references to both the head and the tail; as long as we keep a reference to the tail, we can always find the head by following the tail's next reference.

Code Fragments 7.9 and 7.10 provide an implementation of a `CircularQueue` class based on this model. The only two instance variables are `_tail`, which is a reference to the tail node (or `None` when empty), and `_size`, which is the current number of elements in the queue. When an operation involves the front of the queue, we recognize `self._tail._next` as the head of the queue. When `enqueue` is called, a new node is placed just after the tail but before the current head, and then the new node becomes the tail.

In addition to the traditional queue operations, the `CircularQueue` class supports a `rotate` method that more efficiently enacts the combination of removing the front element and reinserting it at the back of the queue. With the circular representation, we simply set `self._tail = self._tail._next` to make the old head become the new tail (with the node after the old head becoming the new head).

```

1  class CircularQueue:
2      """Queue implementation using circularly linked list for storage."""
3
4      class _Node:
5          """Lightweight, nonpublic class for storing a singly linked node."""
6          (omitted here; identical to that of LinkedStack._Node)
7
8      def __init__(self):
9          """Create an empty queue."""
10         self._tail = None           # will represent tail of queue
11         self._size = 0             # number of queue elements
12
13     def __len__(self):
14         """Return the number of elements in the queue."""
15         return self._size
16
17     def is_empty(self):
18         """Return True if the queue is empty."""
19         return self._size == 0

```

Code Fragment 7.9: Implementation of a `CircularQueue` class, using a circularly linked list as storage (continued in Code Fragment 7.10).

```

20  def first(self):
21      """Return (but do not remove) the element at the front of the queue.
22
23      Raise Empty exception if the queue is empty.
24      """
25      if self.is_empty():
26          raise Empty('Queue is empty')
27      head = self._tail._next
28      return head._element
29
30  def dequeue(self):
31      """Remove and return the first element of the queue (i.e., FIFO).
32
33      Raise Empty exception if the queue is empty.
34      """
35      if self.is_empty():
36          raise Empty('Queue is empty')
37      oldhead = self._tail._next
38      if self._size == 1:                # removing only element
39          self._tail = None             # queue becomes empty
40      else:
41          self._tail._next = oldhead._next    # bypass the old head
42      self._size -= 1
43      return oldhead._element
44
45  def enqueue(self, e):
46      """Add an element to the back of queue."""
47      newest = self._Node(e, None)         # node will be new tail node
48      if self.is_empty():
49          newest._next = newest             # initialize circularly
50      else:
51          newest._next = self._tail._next   # new node points to head
52          self._tail._next = newest         # old tail points to new node
53      self._tail = newest                  # new node becomes the tail
54      self._size += 1
55
56  def rotate(self):
57      """Rotate front element to the back of the queue."""
58      if self._size > 0:
59          self._tail = self._tail._next    # old head becomes new tail

```

Code Fragment 7.10: Implementation of a CircularQueue class, using a circularly linked list as storage (continued from Code Fragment 7.9).

7.3 Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In the opening of Section 7.1, we emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately *precedes* the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a **doubly linked list**. These lists allow a greater variety of $O(1)$ -time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term “next” for the reference to the node that follows another, and we introduce the term “prev” for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a **header** node at the beginning of the list, and a **trailer** node at the end of the list. These “dummy” nodes are known as **sentinels** (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 7.10.

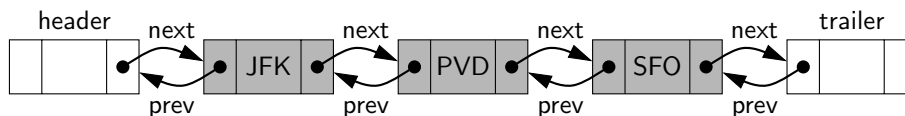


Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header; the remaining fields of the sentinels are irrelevant (presumably None, in Python). For a nonempty list, the header’s next will refer to a node containing the first real element of a sequence, just as the trailer’s prev references the node containing the last element of a sequence.

Advantage of Using Sentinels

Although we could implement a doubly linked list without sentinel nodes (as we did with our singly linked list in Section 7.1), the slight extra space devoted to the sentinels greatly simplifies the logic of our operations. Most notably, the header and trailer nodes never change—only the nodes between them change. Furthermore, we can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

For contrast, look back at our `LinkedList` implementation from Section 7.1.2. Its `enqueue` method, given in Code Fragment 7.8, adds a new node to the end of the list. However, its implementation required a conditional to manage the special case of inserting into an empty list. In the general case, the new node was linked after the existing tail. But when adding to an empty list, there is no existing tail; instead it is necessary to reassign `self._head` to reference the new node. The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting and Deleting with a Doubly Linked List

Every insertion into our doubly linked list representation will take place between a pair of existing nodes, as diagrammed in Figure 7.11. For example, when a new element is inserted at the front of the sequence, we will simply add the new node *between* the header and the node that is currently after the header. (See Figure 7.12.)

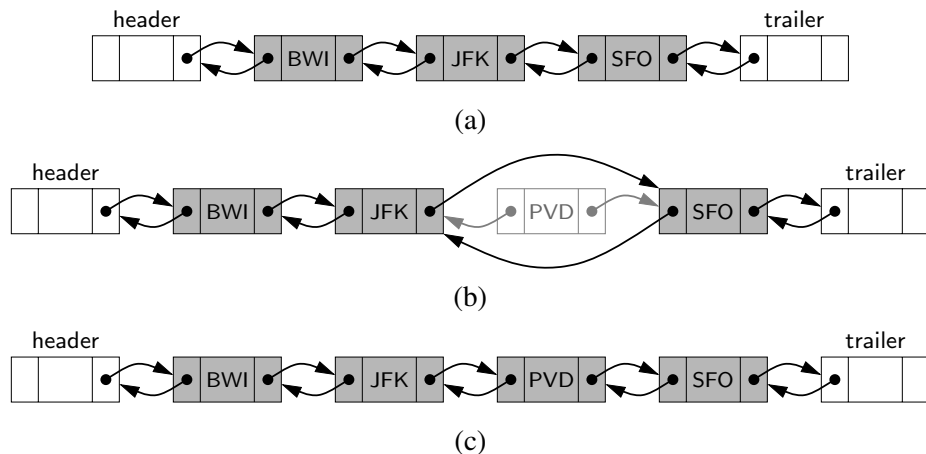


Figure 7.11: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

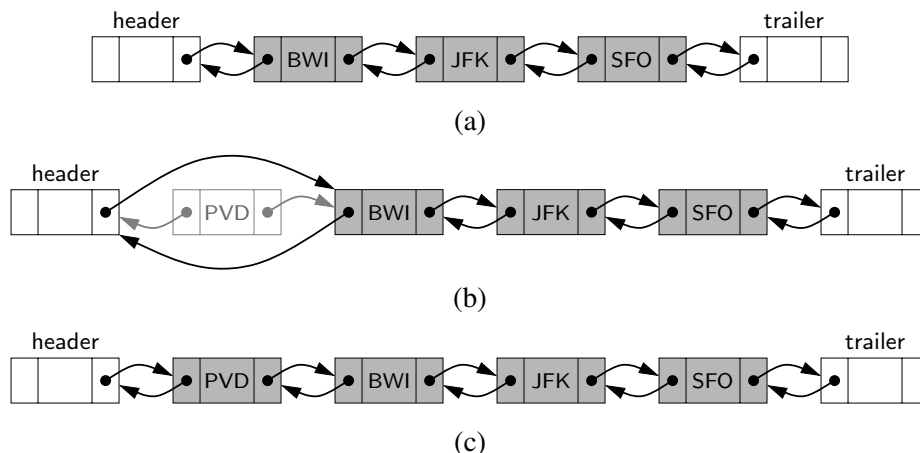


Figure 7.12: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

The deletion of a node, portrayed in Figure 7.13, proceeds in the opposite fashion of an insertion. The two neighbors of the node to be deleted are linked directly to each other, thereby bypassing the original node. As a result, that node will no longer be considered part of the list and it can be reclaimed by the system. Because of our use of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.

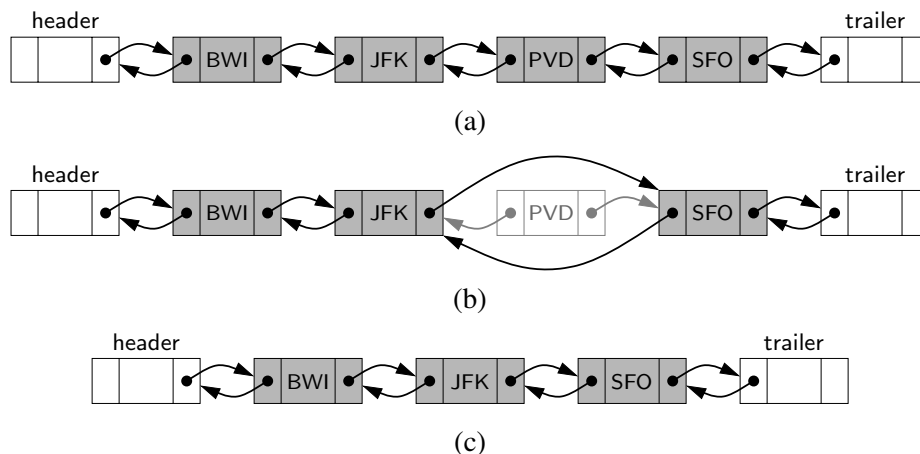


Figure 7.13: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

.Searching - Linear Search and Binary Search

There are two types of searching -

- Linear Search
- Binary Search

Both techniques are widely used to search an element in the given list.

Linear Search

What is a Linear Search?

Linear search is a method of finding elements within a list. It is also called a sequential search. It is the simplest searching algorithm because it searches the desired element in a sequential manner.

It compares each and every element with the value that we are searching for. If both are matched, the element is found, and the algorithm returns the key's index position.

Concept of Linear Search

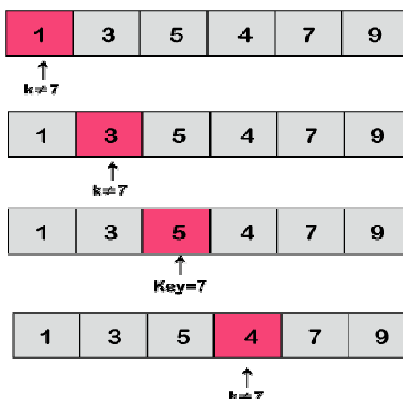
Let's understand the following steps to find the element key = 7 in the given list.

Step - 1: Start the search from the first element and Check key = 7 with each element of list x.

1	3	5	4	7	9
---	---	---	---	---	---

List to be Searched for

Step - 2: If element is found, return the index position of the key.



Step - 3: If element is not found, return element is not present.

1	3	5	4	7	9
---	---	---	---	---	---

Key=7

Linear Search Algorithm

There is list of n elements and key value to be searched.

Below is the linear search algorithm.

1. LinearSearch(list, key)
2. for each item in the list
3. if **item** == value
4. return its index position
5. return -1

Python Program

Let's understand the following Python implementation of the linear search algorithm.

Program

1. def linear_Search(list1, n, key):
- 2.
3. # Searching list1 sequentially
4. for i in range(0, n):
5. if (list1[i] == key):
6. return i
7. return -1
- 8.
- 9.
10. **list1** = [1 ,3, 5, 4, 7, 9]
11. **key** = 7
- 12.
13. **n** = len(list1)
14. **res** = linear_Search(list1, n, key)
15. if(**res** == -1):
16. print("Element not found")

17. else:

18. print("Element found at index: ", res)

Output:

```
Element found at index: 4
```

Explanation:

In the above code, we have created a function **linear_Search()**, which takes three arguments - list1, length of the list, and number to search. We defined for loop and iterate each element and compare to the key value. If element is found, return the index else return -1 which means element is not present in the list.

Linear Search Complexity

Time complexity of linear search algorithm -

- Base Case - $O(1)$
- Average Case - $O(n)$
- Worst Case - $O(n)$

Linear search algorithm is suitable for smaller list (<100) because it check every element to get the desired number. Suppose there are 10,000 element list and desired element is available at the last position, this will consume much time by comparing with each element of the list.

To get the fast result, we can use the binary search algorithm.

Binary Search:

A binary search is an algorithm to find a particular element in the list. Suppose we have a list of thousand elements, and we need to get an index position of a particular element. We can find the element's index position very fast using the binary search algorithm.

There are many searching algorithms but the binary search is most popular among them.

The elements in the list must be sorted to apply the binary search algorithm. If elements are not sorted then sort them first.

Let's understand the concept of binary search.

Concept of Binary Search

The divide and conquer approach technique is followed by the recursive method. In this method, a function is called itself again and again until it found an element in the list.

A set of statements is repeated multiple times to find an element's index position in the iterative method. The **while** loop is used for accomplish this task.

Binary search is more effective than the linear search because we don't need to search each list index. The list must be sorted to achieve the binary search algorithm.

Let's have a step by step implementation of binary search.

We have a sorted list of elements, and we are looking for the index position of 45.

[12, 24, 32, 39, 45, 50, 54]

So, we are setting two pointers in our list. One pointer is used to denote the smaller value called **low** and the second pointer is used to denote the highest value called **high**.

Next, we calculate the value of the **middle** element in the array.

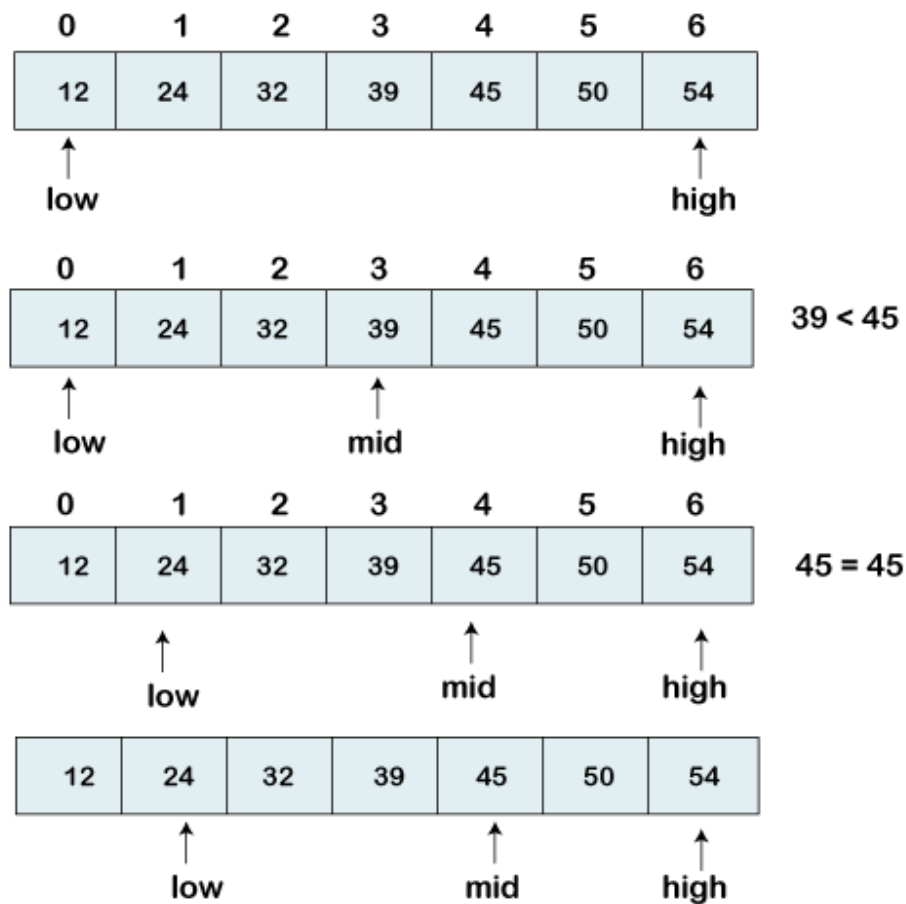
1. **mid** = (low+high)/2
2. Here, the low is 0 and the high is 7.
3. **mid** = (0+7)/2
4. **mid** = 3 (Integer)

Now, we will compare the searched element to the mid index value. In this case, **32** is not equal to **45**. So we need to do further comparison to find the element.

If the number we are searching equal to the mid. Then return **mid** otherwise move to the further comparison.

The number to be search is greater than the **middle** number, we compare the **n** with the middle element of the elements on the right side of **mid** and set low to **low = mid + 1**.

Otherwise, compare the **n** with the **middle element** of the elements on the left side of **mid** and set **high** to **high = mid - 1**.



Repeat until the number that we are searching for is found.

Implement a Binary Search in Python

First, we implement a binary search with the iterative method. We will repeat a set of statements and iterate every item of the list. We will find the middle value until the search is complete.

Let's understand the following program of the iterative method.

Python Implementation

1. # Iterative Binary Search Function method Python Implementation
2. # It returns index of n in given list1 if present,
3. # else returns -1
4. def binary_search(list1, n):
5. low = 0
6. high = len(list1) - 1
7. mid = 0

```
8.
9.  while low <= high:
10.     # for get integer result
11.     mid = (high + low) // 2
12.
13.     # Check if n is present at mid
14.     if list1[mid] < n:
15.         low = mid + 1
16.
17.     # If n is greater, compare to the right of mid
18.     elif list1[mid] > n:
19.         high = mid - 1
20.
21.     # If n is smaller, compared to the left of mid
22.     else:
23.         return mid
24.
25.     # element was not present in the list, return -1
26. return -1
27.
28.
29. # Initial list1
30. list1 = [12, 24, 32, 39, 45, 50, 54]
31. n = 45
32.
33. # Function call
34. result = binary_search(list1, n)
35.
36. if result != -1:
37.     print("Element is present at index", str(result))
38. else:
39.     print("Element is not present in list1 ")
```

Output:

Element is present at index 4

Explanation:

In the above program -

- We have created a function called **binary_search()** function which takes two arguments - a list to sorted and a number to be searched.
- We have declared two variables to store the lowest and highest values in the list. The low is assigned initial value to 0, **high** to **len(list1) - 1** and mid as 0.
- Next, we have declared the **while** loop with the condition that the **lowest** is equal and smaller than the **highest** The while loop will iterate if the number has not been found yet.
- In the while loop, we find the mid value and compare the index value to the number we are searching for.
- If the value of the mid-index is smaller than **n**, we increase the mid value by 1 and assign it to The search moves to the left side.
- Otherwise, decrease the mid value and assign it to the **high**. The search moves to the right side.
- If the n is equal to the mid value then return **mid**.
- This will happen until the **low** is equal and smaller than the **high**.
- If we reach at the end of the function, then the element is not present in the list. We return -1 to the calling function.

Sorting - Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort.

Bubble Sort:

It is a simple sorting algorithm which sorts 'n' number of elements in the list by comparing the each pair of adjacent items and swaps them if they are in wrong order.

Algorithm:

1. Starting with the first element (index=0), compare the current element with the next element of a list.
2. If the current element is greater (>) than the next element of the list then swap them.
3. If the current element is less (<) than the next element of the list move to the next element.
4. Repeat step 1 until it correct order is framed.

For ex: list1= [10, 15, 4, 23, 0]

If > --- yes ---- swap

If < --- No ---- Do nothing/remains same

#Write a python program to arrange the elements in ascending order using bubble sort:

list1=[9,16,6,26,0]

print("unsorted list1 is", list1)

for j in range(len(list1)-1):

 for i in range(len(list1)-1):

 if list1[i]>list1[i+1]:

 list1[i],list1[i+1]=list1[i+1],list1[i]

 print(list1)

 else:

 print(list1)

print()

print("sorted list is",list1)

so here we are comparing values again and again, so we use loops.

Output:

unsorted list1 is [9, 16, 6, 26, 0]

[9, 16, 6, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 0, 16, 26]

[6, 9, 0, 16, 26]

[6, 9, 0, 16, 26]

[6, 0, 9, 16, 26]

[6, 0, 9, 16, 26]

[6, 0, 9, 16, 26]

[0, 6, 9, 16, 26]

[0, 6, 9, 16, 26]

[0, 6, 9, 16, 26]

[0, 6, 9, 16, 26]

sorted list is [0, 6, 9, 16, 26]

#If we want to reduce no of iterations/steps in output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/bubb.py

```
list1=[9,16,6,26,0]
print("unsorted list1 is", list1)
for j in range(len(list1)-1,0,-1):
    for i in range(j):
        if list1[i]>list1[i+1]:
            list1[i],list1[i+1]=list1[i+1],list1[i]
            print(list1)
        else:
            print(list1)
    print( )
print("sorted list is",list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/bubb2.py

unsorted list1 is [9, 16, 6, 26, 0]

[9, 16, 6, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 0, 16, 26]

[6, 9, 0, 16, 26]

[6, 0, 9, 16, 26]

[0, 6, 9, 16, 26]

sorted list is [0, 6, 9, 16, 26]

In a different way:

```
list1=[9,16,6,26,0]
print("unsorted list1 is", list1)
for j in range(len(list1)-1):
    for i in range(len(list1)-1-j):
        if list1[i]>list1[i+1]:
            list1[i],list1[i+1]=list1[i+1],list1[i]
            print(list1)
        else:
            print(list1)
    print( )
print("sorted list is",list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/bubb3.py

unsorted list1 is [9, 16, 6, 26, 0]

[9, 16, 6, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 26, 0]

[9, 6, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 16, 0, 26]

[6, 9, 0, 16, 26]

[6, 9, 0, 16, 26]

[6, 0, 9, 16, 26]

[0, 6, 9, 16, 26]

sorted list is [0, 6, 9, 16, 26]

Program to give input from the user to sort the elements

```
list1=[]
```

```
num=int(input("enter how many numbers:"))
```

```
print("enter values")
```

```
for k in range(num):
```

```
    list1.append(int(input()))
```

```
print("unsorted list1 is", list1)
```

```
for j in range(len(list1)-1):
```

```
    for i in range(len(list1)-1):
```

```
        if list1[i]>list1[i+1]:
```

```
            list1[i],list1[i+1]=list1[i+1],list1[i]
```

```
            print(list1)
```

```
        else:
```

```
            print(list1)
```

```
    print()
```

```
print("sorted list is",list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/bubb4.py

enter how many numbers:5

enter values

5

77

4

66

30

unsorted list1 is [5, 77, 4, 66, 30]

[5, 77, 4, 66, 30]

```
[5, 4, 77, 66, 30]
[5, 4, 66, 77, 30]
[5, 4, 66, 30, 77]
```

```
[4, 5, 66, 30, 77]
[4, 5, 66, 30, 77]
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
```

```
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
```

```
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
[4, 5, 30, 66, 77]
```

sorted list is [4, 5, 30, 66, 77]

#bubble sort program for descending order

```
list1=[9,16,6,26,0]
print("unsorted list1 is", list1)
for j in range(len(list1)-1):
    for i in range(len(list1)-1):
        if list1[i]<list1[i+1]:
            list1[i],list1[i+1]=list1[i+1],list1[i]
            print(list1)
        else:
            print(list1)
    print()
print("sorted list is",list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-2/pyyy/bubbdesc.py

unsorted list1 is [9, 16, 6, 26, 0]

[16, 9, 6, 26, 0]

[16, 9, 6, 26, 0]

[16, 9, 26, 6, 0]

[16, 9, 26, 6, 0]

[16, 9, 26, 6, 0]

[16, 26, 9, 6, 0]

[16, 26, 9, 6, 0]

[16, 26, 9, 6, 0]

[26, 16, 9, 6, 0]

```
[26, 16, 9, 6, 0]
[26, 16, 9, 6, 0]
[26, 16, 9, 6, 0]
```

```
[26, 16, 9, 6, 0]
[26, 16, 9, 6, 0]
[26, 16, 9, 6, 0]
[26, 16, 9, 6, 0]
```

sorted list is [26, 16, 9, 6, 0]

Selection Sort:

Sort (): Built-in list method

Sorted (): built in function

- Generally this algorithm is called as in-place comparison based algorithm. We compare numbers and place them in correct position.
- Search the list and find out the min value, this we can do it by min () method.
- We can take min value as the first element of the list and compare with the next element until we find small value.

Algorithm:

1. Starting from the first element search for smallest/biggest element in the list of numbers.
2. Swap min/max number with first element
3. Take the sub-list (ignore sorted part) and repeat step 1 and 2 until all the elements are sorted.

#Write a python program to arrange the elements in ascending order using selection sort:

```
list1=[5,3,7,1,9,6]
print(list1)
for i in range(len(list1)):
    min_val=min(list1[i:])
    min_ind=list1.index(min_val)
    list1[i],list1[min_ind]=list1[min_ind],list1[i]
print(list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/selectasce.py

```
[5, 3, 7, 1, 9, 6]
[1, 3, 7, 5, 9, 6]
[1, 3, 7, 5, 9, 6]
[1, 3, 5, 7, 9, 6]
[1, 3, 5, 6, 9, 7]
[1, 3, 5, 6, 7, 9]
[1, 3, 5, 6, 7, 9]
```

#Write a python program to arrange the elements in descending order using selection sort:

```
list1=[5,3,7,1,9,6]
print(list1)
for i in range(len(list1)):
    min_val=max(list1[i:])
    min_ind=list1.index(min_val)
```

```
list1[i],list1[min_ind]=list1[min_ind],list1[i]
print(list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/selecdec.py

[5, 3, 7, 1, 9, 6]

[9, 7, 6, 5, 3, 1]

Note: If we want the elements to be sorted in descending order use max () method in place of min ().

Insertion Sort:

- Insertion sort is not a fast sorting algorithm. It is useful only for small datasets.
- It is a simple sorting algorithm that builds the final sorted list one item at a time.

Algorithm:

1. Consider the first element to be sorted & the rest to be unsorted.
2. Take the first element in unsorted order (u1) and compare it with sorted part elements(s1)
3. If $u1 < s1$ then insert u1 in the correct order, else leave as it is.
4. Take the next element in the unsorted part and compare with sorted element.
5. Repeat step 3 and step 4 until all the elements get sorted.

Write a python program to arrange the elements in ascending order using insertion sort (with functions)

```
def insertionsort(my_list):
```

```
#we need to sort the unsorted part at a time.
```

```
for index in range(1,len(my_list)):
```

```
    current_element=my_list[index]
```

```
    pos=index
```

```
    while current_element<my_list[pos-1] and pos>0:
```

```
        my_list[pos]=my_list[pos-1]
```

```
        pos=pos-1
```

```
    my_list[pos]=current_element
```

```
list1=[3,5,1,0,10,2]
```

```
insertionsort(list1)
```

```
print(list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/inserti.py

[0, 1, 2, 3, 5, 10]

Write a python program to arrange the elements in descending order using insertion sort (with functions)

```
def insertionsort(my_list):
```

```
#we need to sort the unsorted part at a time.
```

```
for index in range(1,len(my_list)):
```

```
    current_element=my_list[index]
```

```
    pos=index
```

```
    while current_element>my_list[pos-1] and pos>0:
```

```
        my_list[pos]=my_list[pos-1]
```

```
        pos=pos-1
```

```
    my_list[pos]=current_element
```

```
#list1=[3,5,1,0,10,2]
```

num=int(input("enter how many elements to be in list"))
list1=[int(input()) for i in range (num)]

```
#insertionsort(list1)
#print(list1)
num=int(input("enter how many elements to be in list"))
list1=[int(input())for i in range(num)]
insertionsort(list1)
print(list1)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/insertdesc.py

enter how many elements to be in list 5

8

1

4

10

2

[10, 8, 4, 2, 1]

Merge Sort:

Generally this merge sort works on the basis of divide and conquer algorithm. The three steps need to be followed is divide, conquer and combine. We will be dividing the unsorted list into sub list until the single element in a list is found.

Algorithm:

1. Split the unsorted list.
2. Compare each of the elements and group them
3. Repeat step 2 until whole list is merged and sorted.

Write a python program to arrange the elements in ascending order using Merge sort (with functions)

```
def mergesort(list1):
    if len(list1)>1:
        mid=len(list1)//2
        left_list=list1[:mid]
        right_list=list1[mid:]
        mergesort(left_list)
        mergesort(right_list)
        i=0
        j=0
        k=0
        while i<len(left_list) and j<len(right_list):
            if left_list[i]<right_list[j]:
                list1[k]=left_list[i]
                i=i+1
                k=k+1
            else:
                list1[k]=right_list[j]
                j=j+1
                k=k+1
        while i<len(left_list):
```

```

        list1[k]=left_list[i]
        i=i+1
        k=k+1
    while j<len(right_list):
        list1[k]=right_list[j]
        j=j+1
        k=k+1
num=int(input("how many numbers in list1 "))
list1=[int(input()) for x in range(num)]
mergesort(list1)
print("sorted list1",list1)

```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/merg.py

```

how many numbers in list15
5
9
10
1
66
sorted list1 [1, 5, 9, 10, 66]

```

Quick Sort:

Algorithm:

1. Select the pivot element
2. Find out the correct position of pivot element in the list by rearranging it.
3. Divide the list based on pivot element
4. Sort the sub list recursively

Note: Pivot element can be first, last, random elements or median of three values.

In the following program we are going to write 3 functions. The first function is to find pivot element and its correct position. In second function we divide the list based on pivot element and sort the sub list and third function (main fun) is to print input and output.

Write a python program to arrange the elements in ascending order using Quick sort (with functions)

#To get the correct position of pivot element:

```

def pivot_place(list1,first,last):
    pivot=list1[first]
    left=first+1
    right=last
    while True:
        while left<=right and list1[left]<=pivot:
            left=left+1
        while left<=right and list1[right]>=pivot:
            right=right-1
        if right<left:
            break
    else:
        list1[left],list1[right]=list1[right],list1[left]

```

```

    list1[first],list1[right]=list1[right],list1[first]
    return right
#second function
def quicksort(list1,first,last):
    if first<last:
        p=pivot_place(list1,first,last)
        quicksort(list1,first,p-1)
        quicksort(list1,p+1,last)
#main fun
list1=[56,25,93,15,31,44]
n=len(list1)
quicksort(list1,0,n-1)
print(list1)

```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/quicksort.py
 [15, 25, 31, 44, 56, 93]

Write a python program to arrange the elements in descending order using Quick sort (with functions)

#To get the correct position of pivot element:

```

def pivot_place(list1,first,last):
    pivot=list1[first]
    left=first+1
    right=last
    while True:
        while left<=right and list1[left]>=pivot:
            left=left+1
        while left<=right and list1[right]<=pivot:
            right=right-1
        if right<left:
            break
        else:
            list1[left],list1[right]=list1[right],list1[left]
    list1[first],list1[right]=list1[right],list1[first]
    return right
def quicksort(list1,first,last):
    if first<last:
        p=pivot_place(list1,first,last)
        quicksort(list1,first,p-1)
        quicksort(list1,p+1,last)
#main fun
list1=[56,25,93,15,31,44]
n=len(list1)
quicksort(list1,0,n-1)
print(list1)

```

Output:

[15 ,25 ,31 ,44 ,56 ,93]

4.1.3 Binary Search

In this section, we describe a classic recursive algorithm, *binary search*, that is used to efficiently locate a target value within a sorted sequence of n elements. This is among the most important of computer algorithms, and it is the reason that we so often store data in sorted order (as in Figure 4.4).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37

Figure 4.4: Values stored in sorted order within an indexable sequence, such as a Python list. The numbers at top are the indices.

When the sequence is *unsorted*, the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set. This is known as the *sequential search* algorithm. This algorithm runs in $O(n)$ time (i.e., linear time) since every element is inspected in the worst case.

When the sequence is *sorted* and *indexable*, there is a much more efficient algorithm. (For intuition, think about how you would accomplish this task by hand!) For any index j , we know that all the values stored at indices $0, \dots, j-1$ are less than or equal to the value at index j , and all the values stored at indices $j+1, \dots, n-1$ are greater than or equal to that at index j . This observation allows us to quickly “home in” on a search target using a variant of the children’s game “high-low.” We call an element of the sequence a *candidate* if, at the current stage of the search, we cannot rule out that this item matches the target. The algorithm maintains two parameters, low and high, such that all the candidate entries have index at least low and at most high. Initially, $\text{low} = 0$ and $\text{high} = n - 1$. We then compare the target value to the median candidate, that is, the item $\text{data}[\text{mid}]$ with index

$$\text{mid} = \lfloor (\text{low} + \text{high}) / 2 \rfloor.$$

We consider three cases:

- If the target equals $\text{data}[\text{mid}]$, then we have found the item we are looking for, and the search terminates successfully.
- If $\text{target} < \text{data}[\text{mid}]$, then we recur on the first half of the sequence, that is, on the interval of indices from low to $\text{mid} - 1$.
- If $\text{target} > \text{data}[\text{mid}]$, then we recur on the second half of the sequence, that is, on the interval of indices from $\text{mid} + 1$ to high.

An unsuccessful search occurs if $\text{low} > \text{high}$, as the interval $[\text{low}, \text{high}]$ is empty.

'apple' < 'banana'. Suppose that we have an application in which we have a list of strings that are all known to represent integral values (e.g., '12'), and our goal is to sort the strings according to those integral values.

In Python, the standard approach for customizing the order for a sorting algorithm is to provide, as an optional parameter to the sorting function, an object that is itself a one-parameter function that computes a key for a given element. (See Sections 1.5 and 1.10 for a discussion of this approach in the context of the built-in max function.) For example, with a list of (numeric) strings, we might wish to use the value of `int(s)` as a key for a string `s` of the list. In this case, the constructor for the `int` class can serve as the one-parameter function for computing a key. In that way, the string '4' will be ordered before string '12' because its key `int('4') < int('12')`. We leave it as an exercise to support such an optional key parameter for the `pq_sort` function. (See Exercise C-9.46.)

9.4.1 Selection-Sort and Insertion-Sort

Our `pq_sort` function works correctly given any valid implementation of the priority queue class. However, the running time of the sorting algorithm depends on the running times of the operations `add` and `remove_min` for the given priority queue class. We next discuss a choice of priority queue implementations that in effect cause the `pq_sort` computation to behave as one of several classic sorting algorithms.

Selection-Sort

If we implement P with an unsorted list, then Phase 1 of `pq_sort` takes $O(n)$ time, for we can add each element in $O(1)$ time. In Phase 2, the running time of each `remove_min` operation is proportional to the size of P . Thus, the bottleneck computation is the repeated “selection” of the minimum element in Phase 2. For this reason, this algorithm is better known as *selection-sort*. (See Figure 9.7.)

As noted above, the bottleneck is in Phase 2 where we repeatedly remove an entry with smallest key from the priority queue P . The size of P starts at n and incrementally decreases with each `remove_min` until it becomes 0. Thus, the first operation takes time $O(n)$, the second one takes time $O(n-1)$, and so on. Therefore, the total time needed for the second phase is

$$O(n + (n-1) + \cdots + 2 + 1) = O(\sum_{i=1}^n i).$$

By Proposition 3.3, we have $\sum_{i=1}^n i = n(n+1)/2$. Thus, Phase 2 takes time $O(n^2)$, as does the entire selection-sort algorithm.

		<i>Collection C</i>	<i>Priority Queue P</i>
Input		(7, 4, 8, 2, 5, 3)	()
Phase 1	(a)	(4, 8, 2, 5, 3)	(7)
	(b)	(8, 2, 5, 3)	(7, 4)
	$\vdots$	$\vdots$	$\vdots$
	(f)	()	(7, 4, 8, 2, 5, 3)
Phase 2	(a)	(2)	(7, 4, 8, 5, 3)
	(b)	(2, 3)	(7, 4, 8, 5)
	(c)	(2, 3, 4)	(7, 8, 5)
	(d)	(2, 3, 4, 5)	(7, 8)
	(e)	(2, 3, 4, 5, 7)	(8)
	(f)	(2, 3, 4, 5, 7, 8)	()

Figure 9.7: Execution of selection-sort on collection $C = (7, 4, 8, 2, 5, 3)$.

Insertion-Sort

If we implement the priority queue P using a sorted list, then we improve the running time of Phase 2 to $O(n)$, for each `remove_min` operation on P now takes $O(1)$ time. Unfortunately, Phase 1 becomes the bottleneck for the running time, since, in the worst case, each `add` operation takes time proportional to the current size of P . This sorting algorithm is better known as **insertion-sort** (see Figure 9.8); in fact, our implementation for adding an element to a priority queue is almost identical to a step of insertion-sort as presented in Section 7.5.

The worst-case running time of Phase 1 of insertion-sort is

$$O(1 + 2 + \dots + (n-1) + n) = O(\sum_{i=1}^n i).$$

Again, by Proposition 3.3, this implies a worst-case $O(n^2)$ time for Phase 1, and thus, the entire insertion-sort algorithm. However, unlike selection-sort, insertion-sort has a *best-case* running time of $O(n)$.

		<i>Collection C</i>	<i>Priority Queue P</i>
Input		(7, 4, 8, 2, 5, 3)	()
Phase 1	(a)	(4, 8, 2, 5, 3)	(7)
	(b)	(8, 2, 5, 3)	(4, 7)
	(c)	(2, 5, 3)	(4, 7, 8)
	(d)	(5, 3)	(2, 4, 7, 8)
	(e)	(3)	(2, 4, 5, 7, 8)
	(f)	()	(2, 3, 4, 5, 7, 8)
Phase 2	(a)	(2)	(3, 4, 5, 7, 8)
	(b)	(2, 3)	(4, 5, 7, 8)
	$\vdots$	$\vdots$	$\vdots$
	(f)	(2, 3, 4, 5, 7, 8)	()

Figure 9.8: Execution of insertion-sort on collection $C = (7, 4, 8, 2, 5, 3)$.

12.2 Merge-Sort

12.2.1 Divide-and-Conquer

The first two algorithms we describe in this chapter, merge-sort and quick-sort, use recursion in an algorithmic design pattern called ***divide-and-conquer***. We have already seen the power of recursion in describing algorithms in an elegant manner (see Chapter 4). The divide-and-conquer pattern consists of the following three steps:

1. ***Divide***: If the input size is smaller than a certain threshold (say, one or two elements), solve the problem directly using a straightforward method and return the solution so obtained. Otherwise, divide the input data into two or more disjoint subsets.
2. ***Conquer***: Recursively solve the subproblems associated with the subsets.
3. ***Combine***: Take the solutions to the subproblems and merge them into a solution to the original problem.

Using Divide-and-Conquer for Sorting

We will first describe the merge-sort algorithm at a high level, without focusing on whether the data is an array-based (Python) list or a linked list; we will soon give concrete implementations for each. To sort a sequence S with n elements using the three divide-and-conquer steps, the merge-sort algorithm proceeds as follows:

1. ***Divide***: If S has zero or one element, return S immediately; it is already sorted. Otherwise (S has at least two elements), remove all the elements from S and put them into two sequences, S_1 and S_2 , each containing about half of the elements of S ; that is, S_1 contains the first $\lfloor n/2 \rfloor$ elements of S , and S_2 contains the remaining $\lceil n/2 \rceil$ elements.
2. ***Conquer***: Recursively sort sequences S_1 and S_2 .
3. ***Combine***: Put back the elements into S by merging the sorted sequences S_1 and S_2 into a sorted sequence.

In reference to the divide step, we recall that the notation $\lfloor x \rfloor$ indicates the ***floor*** of x , that is, the largest integer k , such that $k \leq x$. Similarly, the notation $\lceil x \rceil$ indicates the ***ceiling*** of x , that is, the smallest integer m , such that $x \leq m$.

We can visualize an execution of the merge-sort algorithm by means of a binary tree T , called the *merge-sort tree*. Each node of T represents a recursive invocation (or call) of the merge-sort algorithm. We associate with each node v of T the sequence S that is processed by the invocation associated with v . The children of node v are associated with the recursive calls that process the subsequences S_1 and S_2 of S . The external nodes of T are associated with individual elements of S , corresponding to instances of the algorithm that make no recursive calls.

Figure 12.1 summarizes an execution of the merge-sort algorithm by showing the input and output sequences processed at each node of the merge-sort tree. The step-by-step evolution of the merge-sort tree is shown in Figures 12.2 through 12.4.

This algorithm visualization in terms of the merge-sort tree helps us analyze the running time of the merge-sort algorithm. In particular, since the size of the input sequence roughly halves at each recursive call of merge-sort, the height of the merge-sort tree is about $\log n$ (recall that the base of $\log$ is 2 if omitted).

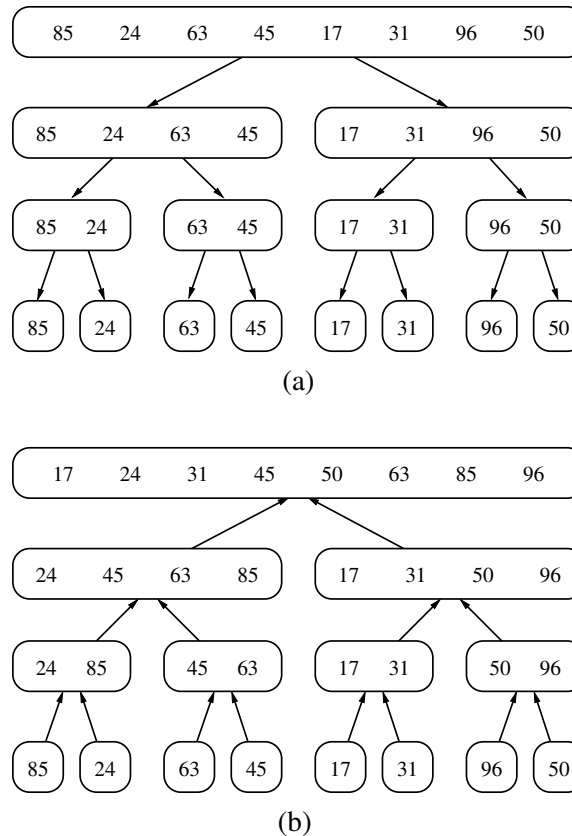


Figure 12.1: Merge-sort tree T for an execution of the merge-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T .

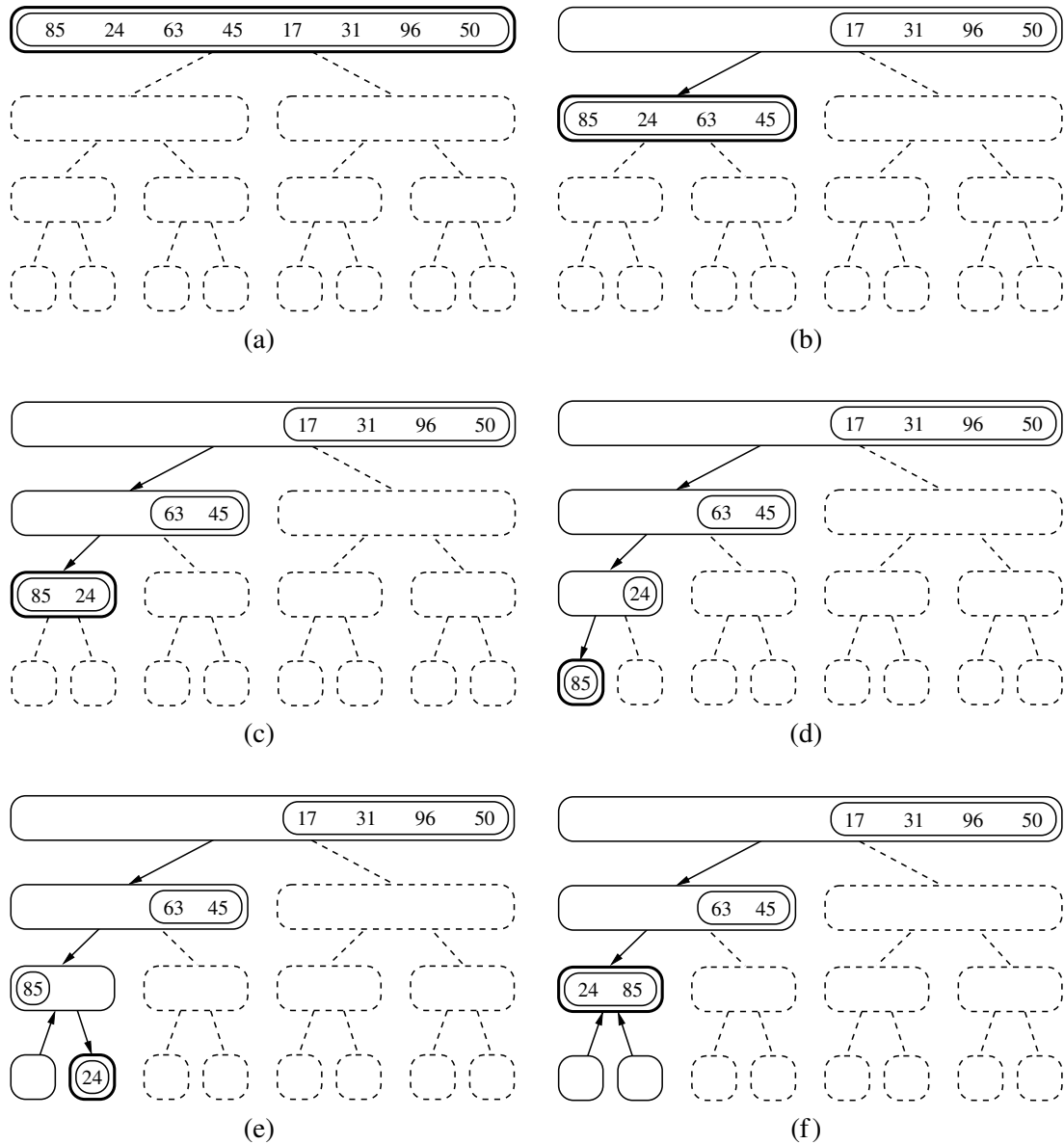


Figure 12.2: Visualization of an execution of merge-sort. Each node of the tree represents a recursive call of merge-sort. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the current call. The empty nodes drawn with thin lines represent completed calls. The remaining nodes (drawn with thin lines and not empty) represent calls that are waiting for a child invocation to return. (Continues in Figure 12.3.)

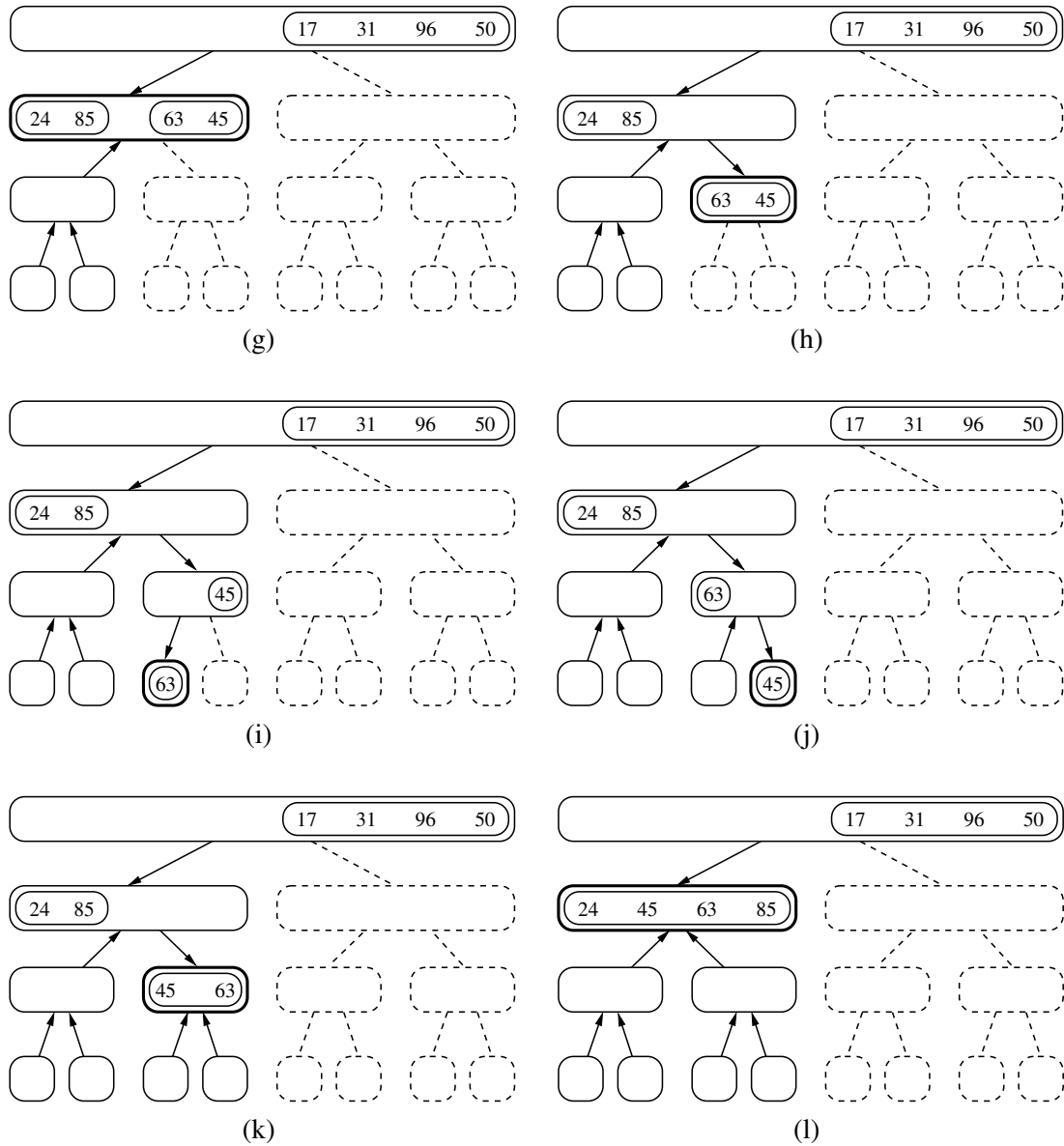


Figure 12.3: Visualization of an execution of merge-sort. (Combined with Figures 12.2 and 12.4.)

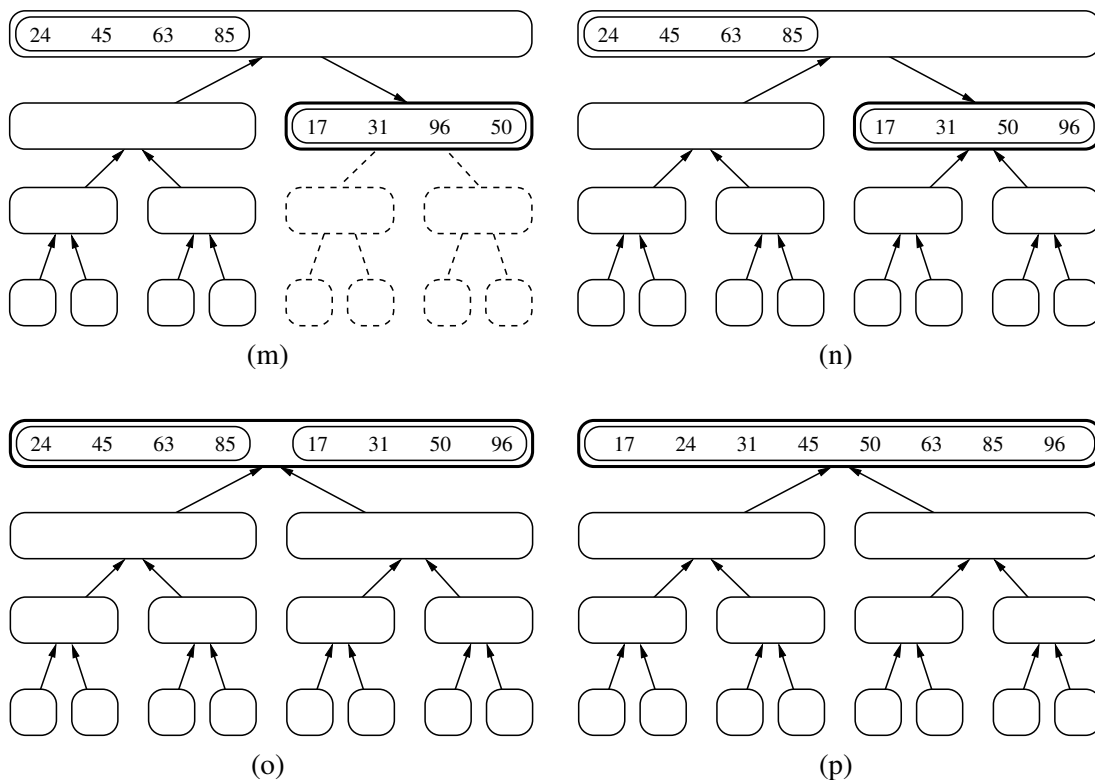


Figure 12.4: Visualization of an execution of merge-sort (continued from Figure 12.3). Several invocations are omitted between (m) and (n). Note the merging of two halves performed in step (p).

Proposition 12.1: *The merge-sort tree associated with an execution of merge-sort on a sequence of size n has height $\lceil \log n \rceil$.*

We leave the justification of Proposition 12.1 as a simple exercise (R-12.1). We will use this proposition to analyze the running time of the merge-sort algorithm.

Having given an overview of merge-sort and an illustration of how it works, let us consider each of the steps of this divide-and-conquer algorithm in more detail. Dividing a sequence of size n involves separating it at the element with index $\lceil n/2 \rceil$, and recursive calls can be started by passing these smaller sequences as parameters. The difficult step is combining the two sorted sequences into a single sorted sequence. Thus, before we present our analysis of merge-sort, we need to say more about how this is done.

12.2.2 Array-Based Implementation of Merge-Sort

We begin by focusing on the case when a sequence of items is represented as an (array-based) Python list. The merge function (Code Fragment 12.1) is responsible for the subtask of merging two previously sorted sequences, S_1 and S_2 , with the output copied into S . We copy one element during each pass of the while loop, conditionally determining whether the next element should be taken from S_1 or S_2 . The divide-and-conquer merge-sort algorithm is given in Code Fragment 12.2.

We illustrate a step of the merge process in Figure 12.5. During the process, index i represents the number of elements of S_1 that have been copied to S , while index j represents the number of elements of S_2 that have been copied to S . Assuming S_1 and S_2 both have at least one uncopied element, we copy the smaller of the two elements being considered. Since $i + j$ objects have been previously copied, the next element is placed in $S[i + j]$. (For example, when $i + j$ is 0, the next element is copied to $S[0]$). If we reach the end of one of the sequences, we must copy the next element from the other.

```

1 def merge(S1, S2, S):
2     """ Merge two sorted Python lists S1 and S2 into properly sized list S. """
3     i = j = 0
4     while i + j < len(S):
5         if j == len(S2) or (i < len(S1) and S1[i] < S2[j]):
6             S[i+j] = S1[i]                # copy ith element of S1 as next item of S
7             i += 1
8         else:
9             S[i+j] = S2[j]                # copy jth element of S2 as next item of S
10            j += 1

```

Code Fragment 12.1: An implementation of the merge operation for Python's array-based list class.

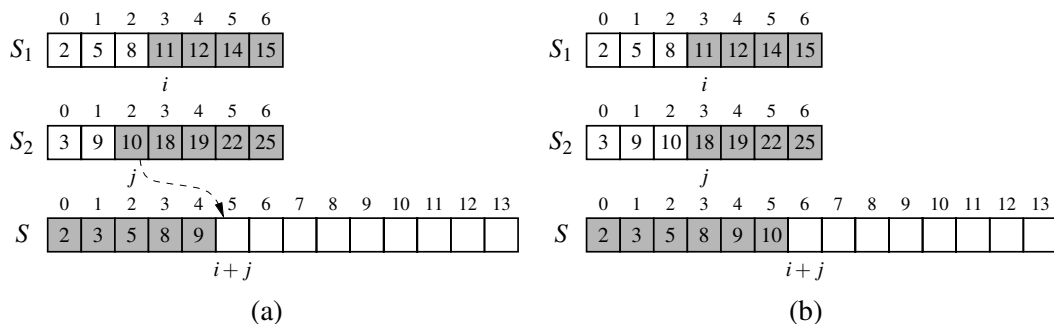


Figure 12.5: A step in the merge of two sorted arrays for which $S_2[j] < S_1[i]$. We show the arrays before the copy step in (a) and after it in (b).

```

1 def merge_sort(S):
2     """Sort the elements of Python list S using the merge-sort algorithm."""
3     n = len(S)
4     if n < 2:
5         return                # list is already sorted
6     # divide
7     mid = n // 2
8     S1 = S[0:mid]              # copy of first half
9     S2 = S[mid:n]              # copy of second half
10    # conquer (with recursion)
11    merge_sort(S1)              # sort copy of first half
12    merge_sort(S2)              # sort copy of second half
13    # merge results
14    merge(S1, S2, S)            # merge sorted halves back into S

```

Code Fragment 12.2: An implementation of the recursive merge-sort algorithm for Python’s array-based list class (using the merge function defined in Code Fragment 12.1).

12.2.3 The Running Time of Merge-Sort

We begin by analyzing the running time of the merge algorithm. Let n_1 and n_2 be the number of elements of S_1 and S_2 , respectively. It is clear that the operations performed inside each pass of the while loop take $O(1)$ time. The key observation is that during each iteration of the loop, one element is copied from either S_1 or S_2 into S (and that element is considered no further). Therefore, the number of iterations of the loop is $n_1 + n_2$. Thus, the running time of algorithm merge is $O(n_1 + n_2)$.

Having analyzed the running time of the merge algorithm used to combine subproblems, let us analyze the running time of the entire merge-sort algorithm, assuming it is given an input sequence of n elements. For simplicity, we restrict our attention to the case where n is a power of 2. We leave it to an exercise (R-12.3) to show that the result of our analysis also holds when n is not a power of 2.

When evaluating the merge-sort recursion, we rely on the analysis technique introduced in Section 4.2. We account for the amount of time spent within each recursive call, but excluding any time spent waiting for successive recursive calls to terminate. In the case of our merge_sort function, we account for the time to divide the sequence into two subsequences, and the call to merge to combine the two sorted sequences, but we exclude the two recursive calls to merge_sort.

A merge-sort tree T , as portrayed in Figures 12.2 through 12.4, can guide our analysis. Consider a recursive call associated with a node v of the merge-sort tree T . The divide step at node v is straightforward; this step runs in time proportional to the size of the sequence for v , based on the use of slicing to create copies of the two list halves. We have already observed that the merging step also takes time that is linear in the size of the merged sequence. If we let i denote the depth of node v , the time spent at node v is $O(n/2^i)$, since the size of the sequence handled by the recursive call associated with v is equal to $n/2^i$.

Looking at the tree T more globally, as shown in Figure 12.6, we see that, given our definition of “time spent at a node,” the running time of merge-sort is equal to the sum of the times spent at the nodes of T . Observe that T has exactly 2^i nodes at depth i . This simple observation has an important consequence, for it implies that the overall time spent at all the nodes of T at depth i is $O(2^i \cdot n/2^i)$, which is $O(n)$. By Proposition 12.1, the height of T is $\lceil \log n \rceil$. Thus, since the time spent at each of the $\lceil \log n \rceil + 1$ levels of T is $O(n)$, we have the following result:

Proposition 12.2: *Algorithm merge-sort sorts a sequence S of size n in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

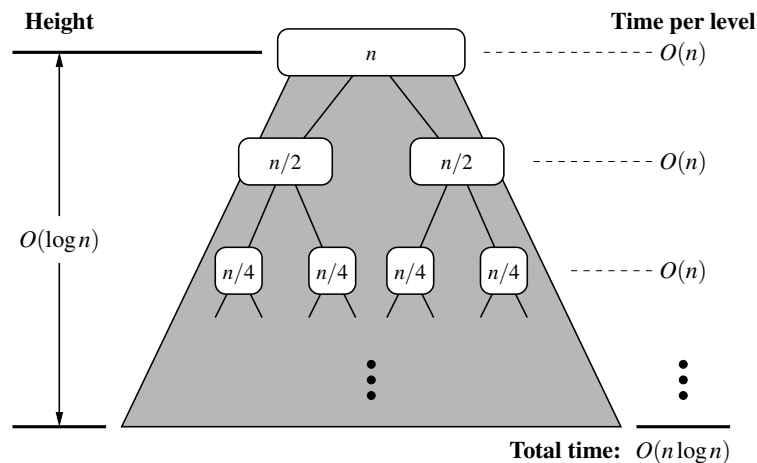


Figure 12.6: A visual analysis of the running time of merge-sort. Each node represents the time spent in a particular recursive call, labeled with the size of its subproblem.

12.2.4 Merge-Sort and Recurrence Equations ★

There is another way to justify that the running time of the merge-sort algorithm is $O(n \log n)$ (Proposition 12.2). Namely, we can deal more directly with the recursive nature of the merge-sort algorithm. In this section, we present such an analysis of the running time of merge-sort, and in so doing, introduce the mathematical concept of a **recurrence equation** (also known as **recurrence relation**).

Let the function $t(n)$ denote the worst-case running time of merge-sort on an input sequence of size n . Since merge-sort is recursive, we can characterize function $t(n)$ by means of an equation where the function $t(n)$ is recursively expressed in terms of itself. In order to simplify our characterization of $t(n)$, let us restrict our attention to the case when n is a power of 2. (We leave the problem of showing that our asymptotic characterization still holds in the general case as an exercise.) In this case, we can specify the definition of $t(n)$ as

$$t(n) = \begin{cases} b & \text{if } n \leq 1 \\ 2t(n/2) + cn & \text{otherwise.} \end{cases}$$

An expression such as the one above is called a recurrence equation, since the function appears on both the left- and right-hand sides of the equal sign. Although such a characterization is correct and accurate, what we really desire is a big-Oh type of characterization of $t(n)$ that does not involve the function $t(n)$ itself. That is, we want a **closed-form** characterization of $t(n)$.

We can obtain a closed-form solution by applying the definition of a recurrence equation, assuming n is relatively large. For example, after one more application of the equation above, we can write a new recurrence for $t(n)$ as

$$\begin{aligned} t(n) &= 2(2t(n/2^2) + (cn/2)) + cn \\ &= 2^2t(n/2^2) + 2(cn/2) + cn = 2^2t(n/2^2) + 2cn. \end{aligned}$$

If we apply the equation again, we get $t(n) = 2^3t(n/2^3) + 3cn$. At this point, we should see a pattern emerging, so that after applying this equation i times, we get

$$t(n) = 2^i t(n/2^i) + icn.$$

The issue that remains, then, is to determine when to stop this process. To see when to stop, recall that we switch to the closed form $t(n) = b$ when $n \leq 1$, which will occur when $2^i = n$. In other words, this will occur when $i = \log n$. Making this substitution, then, yields

$$\begin{aligned} t(n) &= 2^{\log n} t(n/2^{\log n}) + (\log n)cn \\ &= nt(1) + cn \log n \\ &= nb + cn \log n. \end{aligned}$$

That is, we get an alternative justification of the fact that $t(n)$ is $O(n \log n)$.

12.2.5 Alternative Implementations of Merge-Sort

Sorting Linked Lists

The merge-sort algorithm can easily be adapted to use any form of a basic queue as its container type. In Code Fragment 12.3, we provide such an implementation, based on use of the `LinkedQueue` class from Section 7.1.2. The $O(n \log n)$ bound for merge-sort from Proposition 12.2 applies to this implementation as well, since each basic operation runs in $O(1)$ time when implemented with a linked list. We show an example execution of this version of the merge algorithm in Figure 12.7.

```

1  def merge(S1, S2, S):
2      """ Merge two sorted queue instances S1 and S2 into empty queue S."""
3      while not S1.is_empty() and not S2.is_empty():
4          if S1.first() < S2.first():
5              S.enqueue(S1.dequeue())
6          else:
7              S.enqueue(S2.dequeue())
8      while not S1.is_empty():           # move remaining elements of S1 to S
9          S.enqueue(S1.dequeue())
10     while not S2.is_empty():           # move remaining elements of S2 to S
11         S.enqueue(S2.dequeue())
12
13  def merge_sort(S):
14      """ Sort the elements of queue S using the merge-sort algorithm."""
15      n = len(S)
16      if n < 2:
17          return                        # list is already sorted
18      # divide
19      S1 = LinkedQueue()                # or any other queue implementation
20      S2 = LinkedQueue()
21      while len(S1) < n // 2:           # move the first n//2 elements to S1
22          S1.enqueue(S.dequeue())
23      while not S.is_empty():           # move the rest to S2
24          S2.enqueue(S.dequeue())
25      # conquer (with recursion)
26      merge_sort(S1)                   # sort first half
27      merge_sort(S2)                   # sort second half
28      # merge results
29      merge(S1, S2, S)                 # merge sorted halves back into S

```

Code Fragment 12.3: An implementation of merge-sort using a basic queue.

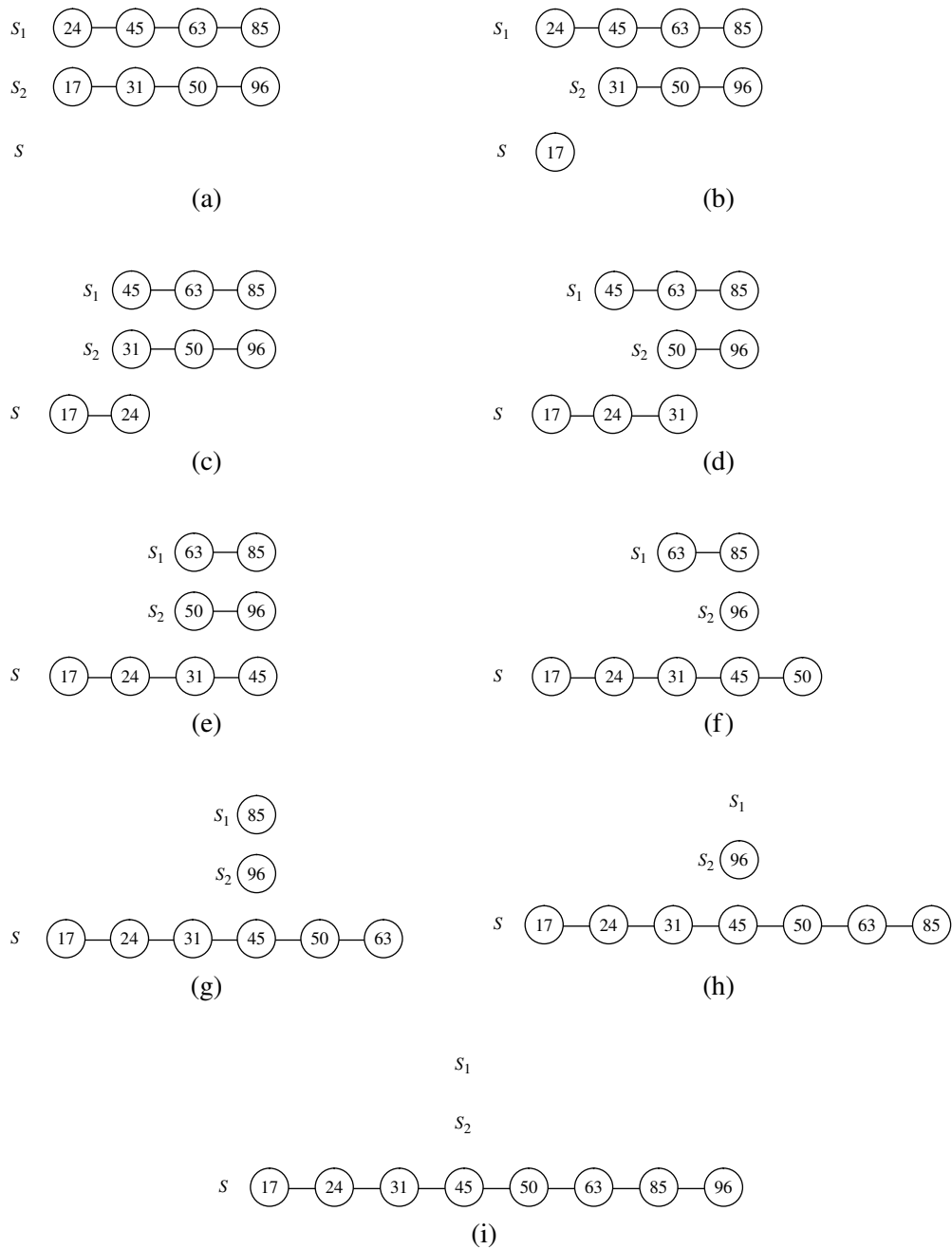


Figure 12.7: Example of an execution of the merge algorithm, as implemented in Code Fragment 12.3 using queues.

A Bottom-Up (Nonrecursive) Merge-Sort

There is a nonrecursive version of array-based merge-sort, which runs in $O(n \log n)$ time. It is a bit faster than recursive merge-sort in practice, as it avoids the extra overheads of recursive calls and temporary memory at each level. The main idea is to perform merge-sort bottom-up, performing the merges level by level going up the merge-sort tree. Given an input array of elements, we begin by merging every successive pair of elements into sorted runs of length two. We merge these runs into runs of length four, merge these new runs into runs of length eight, and so on, until the array is sorted. To keep the space usage reasonable, we deploy a second array that stores the merged runs (swapping input and output arrays after each iteration). We give a Python implementation in Code Fragment 12.4. A similar bottom-up approach can be used for sorting linked lists. (See Exercise C-12.29.)

```

1  def merge(src, result, start, inc):
2      """ Merge src[start:start+inc] and src[start+inc:start+2*inc] into result."""
3      end1 = start+inc                # boundary for run 1
4      end2 = min(start+2*inc, len(src)) # boundary for run 2
5      x, y, z = start, start+inc, start # index into run 1, run 2, result
6      while x < end1 and y < end2:
7          if src[x] < src[y]:
8              result[z] = src[x]; x += 1    # copy from run 1 and increment x
9          else:
10             result[z] = src[y]; y += 1    # copy from run 2 and increment y
11             z += 1                        # increment z to reflect new result
12     if x < end1:
13         result[z:end2] = src[x:end1]      # copy remainder of run 1 to output
14     elif y < end2:
15         result[z:end2] = src[y:end2]      # copy remainder of run 2 to output
16
17 def merge_sort(S):
18     """ Sort the elements of Python list S using the merge-sort algorithm."""
19     n = len(S)
20     logn = math.ceil(math.log(n,2))
21     src, dest = S, [None] * n            # make temporary storage for dest
22     for i in (2**k for k in range(logn)): # pass i creates all runs of length 2i
23         for j in range(0, n, 2*i):       # each pass merges two length i runs
24             merge(src, dest, j, i)
25         src, dest = dest, src             # reverse roles of lists
26     if S is not src:
27         S[0:n] = src[0:n]                # additional copy to get results to S

```

Code Fragment 12.4: An implementation of the nonrecursive merge-sort algorithm.

12.3 Quick-Sort

The next sorting algorithm we discuss is called *quick-sort*. Like merge-sort, this algorithm is also based on the *divide-and-conquer* paradigm, but it uses this technique in a somewhat opposite manner, as all the hard work is done *before* the recursive calls.

High-Level Description of Quick-Sort

The quick-sort algorithm sorts a sequence S using a simple recursive approach. The main idea is to apply the divide-and-conquer technique, whereby we divide S into subsequences, recur to sort each subsequence, and then combine the sorted subsequences by a simple concatenation. In particular, the quick-sort algorithm consists of the following three steps (see Figure 12.8):

1. **Divide:** If S has at least two elements (nothing needs to be done if S has zero or one element), select a specific element x from S , which is called the *pivot*. As is common practice, choose the pivot x to be the last element in S . Remove all the elements from S and put them into three sequences:
 - L , storing the elements in S less than x
 - E , storing the elements in S equal to x
 - G , storing the elements in S greater than x
 Of course, if the elements of S are distinct, then E holds just one element—the pivot itself.
2. **Conquer:** Recursively sort sequences L and G .
3. **Combine:** Put back the elements into S in order by first inserting the elements of L , then those of E , and finally those of G .

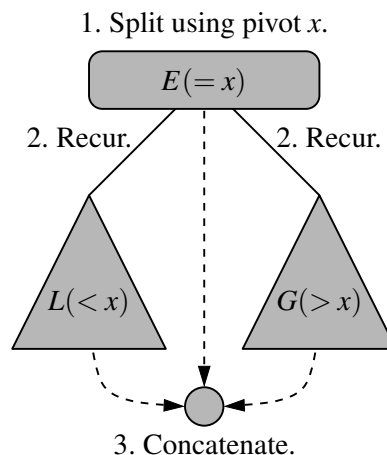


Figure 12.8: A visual schematic of the quick-sort algorithm.

Like merge-sort, the execution of quick-sort can be visualized by means of a binary recursion tree, called the *quick-sort tree*. Figure 12.9 summarizes an execution of the quick-sort algorithm by showing the input and output sequences processed at each node of the quick-sort tree. The step-by-step evolution of the quick-sort tree is shown in Figures 12.10, 12.11, and 12.12.

Unlike merge-sort, however, the height of the quick-sort tree associated with an execution of quick-sort is linear in the worst case. This happens, for example, if the sequence consists of n distinct elements and is already sorted. Indeed, in this case, the standard choice of the last element as pivot yields a subsequence L of size $n - 1$, while subsequence E has size 1 and subsequence G has size 0. At each invocation of quick-sort on subsequence L , the size decreases by 1. Hence, the height of the quick-sort tree is $n - 1$.

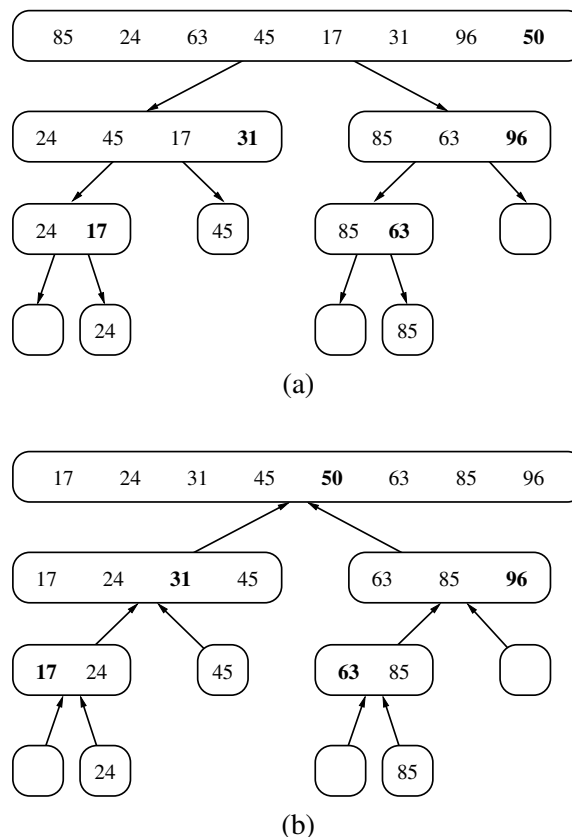


Figure 12.9: Quick-sort tree T for an execution of the quick-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T . The pivot used at each level of the recursion is shown in bold.

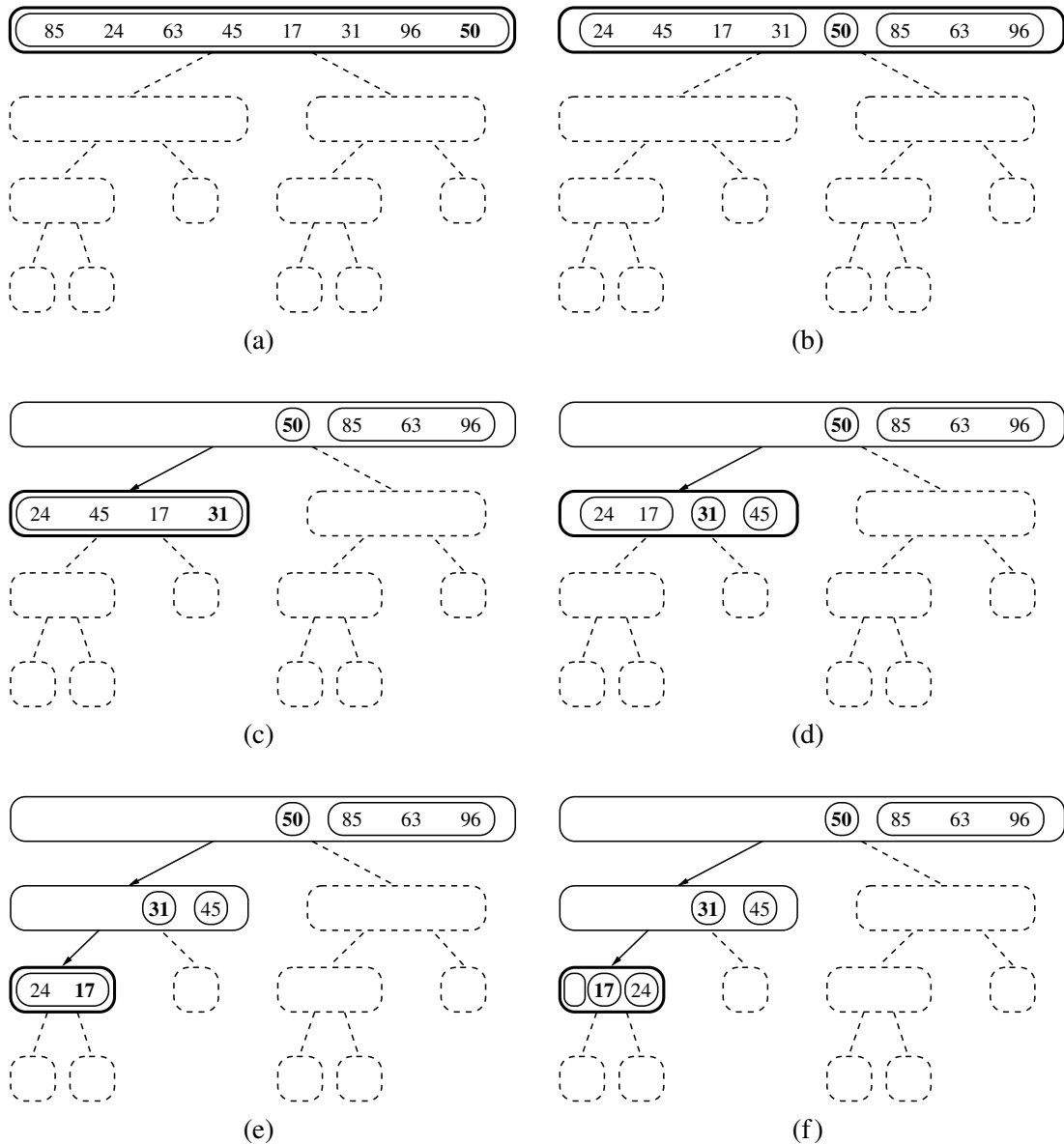


Figure 12.10: Visualization of quick-sort. Each node of the tree represents a recursive call. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the running invocation. The empty nodes drawn with thin lines represent terminated calls. The remaining nodes represent suspended calls (that is, active invocations that are waiting for a child invocation to return). Note the divide steps performed in (b), (d), and (f). (Continues in Figure 12.11.)

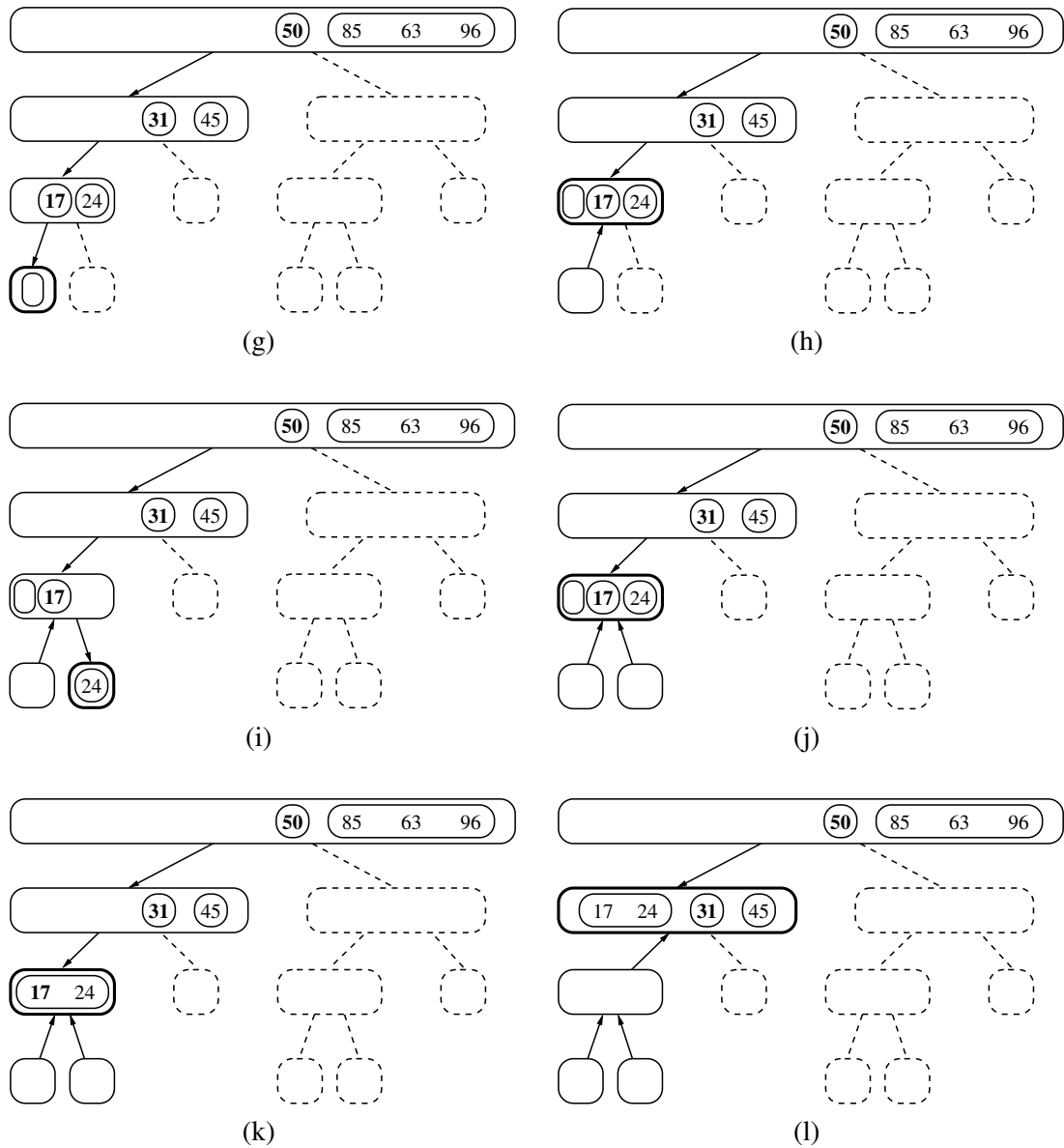


Figure 12.11: Visualization of an execution of quick-sort. Note the concatenation step performed in (k). (Continues in Figure 12.12.)

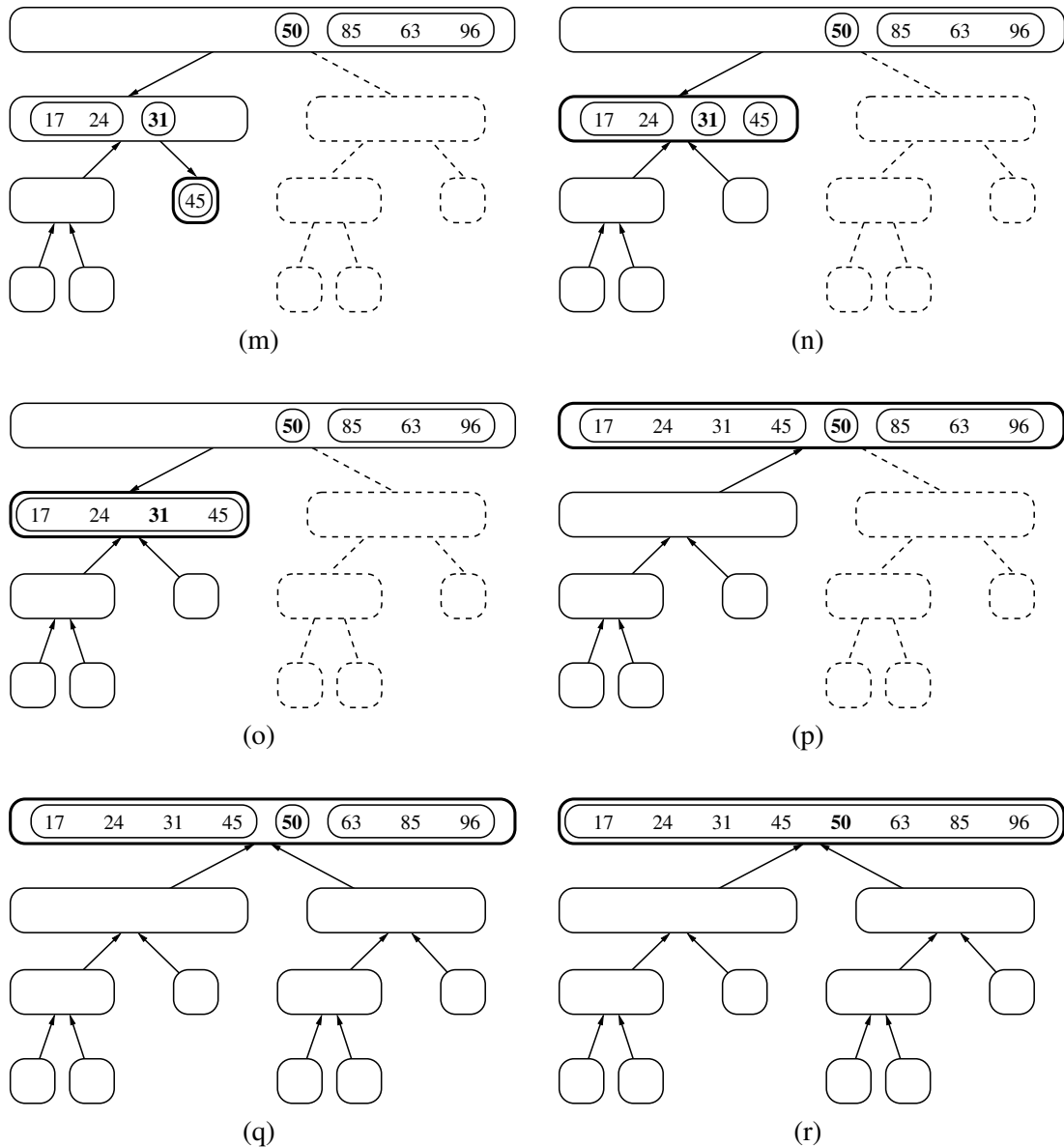


Figure 12.12: Visualization of an execution of quick-sort. Several invocations between (p) and (q) have been omitted. Note the concatenation steps performed in (o) and (r). (Continued from Figure 12.11.)

Performing Quick-Sort on General Sequences

In Code Fragment 12.5, we give an implementation of the quick-sort algorithm that works on any sequence type that operates as a queue. This particular version relies on the `LinkedQueue` class from Section 7.1.2; we provide a more streamlined implementation of quick-sort using an array-based sequence in Section 12.3.2.

Our implementation chooses the first item of the queue as the pivot (since it is easily accessible), and then it divides sequence S into queues L , E , and G of elements that are respectively less than, equal to, and greater than the pivot. We then recur on the L and G lists, and transfer elements from the sorted lists L , E , and G back to S . All of the queue operations run in $O(1)$ worst-case time when implemented with a linked list.

```

1  def quick_sort(S):
2      """Sort the elements of queue S using the quick-sort algorithm."""
3      n = len(S)
4      if n < 2:
5          return                                # list is already sorted
6      # divide
7      p = S.first( )                            # using first as arbitrary pivot
8      L = LinkedQueue()
9      E = LinkedQueue()
10     G = LinkedQueue()
11     while not S.is_empty():                    # divide S into L, E, and G
12         if S.first( ) < p:
13             L.enqueue(S.dequeue())
14         elif p < S.first():
15             G.enqueue(S.dequeue())
16         else:                                  # S.first() must equal pivot
17             E.enqueue(S.dequeue())
18     # conquer (with recursion)
19     quick_sort(L)                             # sort elements less than p
20     quick_sort(G)                             # sort elements greater than p
21     # concatenate results
22     while not L.is_empty():
23         S.enqueue(L.dequeue())
24     while not E.is_empty():
25         S.enqueue(E.dequeue())
26     while not G.is_empty():
27         S.enqueue(G.dequeue())

```

Code Fragment 12.5: Quick-sort for a sequence S implemented as a queue.

Running Time of Quick-Sort

We can analyze the running time of quick-sort with the same technique used for merge-sort in Section 12.2.3. Namely, we can identify the time spent at each node of the quick-sort tree T and sum up the running times for all the nodes.

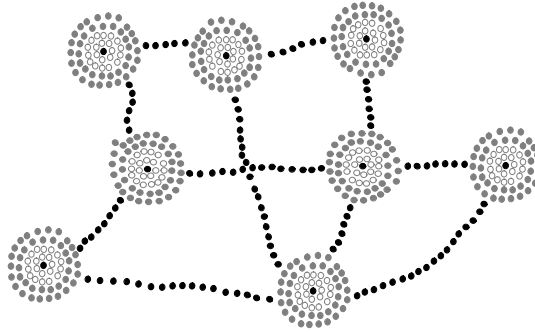
Examining Code Fragment 12.5, we see that the divide step and the final concatenation of quick-sort can be implemented in linear time. Thus, the time spent at a node v of T is proportional to the *input size* $s(v)$ of v , defined as the size of the sequence handled by the invocation of quick-sort associated with node v . Since subsequence E has at least one element (the pivot), the sum of the input sizes of the children of v is at most $s(v) - 1$.

Let s_i denote the sum of the input sizes of the nodes at depth i for a particular quick-sort tree T . Clearly, $s_0 = n$, since the root r of T is associated with the entire sequence. Also, $s_1 \leq n - 1$, since the pivot is not propagated to the children of r . More generally, it must be that $s_i < s_{i-1}$ since the elements of the subsequences at depth i all come from distinct subsequences at depth $i - 1$, and at least one element from depth $i - 1$ does not propagate to depth i because it is in a set E (in fact, one element from *each node* at depth $i - 1$ does not propagate to depth i).

We can therefore bound the overall running time of an execution of quick-sort as $O(n \cdot h)$ where h is the overall height of the quick-sort tree T for that execution. Unfortunately, in the worst case, the height of a quick-sort tree is $\Theta(n)$, as observed in Section 12.3. Thus, quick-sort runs in $O(n^2)$ worst-case time. Paradoxically, if we choose the pivot as the last element of the sequence, this worst-case behavior occurs for problem instances when sorting should be easy—if the sequence is already sorted.

Given its name, we would expect quick-sort to run quickly, and it often does in practice. The best case for quick-sort on a sequence of distinct elements occurs when subsequences L and G have roughly the same size. In that case, as we saw with merge-sort, the tree has height $O(\log n)$ and therefore quick-sort runs in $O(n \log n)$ time; we leave the justification of this fact as an exercise (R-12.10). More so, we can observe an $O(n \log n)$ running time even if the split between L and G is not as perfect. For example, if every divide step caused one subsequence to have one-fourth of those elements and the other to have three-fourths of the elements, the height of the tree would remain $O(\log n)$ and thus the overall performance $O(n \log n)$.

We will see in the next section that introducing randomization in the choice of a pivot will make quick-sort essentially behave in this way on average, with an expected running time that is $O(n \log n)$.



Contents

14.1 Graphs	620
14.1.1 The Graph ADT	626
14.2 Data Structures for Graphs	627
14.2.1 Edge List Structure	628
14.2.2 Adjacency List Structure	630
14.2.3 Adjacency Map Structure	632
14.2.4 Adjacency Matrix Structure	633
14.2.5 Python Implementation	634
14.3 Graph Traversals	638
14.3.1 Depth-First Search	639
14.3.2 DFS Implementation and Extensions	644
14.3.3 Breadth-First Search	648
14.4 Transitive Closure	651
14.5 Directed Acyclic Graphs	655
14.5.1 Topological Ordering	655
14.6 Shortest Paths	659
14.6.1 Weighted Graphs	659
14.6.2 Dijkstra's Algorithm	661
14.7 Minimum Spanning Trees	670
14.7.1 Prim-Jarník Algorithm	672
14.7.2 Kruskal's Algorithm	676
14.7.3 Disjoint Partitions and Union-Find Structures	681
14.8 Exercises	686

14.1 Graphs

A **graph** is a way of representing relationships that exist between pairs of objects. That is, a graph is a set of objects, called **vertices**, together with a collection of pairwise connections between them, called **edges**. Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering. By the way, this notion of a “graph” should not be confused with bar charts and function plots, as these kinds of “graphs” are unrelated to the topic of this chapter.

Viewed abstractly, a **graph** G is simply a set V of **vertices** and a collection E of pairs of vertices from V , called **edges**. Thus, a graph is a way of representing connections or relationships between pairs of objects from some set V . Incidentally, some books use different terminology for graphs and refer to what we call vertices as **nodes** and what we call edges as **arcs**. We use the terms “vertices” and “edges.”

Edges in a graph are either **directed** or **undirected**. An edge (u, v) is said to be **directed** from u to v if the pair (u, v) is ordered, with u preceding v . An edge (u, v) is said to be **undirected** if the pair (u, v) is not ordered. Undirected edges are sometimes denoted with set notation, as $\{u, v\}$, but for simplicity we use the pair notation (u, v) , noting that in the undirected case (u, v) is the same as (v, u) . Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles. The following are some examples of directed and undirected graphs.

Example 14.1: We can visualize collaborations among the researchers of a certain discipline by constructing a graph whose vertices are associated with the researchers themselves, and whose edges connect pairs of vertices associated with researchers who have coauthored a paper or book. (See Figure 14.1.) Such edges are undirected because coauthorship is a **symmetric** relation; that is, if A has coauthored something with B , then B necessarily has coauthored something with A .

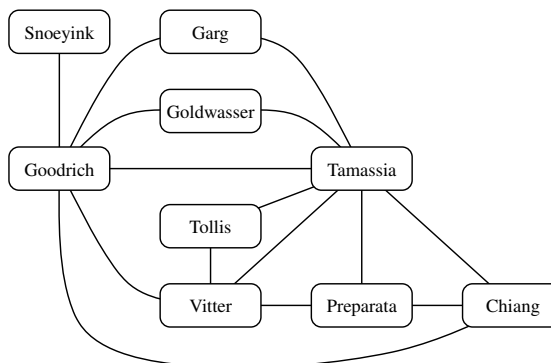


Figure 14.1: Graph of coauthorship among some authors.

Example 14.2: We can associate with an object-oriented program a graph whose vertices represent the classes defined in the program, and whose edges indicate inheritance between classes. There is an edge from a vertex v to a vertex u if the class for v inherits from the class for u . Such edges are directed because the inheritance relation only goes in one direction (that is, it is **asymmetric**).

If all the edges in a graph are undirected, then we say the graph is an **undirected graph**. Likewise, a **directed graph**, also called a **digraph**, is a graph whose edges are all directed. A graph that has both directed and undirected edges is often called a **mixed graph**. Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u, v) by the pair of directed edges (u, v) and (v, u) . It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications, as in the following example.

Example 14.3: A city map can be modeled as a graph whose vertices are intersections or dead ends, and whose edges are stretches of streets without intersections. This graph has both undirected edges, which correspond to stretches of two-way streets, and directed edges, which correspond to stretches of one-way streets. Thus, in this way, a graph modeling a city map is a mixed graph.

Example 14.4: Physical examples of graphs are present in the electrical wiring and plumbing networks of a building. Such networks can be modeled as graphs, where each connector, fixture, or outlet is viewed as a vertex, and each uninterrupted stretch of wire or pipe is viewed as an edge. Such graphs are actually components of much larger graphs, namely the local power and water distribution networks. Depending on the specific aspects of these graphs that we are interested in, we may consider their edges as undirected or directed, for, in principle, water can flow in a pipe and current can flow in a wire in either direction.

The two vertices joined by an edge are called the **end vertices** (or **endpoints**) of the edge. If an edge is directed, its first endpoint is its **origin** and the other is the **destination** of the edge. Two vertices u and v are said to be **adjacent** if there is an edge whose end vertices are u and v . An edge is said to be **incident** to a vertex if the vertex is one of the edge's endpoints. The **outgoing edges** of a vertex are the directed edges whose origin is that vertex. The **incoming edges** of a vertex are the directed edges whose destination is that vertex. The **degree** of a vertex v , denoted $\deg(v)$, is the number of incident edges of v . The **in-degree** and **out-degree** of a vertex v are the number of the incoming and outgoing edges of v , and are denoted $\text{indeg}(v)$ and $\text{outdeg}(v)$, respectively.

Example 14.5: We can study air transportation by constructing a graph G , called a **flight network**, whose vertices are associated with airports, and whose edges are associated with flights. (See Figure 14.2.) In graph G , the edges are directed because a given flight has a specific travel direction. The endpoints of an edge e in G correspond respectively to the origin and destination of the flight corresponding to e . Two airports are adjacent in G if there is a flight that flies between them, and an edge e is incident to a vertex v in G if the flight for e flies to or from the airport for v . The outgoing edges of a vertex v correspond to the outbound flights from v 's airport, and the incoming edges correspond to the inbound flights to v 's airport. Finally, the in-degree of a vertex v of G corresponds to the number of inbound flights to v 's airport, and the out-degree of a vertex v in G corresponds to the number of outbound flights.

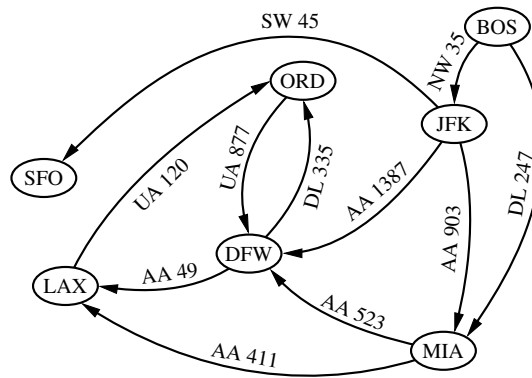


Figure 14.2: Example of a directed graph representing a flight network. The endpoints of edge UA 120 are LAX and ORD; hence, LAX and ORD are adjacent. The in-degree of DFW is 3, and the out-degree of DFW is 2.

The definition of a graph refers to the group of edges as a **collection**, not a **set**, thus allowing two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called **parallel edges** or **multiple edges**. A flight network can contain parallel edges (Example 14.5), such that multiple edges between the same pair of vertices could indicate different flights operating on the same route at different times of the day. Another special type of edge is one that connects a vertex to itself. Namely, we say that an edge (undirected or directed) is a **self-loop** if its two endpoints coincide. A self-loop may occur in a graph associated with a city map (Example 14.3), where it would correspond to a “circle” (a curving street that returns to its starting point).

With few exceptions, graphs do not have parallel edges or self-loops. Such graphs are said to be **simple**. Thus, we can usually say that the edges of a simple graph are a **set** of vertex pairs (and not just a collection). Throughout this chapter, we assume that a graph is simple unless otherwise specified.

A **path** is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex such that each edge is incident to its predecessor and successor vertex. A **cycle** is a path that starts and ends at the same vertex, and that includes at least one edge. We say that a path is **simple** if each vertex in the path is distinct, and we say that a cycle is **simple** if each vertex in the cycle is distinct, except for the first and last one. A **directed path** is a path such that all edges are directed and are traversed along their direction. A **directed cycle** is similarly defined. For example, in Figure 14.2, (BOS, NW 35, JFK, AA 1387, DFW) is a directed simple path, and (LAX, UA 120, ORD, UA 877, DFW, AA 49, LAX) is a directed simple cycle. Note that a directed graph may have a cycle consisting of two edges with opposite direction between the same pair of vertices, for example (ORD, UA 877, DFW, DL 335, ORD) in Figure 14.2. A directed graph is **acyclic** if it has no directed cycles. For example, if we were to remove the edge UA 877 from the graph in Figure 14.2, the remaining graph is acyclic. If a graph is simple, we may omit the edges when describing path P or cycle C , as these are well defined, in which case P is a list of adjacent vertices and C is a cycle of adjacent vertices.

Example 14.6: Given a graph G representing a city map (see Example 14.3), we can model a couple driving to dinner at a recommended restaurant as traversing a path through G . If they know the way, and do not accidentally go through the same intersection twice, then they traverse a simple path in G . Likewise, we can model the entire trip the couple takes, from their home to the restaurant and back, as a cycle. If they go home from the restaurant in a completely different way than how they went, not even going through the same intersection twice, then their entire round trip is a simple cycle. Finally, if they travel along one-way streets for their entire trip, we can model their night out as a directed cycle.

Given vertices u and v of a (directed) graph G , we say that u **reaches** v , and that v is **reachable** from u , if G has a (directed) path from u to v . In an undirected graph, the notion of **reachability** is symmetric, that is to say, u reaches v if and only if v reaches u . However, in a directed graph, it is possible that u reaches v but v does not reach u , because a directed path must be traversed according to the respective directions of the edges. A graph is **connected** if, for any two vertices, there is a path between them. A directed graph $\vec{G}$ is **strongly connected** if for any two vertices u and v of $\vec{G}$, u reaches v and v reaches u . (See Figure 14.3 for some examples.)

A **subgraph** of a graph G is a graph H whose vertices and edges are subsets of the vertices and edges of G , respectively. A **spanning subgraph** of G is a subgraph of G that contains all the vertices of the graph G . If a graph G is not connected, its maximal connected subgraphs are called the **connected components** of G . A **forest** is a graph without cycles. A **tree** is a connected forest, that is, a connected graph without cycles. A **spanning tree** of a graph is a spanning subgraph that is a tree. (Note that this definition of a tree is somewhat different from the one given in Chapter 8, as there is not necessarily a designated root.)

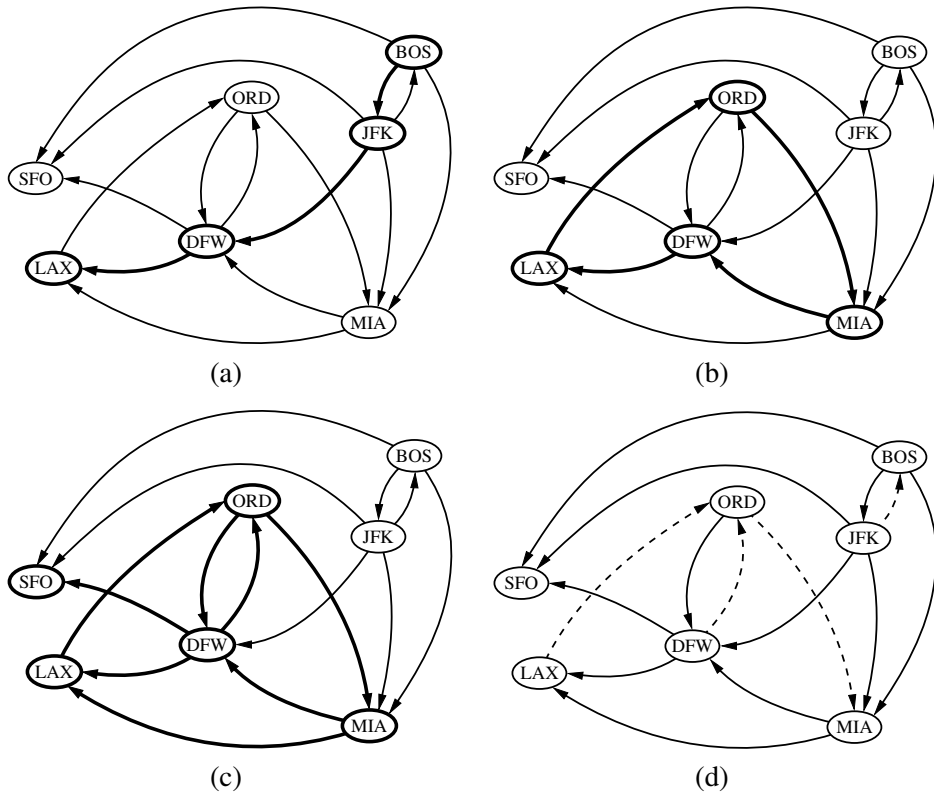


Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in an acyclic directed graph.

Example 14.7: Perhaps the most talked about graph today is the Internet, which can be viewed as a graph whose vertices are computers and whose (undirected) edges are communication connections between pairs of computers on the Internet. The computers and the connections between them in a single domain, like wiley.com, form a subgraph of the Internet. If this subgraph is connected, then two users on computers in this domain can send email to one another without having their information packets ever leave their domain. Suppose the edges of this subgraph form a spanning tree. This implies that, if even a single connection goes down (for example, because someone pulls a communication cable out of the back of a computer in this domain), then this subgraph will no longer be connected.

In the propositions that follow, we explore a few important properties of graphs.

Proposition 14.8: *If G is a graph with m edges and vertex set V , then*

$$\sum_{v \text{ in } V} \deg(v) = 2m.$$

Justification: An edge (u, v) is counted twice in the summation above; once by its endpoint u and once by its endpoint v . Thus, the total contribution of the edges to the degrees of the vertices is twice the number of edges. ■

Proposition 14.9: *If G is a directed graph with m edges and vertex set V , then*

$$\sum_{v \text{ in } V} \text{indeg}(v) = \sum_{v \text{ in } V} \text{outdeg}(v) = m.$$

Justification: In a directed graph, an edge (u, v) contributes one unit to the out-degree of its origin u and one unit to the in-degree of its destination v . Thus, the total contribution of the edges to the out-degrees of the vertices is equal to the number of edges, and similarly for the in-degrees. ■

We next show that a simple graph with n vertices has $O(n^2)$ edges.

Proposition 14.10: *Let G be a simple graph with n vertices and m edges. If G is undirected, then $m \leq n(n-1)/2$, and if G is directed, then $m \leq n(n-1)$.*

Justification: Suppose that G is undirected. Since no two edges can have the same endpoints and there are no self-loops, the maximum degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.8, $2m \leq n(n-1)$. Now suppose that G is directed. Since no two edges can have the same origin and destination, and there are no self-loops, the maximum in-degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.9, $m \leq n(n-1)$. ■

There are a number of simple properties of trees, forests, and connected graphs.

Proposition 14.11: *Let G be an undirected graph with n vertices and m edges.*

- *If G is connected, then $m \geq n-1$.*
- *If G is a tree, then $m = n-1$.*
- *If G is a forest, then $m \leq n-1$.*

14.1.1 The Graph ADT

A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph. A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume it supports a method, `element()`, to retrieve the stored element. An Edge also stores an associated object (e.g., a flight number, travel distance, cost), retrieved with the `element()` method. In addition, we assume that an Edge supports the following methods:

endpoints(): Return a tuple (u, v) such that vertex u is the origin of the edge and vertex v is the destination; for an undirected graph, the orientation is arbitrary.

opposite(v): Assuming vertex v is one endpoint of the edge (either origin or destination), return the other endpoint.

The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either *undirected* or *directed*, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge $\{u, v\}$ as a pair of directed edges (u, v) and (v, u) . The Graph ADT includes the following methods:

vertex_count(): Return the number of vertices of the graph.

vertices(): Return an iteration of all the vertices of the graph.

edge_count(): Return the number of edges of the graph.

edges(): Return an iteration of all the edges of the graph.

get_edge(u,v): Return the edge from vertex u to vertex v , if one exists; otherwise return None. For an undirected graph, there is no difference between `get_edge(u,v)` and `get_edge(v,u)`.

degree(v, out=True): For an undirected graph, return the number of edges incident to vertex v . For a directed graph, return the number of outgoing (resp. incoming) edges incident to vertex v , as designated by the optional parameter.

incident_edges(v, out=True): Return an iteration of all edges incident to vertex v . In the case of a directed graph, report outgoing edges by default; report incoming edges if the optional parameter is set to False.

insert_vertex(x=None): Create and return a new Vertex storing element x .

insert_edge(u, v, x=None): Create and return a new Edge from vertex u to vertex v , storing element x (None by default).

remove_vertex(v): Remove vertex v and all its incident edges from the graph.

remove_edge(e): Remove edge e from the graph.

14.2 Data Structures for Graphs

In this section, we introduce four data structures for representing a graph. In each representation, we maintain a collection to store the vertices of a graph. However, the four representations differ greatly in the way they organize the edges.

- In an **edge list**, we maintain an unordered list of all edges. This minimally suffices, but there is no efficient way to locate a particular edge (u, v) , or the set of all edges incident to a vertex v .
- In an **adjacency list**, we maintain, for each vertex, a separate list containing those edges that are incident to the vertex. The complete set of edges can be determined by taking the union of the smaller sets, while the organization allows us to more efficiently find all edges incident to a given vertex.
- An **adjacency map** is very similar to an adjacency list, but the secondary container of all edges incident to a vertex is organized as a map, rather than as a list, with the adjacent vertex serving as a key. This allows for access to a specific edge (u, v) in $O(1)$ expected time.
- An **adjacency matrix** provides worst-case $O(1)$ access to a specific edge (u, v) by maintaining an $n \times n$ matrix, for a graph with n vertices. Each entry is dedicated to storing a reference to the edge (u, v) for a particular pair of vertices u and v ; if no such edge exists, the entry will be None.

A summary of the performance of these structures is given in Table 14.1. We give further explanation of the structures in the remainder of this section.

Operation	Edge List	Adj. List	Adj. Map	Adj. Matrix
vertex_count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
edge_count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$	$O(m)$
get_edge(u, v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
degree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
incident_edges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
insert_vertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
remove_vertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insert_edge(u, v, x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
remove_edge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

Table 14.1: A summary of the running times for the methods of the graph ADT, using the graph representations discussed in this section. We let n denote the number of vertices, m the number of edges, and d_v the degree of vertex v . Note that the adjacency matrix uses $O(n^2)$ space, while all other structures use $O(n + m)$ space.

14.2.1 Edge List Structure

The **edge list** structure is possibly the simplest, though not the most efficient, representation of a graph G . All vertex objects are stored in an unordered list V , and all edge objects are stored in an unordered list E . We illustrate an example of the edge list structure for a graph G in Figure 14.4.

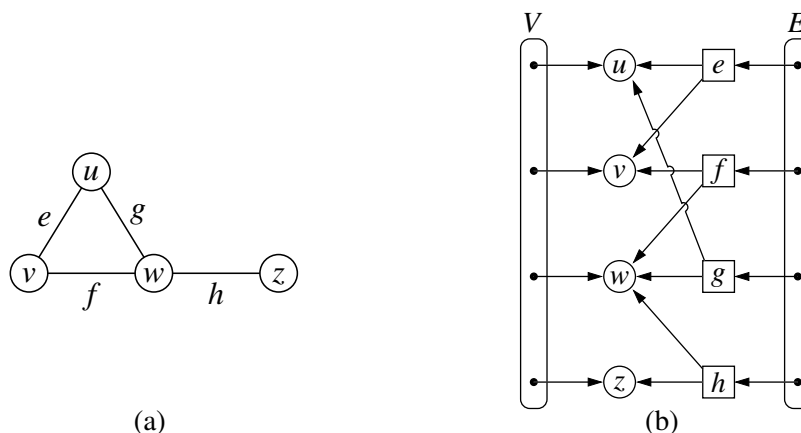


Figure 14.4: (a) A graph G ; (b) schematic representation of the edge list structure for G . Notice that an edge object refers to the two vertex objects that correspond to its endpoints, but that vertices do not refer to incident edges.

To support the many methods of the Graph ADT (Section 14.1), we assume the following additional features of an edge list representation. Collections V and E are represented with doubly linked lists using our `PositionalList` class from Chapter 7.

Vertex Objects

The vertex object for a vertex v storing element x has instance variables for:

- A reference to element x , to support the `element()` method.
- A reference to the position of the vertex instance in the list V , thereby allowing v to be efficiently removed from V if it were removed from the graph.

Edge Objects

The edge object for an edge e storing element x has instance variables for:

- A reference to element x , to support the `element()` method.
- References to the vertex objects associated with the endpoint vertices of e . These allow the edge instance to provide constant-time support for methods `endpoints()` and `opposite(v)`.
- A reference to the position of the edge instance in list E , thereby allowing e to be efficiently removed from E if it were removed from the graph.

Performance of the Edge List Structure

The performance of an edge list structure in fulfilling the graph ADT is summarized in Table 14.2. We begin by discussing the space usage, which is $O(n + m)$ for representing a graph with n vertices and m edges. Each individual vertex or edge instance uses $O(1)$ space, and the additional lists V and E use space proportional to their number of entries.

In terms of running time, the edge list structure does as well as one could hope in terms of reporting the number of vertices or edges, or in producing an iteration of those vertices or edges. By querying the respective list V or E , the `vertex_count` and `edge_count` methods run in $O(1)$ time, and by iterating through the appropriate list, the methods `vertices` and `edges` run respectively in $O(n)$ and $O(m)$ time.

The most significant limitations of an edge list structure, especially when compared to the other graph representations, are the $O(m)$ running times of methods `get_edge(u,v)`, `degree(v)`, and `incident_edges(v)`. The problem is that with all edges of the graph in an unordered list E , the only way to answer those queries is through an exhaustive inspection of all edges. The other data structures introduced in this section will implement these methods more efficiently.

Finally, we consider the methods that update the graph. It is easy to add a new vertex or a new edge to the graph in $O(1)$ time. For example, a new edge can be added to the graph by creating an `Edge` instance storing the given element as data, adding that instance to the positional list E , and recording its resulting `Position` within E as an attribute of the edge. That stored position can later be used to locate and remove this edge from E in $O(1)$ time, and thus implement the method `remove_edge(e)`.

It is worth discussing why the `remove_vertex(v)` method has a running time of $O(m)$. As stated in the graph ADT, when a vertex v is removed from the graph, all edges incident to v must also be removed (otherwise, we would have a contradiction of edges that refer to vertices that are not part of the graph). To locate the incident edges to the vertex, we must examine all edges of E .

Operation	Running Time
<code>vertex_count()</code> , <code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>get_edge(u,v)</code> , <code>degree(v)</code> , <code>incident_edges(v)</code>	$O(m)$
<code>insert_vertex(x)</code> , <code>insert_edge(u,v,x)</code> , <code>remove_edge(e)</code>	$O(1)$
<code>remove_vertex(v)</code>	$O(m)$

Table 14.2: Running times of the methods of a graph implemented with the edge list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.2 Adjacency List Structure

In contrast to the edge list representation of a graph, the **adjacency list** structure groups the edges of a graph by storing them in smaller, secondary containers that are associated with each individual vertex. Specifically, for each vertex v , we maintain a collection $I(v)$, called the **incidence collection** of v , whose entries are edges incident to v . (In the case of a directed graph, outgoing and incoming edges can be respectively stored in two separate collections, $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$.) Traditionally, the incidence collection $I(v)$ for a vertex v is a list, which is why we call this way of representing a graph the **adjacency list** structure.

We require that the primary structure for an adjacency list maintain the collection V of vertices in a way so that we can locate the secondary structure $I(v)$ for a given vertex v in $O(1)$ time. This could be done by using a positional list to represent V , with each Vertex instance maintaining a direct reference to its $I(v)$ incidence collection; we illustrate such an adjacency list structure of a graph in Figure 14.5. If vertices can be uniquely numbered from 0 to $n - 1$, we could instead use a primary array-based structure to access the appropriate secondary lists.

The primary benefit of an adjacency list is that the collection $I(v)$ contains exactly those edges that should be reported by the method `incident_edges(v)`. Therefore, we can implement this method by iterating the edges of $I(v)$ in $O(\deg(v))$ time, where $\deg(v)$ is the degree of vertex v . This is the best possible outcome for any graph representation, because there are $\deg(v)$ edges to be reported.

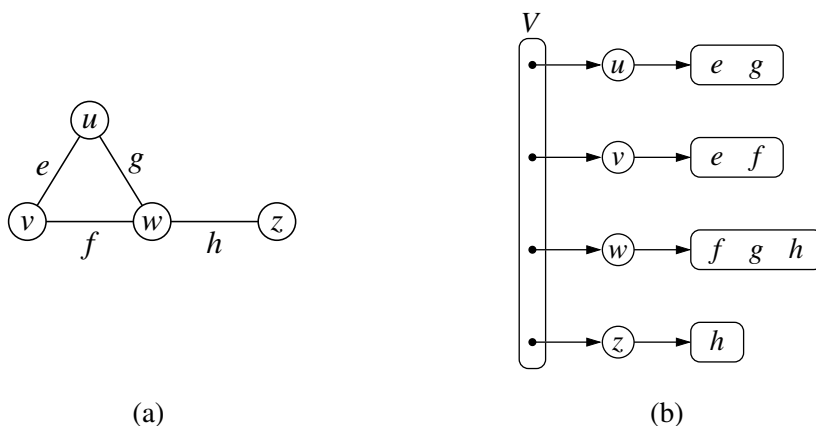


Figure 14.5: (a) An undirected graph G ; (b) a schematic representation of the adjacency list structure for G . Collection V is the primary list of vertices, and each vertex has an associated list of incident edges. Although not diagrammed as such, we presume that each edge of the graph is represented with a unique Edge instance that maintains references to its endpoint vertices.

Performance of the Adjacency List Structure

Table 14.3 summarizes the performance of the adjacency list structure implementation of a graph, assuming that the primary collection V and all secondary collections $I(v)$ are implemented with doubly linked lists.

Asymptotically, the space requirements for an adjacency list are the same as an edge list structure, using $O(n + m)$ space for a graph with n vertices and m edges. The primary list of vertices uses $O(n)$ space. The sum of the lengths of all secondary lists is $O(m)$, for reasons that were formalized in Propositions 14.8 and 14.9. In short, an undirected edge (u, v) is referenced in both $I(u)$ and $I(v)$, but its presence in the graph results in only a constant amount of additional space.

We have already noted that the `incident_edges(v)` method can be achieved in $O(\deg(v))$ time based on use of $I(v)$. We can achieve the `degree(v)` method of the graph ADT to use $O(1)$ time, assuming collection $I(v)$ can report its size in similar time. To locate a specific edge for implementing `get_edge(u, v)`, we can search through either $I(u)$ and $I(v)$. By choosing the smaller of the two, we get $O(\min(\deg(u), \deg(v)))$ running time.

The rest of the bounds in Table 14.3 can be achieved with additional care. To efficiently support deletions of edges, an edge (u, v) would need to maintain a reference to its positions within both $I(u)$ and $I(v)$, so that it could be deleted from those collections in $O(1)$ time. To remove a vertex v , we must also remove any incident edges, but at least we can locate those edges in $O(\deg(v))$ time.

The easiest way to support `edges()` in $O(m)$ and `count_edges()` in $O(1)$ is to maintain an auxiliary list E of edges, as in the edge list representation. Otherwise, we can implement the `edges` method in $O(n + m)$ time by accessing each secondary list and reporting its edges, taking care not to report an undirected edge (u, v) twice.

Operation	Running Time
<code>vertex_count()</code> , <code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>get_edge(u, v)</code>	$O(\min(\deg(u), \deg(v)))$
<code>degree(v)</code>	$O(1)$
<code>incident_edges(v)</code>	$O(\deg(v))$
<code>insert_vertex(x)</code> , <code>insert_edge(u, v, x)</code>	$O(1)$
<code>remove_edge(e)</code>	$O(1)$
<code>remove_vertex(v)</code>	$O(\deg(v))$

Table 14.3: Running times of the methods of a graph implemented with the adjacency list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.3 Adjacency Map Structure

In the adjacency list structure, we assume that the secondary incidence collections are implemented as unordered linked lists. Such a collection $I(v)$ uses space proportional to $O(\deg(v))$, allows an edge to be added or removed in $O(1)$ time, and allows an iteration of all edges incident to vertex v in $O(\deg(v))$ time. However, the best implementation of $\text{get_edge}(u, v)$ requires $O(\min(\deg(u), \deg(v)))$ time, because we must search through either $I(u)$ or $I(v)$.

We can improve the performance by using a hash-based map to implement $I(v)$ for each vertex v . Specifically, we let the opposite endpoint of each incident edge serve as a key in the map, with the edge structure serving as the value. We call such a graph representation an **adjacency map**. (See Figure 14.6.) The space usage for an adjacency map remains $O(n + m)$, because $I(v)$ uses $O(\deg(v))$ space for each vertex v , as with the adjacency list.

The advantage of the adjacency map, relative to an adjacency list, is that the $\text{get_edge}(u, v)$ method can be implemented in **expected** $O(1)$ time by searching for vertex u as a key in $I(v)$, or vice versa. This provides a likely improvement over the adjacency list, while retaining the worst-case bound of $O(\min(\deg(u), \deg(v)))$.

In comparing the performance of adjacency map to other representations (see Table 14.1), we find that it essentially achieves optimal running times for all methods, making it an excellent all-purpose choice as a graph representation.

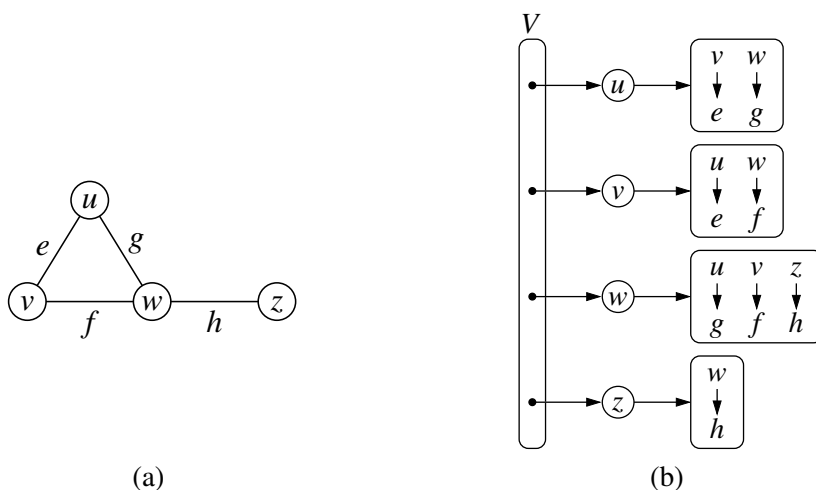


Figure 14.6: (a) An undirected graph G ; (b) a schematic representation of the adjacency map structure for G . Each vertex maintains a secondary map in which neighboring vertices serve as keys, with the connecting edges as associated values. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge of the graph, and that it maintains references to its endpoint vertices.

14.2.4 Adjacency Matrix Structure

The **adjacency matrix** structure for a graph G augments the edge list structure with a matrix A (that is, a two-dimensional array, as in Section 5.6), which allows us to locate an edge between a given pair of vertices in *worst-case* constant time. In the adjacency matrix representation, we think of the vertices as being the integers in the set $\{0, 1, \dots, n-1\}$ and the edges as being pairs of such integers. This allows us to store references to edges in the cells of a two-dimensional $n \times n$ array A . Specifically, the cell $A[i, j]$ holds a reference to the edge (u, v) , if it exists, where u is the vertex with index i and v is the vertex with index j . If there is no such edge, then $A[i, j] = \text{None}$. We note that array A is symmetric if graph G is undirected, as $A[i, j] = A[j, i]$ for all pairs i and j . (See Figure 14.7.)

The most significant advantage of an adjacency matrix is that any edge (u, v) can be accessed in worst-case $O(1)$ time; recall that the adjacency map supports that operation in $O(1)$ *expected* time. However, several operation are less efficient with an adjacency matrix. For example, to find the edges incident to vertex v , we must presumably examine all n entries in the row associated with v ; recall that an adjacency list or map can locate those edges in optimal $O(\deg(v))$ time. Adding or removing vertices from a graph is problematic, as the matrix must be resized.

Furthermore, the $O(n^2)$ space usage of an adjacency matrix is typically far worse than the $O(n + m)$ space required of the other representations. Although, in the worst case, the number of edges in a **dense** graph will be proportional to n^2 , most real-world graphs are **sparse**. In such cases, use of an adjacency matrix is inefficient. However, if a graph is dense, the constants of proportionality of an adjacency matrix can be smaller than that of an adjacency list or map. In fact, if edges do not have auxiliary data, a Boolean adjacency matrix can use one bit per edge slot, such that $A[i, j] = \text{True}$ if and only if associated (u, v) is an edge.

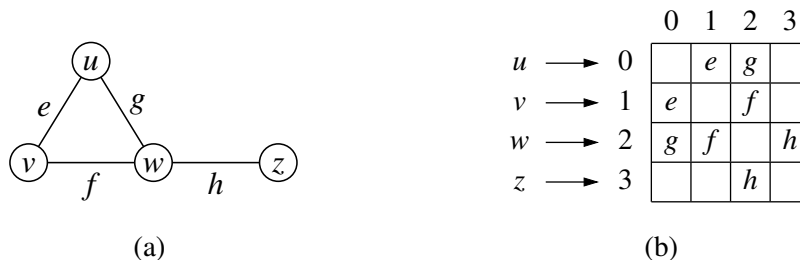


Figure 14.7: (a) An undirected graph G ; (b) a schematic representation of the auxiliary adjacency matrix structure for G , in which n vertices are mapped to indices 0 to $n-1$. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge, and that it maintains references to its endpoint vertices. We also assume that there is a secondary edge list (not pictured), to allow the `edges()` method to run in $O(m)$ time, for a graph with m edges.

14.2.5 Python Implementation

In this section, we provide an implementation of the Graph ADT. Our implementation will support directed or undirected graphs, but for ease of explanation, we first describe it in the context of an undirected graph.

We use a variant of the *adjacency map* representation. For each vertex v , we use a Python dictionary to represent the secondary incidence map $I(v)$. However, we do not explicitly maintain lists V and E , as originally described in the edge list representation. The list V is replaced by a top-level dictionary D that maps each vertex v to its incidence map $I(v)$; note that we can iterate through all vertices by generating the set of keys for dictionary D . By using such a dictionary D to map vertices to the secondary incidence maps, we need not maintain references to those incidence maps as part of the vertex structures. Also, a vertex does not need to explicitly maintain a reference to its position in D , because it can be determined in $O(1)$ expected time. This greatly simplifies our implementation. However, a consequence of our design is that some of the worst-case running time bounds for the graph ADT operations, given in Table 14.1, become *expected* bounds. Rather than maintain list E , we are content with taking the union of the edges found in the various incidence maps; technically, this runs in $O(n + m)$ time rather than strictly $O(m)$ time, as the dictionary D has n keys, even if some incidence maps are empty.

Our implementation of the graph ADT is given in Code Fragments 14.1 through 14.3. Classes `Vertex` and `Edge`, given in Code Fragment 14.1, are rather simple, and can be nested within the more complex `Graph` class. Note that we define the `__hash__` method for both `Vertex` and `Edge` so that those instances can be used as keys in Python's hash-based sets and dictionaries. The rest of the `Graph` class is given in Code Fragments 14.2 and 14.3. Graphs are undirected by default, but can be declared as directed with an optional parameter to the constructor.

Internally, we manage the directed case by having two different top-level dictionary instances, `_outgoing` and `_incoming`, such that `_outgoing[v]` maps to another dictionary representing $I_{\text{out}}(v)$, and `_incoming[v]` maps to a representation of $I_{\text{in}}(v)$. In order to unify our treatment of directed and undirected graphs, we continue to use the `_outgoing` and `_incoming` identifiers in the undirected case, yet as aliases to the same dictionary. For convenience, we define a utility named `is_directed` to allow us to distinguish between the two cases.

For methods `degree` and `incident_edges`, which each accept an optional parameter to differentiate between the outgoing and incoming orientations, we choose the appropriate map before proceeding. For method `insert_vertex`, we always initialize `_outgoing[v]` to an empty dictionary for new vertex v . In the directed case, we independently initialize `_incoming[v]` as well. For the undirected case, that step is unnecessary as `_outgoing` and `_incoming` are aliases. We leave the implementations of methods `remove_vertex` and `remove_edge` as exercises (C-14.37 and C-14.38).

```

1  #----- nested Vertex class -----
2  class Vertex:
3      """Lightweight vertex structure for a graph."""
4      __slots__ = '_element'
5
6      def __init__(self, x):
7          """Do not call constructor directly. Use Graph's insert_vertex(x)."""
8          self._element = x
9
10     def element(self):
11         """Return element associated with this vertex."""
12         return self._element
13
14     def __hash__(self):          # will allow vertex to be a map/set key
15         return hash(id(self))
16
17  #----- nested Edge class -----
18  class Edge:
19      """Lightweight edge structure for a graph."""
20      __slots__ = '_origin', '_destination', '_element'
21
22     def __init__(self, u, v, x):
23         """Do not call constructor directly. Use Graph's insert_edge(u,v,x)."""
24         self._origin = u
25         self._destination = v
26         self._element = x
27
28     def endpoints(self):
29         """Return (u,v) tuple for vertices u and v."""
30         return (self._origin, self._destination)
31
32     def opposite(self, v):
33         """Return the vertex that is opposite v on this edge."""
34         return self._destination if v is self._origin else self._origin
35
36     def element(self):
37         """Return element associated with this edge."""
38         return self._element
39
40     def __hash__(self):          # will allow edge to be a map/set key
41         return hash( (self._origin, self._destination) )

```

Code Fragment 14.1: Vertex and Edge classes (to be nested within Graph class).

```

1  class Graph:
2      """Representation of a simple graph using an adjacency map."""
3
4      def __init__(self, directed=False):
5          """Create an empty graph (undirected, by default).
6
7          Graph is directed if optional paramter is set to True.
8          """
9          self._outgoing = { }
10         # only create second map for directed graph; use alias for undirected
11         self._incoming = { } if directed else self._outgoing
12
13     def is_directed(self):
14         """Return True if this is a directed graph; False if undirected.
15
16         Property is based on the original declaration of the graph, not its contents.
17         """
18         return self._incoming is not self._outgoing # directed if maps are distinct
19
20     def vertex_count(self):
21         """Return the number of vertices in the graph."""
22         return len(self._outgoing)
23
24     def vertices(self):
25         """Return an iteration of all vertices of the graph."""
26         return self._outgoing.keys()
27
28     def edge_count(self):
29         """Return the number of edges in the graph."""
30         total = sum(len(self._outgoing[v]) for v in self._outgoing)
31         # for undirected graphs, make sure not to double-count edges
32         return total if self.is_directed( ) else total // 2
33
34     def edges(self):
35         """Return a set of all edges of the graph."""
36         result = set( ) # avoid double-reporting edges of undirected graph
37         for secondary_map in self._outgoing.values():
38             result.update(secondary_map.values()) # add edges to resulting set
39         return result

```

Code Fragment 14.2: Graph class definition (continued in Code Fragment 14.3).

```

40  def get_edge(self, u, v):
41      """Return the edge from u to v, or None if not adjacent."""
42      return self._outgoing[u].get(v)          # returns None if v not adjacent
43
44  def degree(self, v, outgoing=True):
45      """Return number of (outgoing) edges incident to vertex v in the graph.
46
47      If graph is directed, optional parameter used to count incoming edges.
48      """
49      adj = self._outgoing if outgoing else self._incoming
50      return len(adj[v])
51
52  def incident_edges(self, v, outgoing=True):
53      """Return all (outgoing) edges incident to vertex v in the graph.
54
55      If graph is directed, optional parameter used to request incoming edges.
56      """
57      adj = self._outgoing if outgoing else self._incoming
58      for edge in adj[v].values():
59          yield edge
60
61  def insert_vertex(self, x=None):
62      """Insert and return a new Vertex with element x."""
63      v = self.Vertex(x)
64      self._outgoing[v] = { }
65      if self.is_directed():
66          self._incoming[v] = { }          # need distinct map for incoming edges
67      return v
68
69  def insert_edge(self, u, v, x=None):
70      """Insert and return a new Edge from u to v with auxiliary element x."""
71      e = self.Edge(u, v, x)
72      self._outgoing[u][v] = e
73      self._incoming[v][u] = e

```

Code Fragment 14.3: Graph class definition (continued from Code Fragment 14.2). We omit error-checking of parameters for brevity.

14.3 Graph Traversals

Greek mythology tells of an elaborate labyrinth that was built to house the monstrous Minotaur, which was part bull and part man. This labyrinth was so complex that neither beast nor human could escape it. No human, that is, until the Greek hero, Theseus, with the help of the king's daughter, Ariadne, decided to implement a **graph traversal** algorithm. Theseus fastened a ball of thread to the door of the labyrinth and unwound it as he traversed the twisting passages in search of the monster. Theseus obviously knew about good algorithm design, for, after finding and defeating the beast, Theseus easily followed the string back out of the labyrinth to the loving arms of Ariadne.

Formally, a **traversal** is a systematic procedure for exploring a graph by examining all of its vertices and edges. A traversal is efficient if it visits all the vertices and edges in time proportional to their number, that is, in linear time.

Graph traversal algorithms are key to answering many fundamental questions about graphs involving the notion of **reachability**, that is, in determining how to travel from one vertex to another while following paths of a graph. Interesting problems that deal with reachability in an undirected graph G include the following:

- Computing a path from vertex u to vertex v , or reporting that no such path exists.
- Given a start vertex s of G , computing, for every vertex v of G , a path with the minimum number of edges between s and v , or reporting that no such path exists.
- Testing whether G is connected.
- Computing a spanning tree of G , if G is connected.
- Computing the connected components of G .
- Computing a cycle in G , or reporting that G has no cycles.

Interesting problems that deal with reachability in a directed graph $\vec{G}$ include the following:

- Computing a directed path from vertex u to vertex v , or reporting that no such path exists.
- Finding all the vertices of $\vec{G}$ that are reachable from a given vertex s .
- Determine whether $\vec{G}$ is acyclic.
- Determine whether $\vec{G}$ is strongly connected.

In the remainder of this section, we present two efficient graph traversal algorithms, called **depth-first search** and **breadth-first search**, respectively.

14.3.1 Depth-First Search

The first traversal algorithm we consider in this section is *depth-first search* (DFS). Depth-first search is useful for testing a number of properties of graphs, including whether there is a path from one vertex to another and whether or not a graph is connected.

Depth-first search in a graph G is analogous to wandering in a labyrinth with a string and a can of paint without getting lost. We begin at a specific starting vertex s in G , which we initialize by fixing one end of our string to s and painting s as “visited.” The vertex s is now our “current” vertex—call our current vertex u . We then traverse G by considering an (arbitrary) edge (u, v) incident to the current vertex u . If the edge (u, v) leads us to a vertex v that is already visited (that is, painted), we ignore that edge. If, on the other hand, (u, v) leads to an unvisited vertex v , then we unroll our string, and go to v . We then paint v as “visited,” and make it the current vertex, repeating the computation above. Eventually, we will get to a “dead end,” that is, a current vertex v such that all the edges incident to v lead to vertices already visited. To get out of this impasse, we roll our string back up, backtracking along the edge that brought us to v , going back to a previously visited vertex u . We then make u our current vertex and repeat the computation above for any edges incident to u that we have not yet considered. If all of u ’s incident edges lead to visited vertices, then we again roll up our string and backtrack to the vertex we came from to get to u , and repeat the procedure at that vertex. Thus, we continue to backtrack along the path that we have traced so far until we find a vertex that has yet unexplored edges, take one such edge, and continue the traversal. The process terminates when our backtracking leads us back to the start vertex s , and there are no more unexplored edges incident to s .

The pseudo-code for a depth-first search traversal starting at a vertex u (see Code Fragment 14.4) follows our analogy with string and paint. We use recursion to implement the string analogy, and we assume that we have a mechanism (the paint analogy) to determine whether a vertex or edge has been previously explored.

Algorithm DFS(G, u): {We assume u has already been marked as visited}

Input: A graph G and a vertex u of G

Output: A collection of vertices reachable from u , with their discovery edges

for each outgoing edge $e = (u, v)$ of u **do**

if vertex v has not been visited **then**

 Mark vertex v as visited (via edge e).

 Recursively call DFS(G, v).

Code Fragment 14.4: The DFS algorithm.

Classifying Graph Edges with DFS

An execution of depth-first search can be used to analyze the structure of a graph, based upon the way in which edges are explored during the traversal. The DFS process naturally identifies what is known as the *depth-first search tree* rooted at a starting vertex s . Whenever an edge $e = (u, v)$ is used to discover a new vertex v during the DFS algorithm of Code Fragment 14.4, that edge is known as a *discovery edge* or *tree edge*, as oriented from u to v . All other edges that are considered during the execution of DFS are known as *nontree edges*, which take us to a previously visited vertex. In the case of an undirected graph, we will find that all nontree edges that are explored connect the current vertex to one that is an ancestor of it in the DFS tree. We will call such an edge a *back edge*. When performing a DFS on a directed graph, there are three possible kinds of nontree edges:

- *back edges*, which connect a vertex to an ancestor in the DFS tree
- *forward edges*, which connect a vertex to a descendant in the DFS tree
- *cross edges*, which connect a vertex to a vertex that is neither its ancestor nor its descendant.

An example application of the DFS algorithm on a directed graph is shown in Figure 14.8, demonstrating each type of nontree edge. An example application of the DFS algorithm on an undirected graph is shown in Figure 14.9.

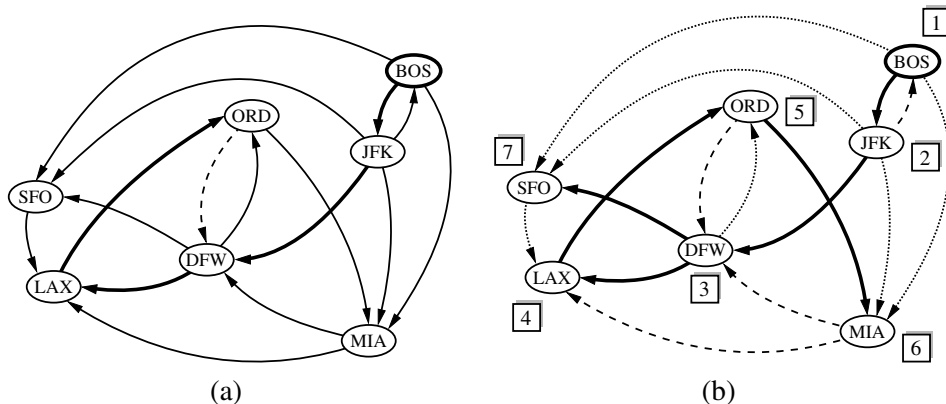


Figure 14.8: An example of a DFS in a directed graph, starting at vertex (BOS): (a) intermediate step, where, for the first time, a considered edge leads to an already visited vertex (DFW); (b) the completed DFS. The tree edges are shown with thick lines, the back edges are shown with dashed lines, and the forward and cross edges are shown with dotted lines. The order in which the vertices are visited is indicated by a label next to each vertex. The edge (ORD,DFW) is a back edge, but (DFW,ORD) is a forward edge. Edge (BOS,SFO) is a forward edge, and (SFO,LAX) is a cross edge.

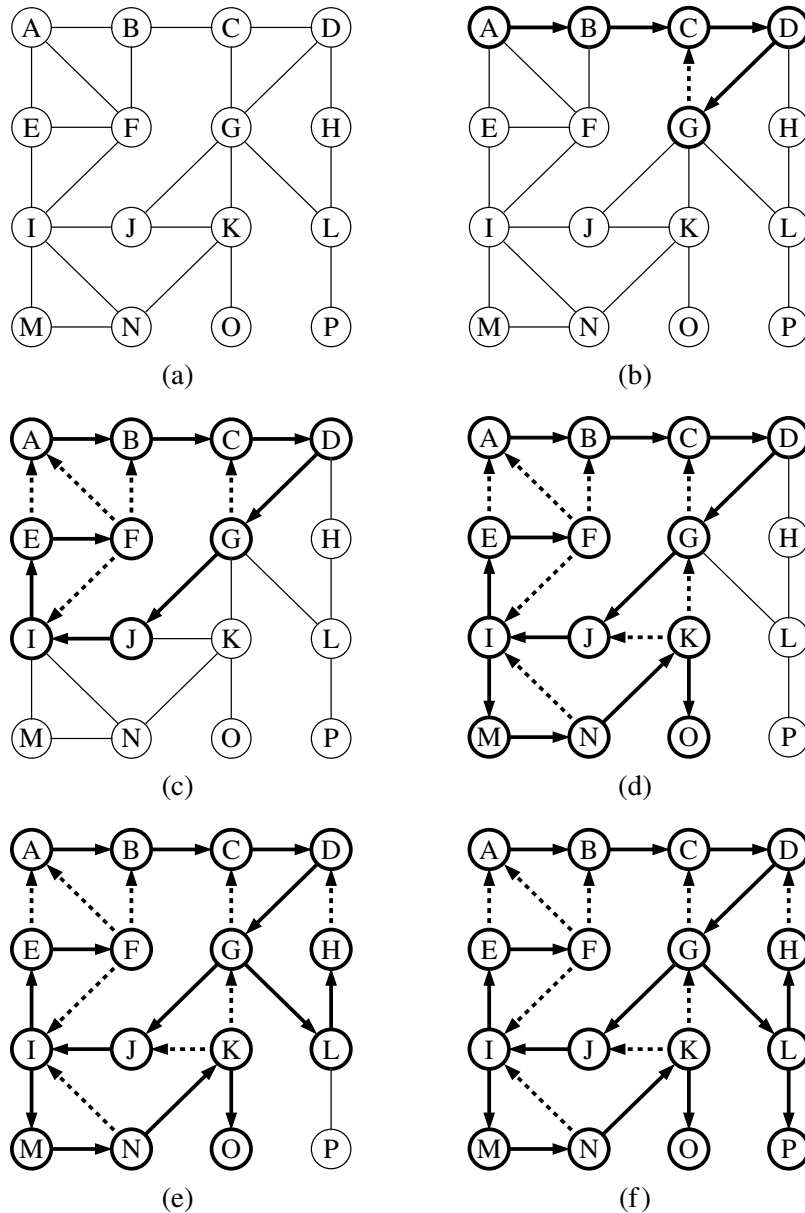


Figure 14.9: Example of depth-first search traversal on an undirected graph starting at vertex A. We assume that a vertex's adjacencies are considered in alphabetical order. Visited vertices and explored edges are highlighted, with discovery edges drawn as solid lines and nontree (back) edges as dashed lines: (a) input graph; (b) path of tree edges, traced from A until back edge (G,C) is examined; (c) reaching F, which is a dead end; (d) after backtracking to I, resuming with edge (I,M), and hitting another dead end at O; (e) after backtracking to G, continuing with edge (G,L), and hitting another dead end at H; (f) final result.

Properties of a Depth-First Search

There are a number of observations that we can make about the depth-first search algorithm, many of which derive from the way the DFS algorithm partitions the edges of a graph G into groups. We begin with the most significant property.

Proposition 14.12: *Let G be an undirected graph on which a DFS traversal starting at a vertex s has been performed. Then the traversal visits all vertices in the connected component of s , and the discovery edges form a spanning tree of the connected component of s .*

Justification: Suppose there is at least one vertex w in s 's connected component not visited, and let v be the first unvisited vertex on some path from s to w (we may have $v = w$). Since v is the first unvisited vertex on this path, it has a neighbor u that was visited. But when we visited u , we must have considered the edge (u, v) ; hence, it cannot be correct that v is unvisited. Therefore, there are no unvisited vertices in s 's connected component.

Since we only follow a discovery edge when we go to an unvisited vertex, we will never form a cycle with such edges. Therefore, the discovery edges form a connected subgraph without cycles, hence a tree. Moreover, this is a spanning tree because, as we have just seen, the depth-first search visits each vertex in the connected component of s . ■

Proposition 14.13: *Let $\vec{G}$ be a directed graph. Depth-first search on $\vec{G}$ starting at a vertex s visits all the vertices of $\vec{G}$ that are reachable from s . Also, the DFS tree contains directed paths from s to every vertex reachable from s .*

Justification: Let V_s be the subset of vertices of $\vec{G}$ visited by DFS starting at vertex s . We want to show that V_s contains s and every vertex reachable from s belongs to V_s . Suppose now, for the sake of a contradiction, that there is a vertex w reachable from s that is not in V_s . Consider a directed path from s to w , and let (u, v) be the first edge on such a path taking us out of V_s , that is, u is in V_s but v is not in V_s . When DFS reaches u , it explores all the outgoing edges of u , and thus must reach also vertex v via edge (u, v) . Hence, v should be in V_s , and we have obtained a contradiction. Therefore, V_s must contain every vertex reachable from s .

We prove the second fact by induction on the steps of the algorithm. We claim that each time a discovery edge (u, v) is identified, there exists a directed path from s to v in the DFS tree. Since u must have previously been discovered, there exists a path from s to u , so by appending the edge (u, v) to that path, we have a directed path from s to v . ■

Note that since back edges always connect a vertex v to a previously visited vertex u , each back edge implies a cycle in G , consisting of the discovery edges from u to v plus the back edge (u, v) .

Running Time of Depth-First Search

In terms of its running time, depth-first search is an efficient method for traversing a graph. Note that DFS is called at most once on each vertex (since it gets marked as visited), and therefore every edge is examined at most twice for an undirected graph, once from each of its end vertices, and at most once in a directed graph, from its origin vertex. If we let $n_s \leq n$ be the number of vertices reachable from a vertex s , and $m_s \leq m$ be the number of incident edges to those vertices, a DFS starting at s runs in $O(n_s + m_s)$ time, provided the following conditions are satisfied:

- The graph is represented by a data structure such that creating and iterating through the `incident_edges(v)` takes $O(\deg(v))$ time, and the `e.opposite(v)` method takes $O(1)$ time. The adjacency list structure is one such structure, but the adjacency matrix structure is not.
- We have a way to “mark” a vertex or edge as explored, and to test if a vertex or edge has been explored in $O(1)$ time. We discuss ways of implementing DFS to achieve this goal in the next section.

Given the assumptions above, we can solve a number of interesting problems.

Proposition 14.14: *Let G be an undirected graph with n vertices and m edges. A DFS traversal of G can be performed in $O(n + m)$ time, and can be used to solve the following problems in $O(n + m)$ time:*

- *Computing a path between two given vertices of G , if one exists.*
- *Testing whether G is connected.*
- *Computing a spanning tree of G , if G is connected.*
- *Computing the connected components of G .*
- *Computing a cycle in G , or reporting that G has no cycles.*

Proposition 14.15: *Let $\vec{G}$ be a directed graph with n vertices and m edges. A DFS traversal of $\vec{G}$ can be performed in $O(n + m)$ time, and can be used to solve the following problems in $O(n + m)$ time:*

- *Computing a directed path between two given vertices of $\vec{G}$, if one exists.*
- *Computing the set of vertices of $\vec{G}$ that are reachable from a given vertex s .*
- *Testing whether $\vec{G}$ is strongly connected.*
- *Computing a directed cycle in $\vec{G}$, or reporting that $\vec{G}$ is acyclic.*
- *Computing the **transitive closure** of $\vec{G}$ (see Section 14.4).*

The justification of Propositions 14.14 and 14.15 is based on algorithms that use slightly modified versions of the DFS algorithm as subroutines. We will explore some of those extensions in the remainder of this section.

14.3.2 DFS Implementation and Extensions

We begin by providing a Python implementation of the basic depth-first search algorithm, originally described with pseudo-code in Code Fragment 14.4. Our DFS function is presented in Code Fragment 14.5.

```

1 def DFS(g, u, discovered):
2     """ Perform DFS of the undiscovered portion of Graph g starting at Vertex u.
3
4     discovered is a dictionary mapping each vertex to the edge that was used to
5     discover it during the DFS. (u should be "discovered" prior to the call.)
6     Newly discovered vertices will be added to the dictionary as a result.
7     """
8     for e in g.incident_edges(u):           # for every outgoing edge from u
9         v = e.opposite(u)
10        if v not in discovered:             # v is an unvisited vertex
11            discovered[v] = e               # e is the tree edge that discovered v
12            DFS(g, v, discovered)          # recursively explore from v

```

Code Fragment 14.5: Recursive implementation of depth-first search on a graph, starting at a designated vertex u .

In order to track which vertices have been visited, and to build a representation of the resulting DFS tree, our implementation introduces a third parameter, named *discovered*. This parameter should be a Python dictionary that maps a vertex of the graph to the tree edge that was used to discover that vertex. As a technicality, we assume that the source vertex u occurs as a key of the dictionary, with *None* as its value. Thus, a caller might start the traversal as follows:

```

result = {u : None}           # a new dictionary, with u trivially discovered
DFS(g, u, result)

```

The dictionary serves two purposes. Internally, the dictionary provides a mechanism for recognizing visited vertices, as they will appear as keys in the dictionary. Externally, the DFS function augments this dictionary as it proceeds, and thus the values within the dictionary are the DFS tree edges at the conclusion of the process.

Because the dictionary is hash-based, the test, “**if v not in discovered**,” and the record-keeping step, “*discovered[v] = e*,” run in $O(1)$ *expected* time, rather than worst-case time. In practice, this is a compromise we are willing to accept, but it does violate the formal analysis of the algorithm, as given on page 643. If we could assume that vertices could be numbered from 0 to $n - 1$, then those numbers could be used as indices into an array-based lookup table rather than a hash-based map. Alternatively, we could store each vertex’s discovery status and associated tree edge directly as part of the vertex instance.

Reconstructing a Path from u to v

We can use the basic DFS function as a tool to identify the (directed) path leading from vertex u to v , if v is reachable from u . This path can easily be reconstructed from the information that was recorded in the discovery dictionary during the traversal. Code Fragment 14.6 provides an implementation of a secondary function that produces an ordered list of vertices on the path from u to v .

To reconstruct the path, we begin at the *end* of the path, examining the discovery dictionary to determine what edge was used to reach vertex v , and then what the other endpoint of that edge is. We add that vertex to a list, and then repeat the process to determine what edge was used to discover it. Once we have traced the path all the way back to the starting vertex u , we can reverse the list so that it is properly oriented from u to v , and return it to the caller. This process takes time proportional to the length of the path, and therefore it runs in $O(n)$ time (in addition to the time originally spent calling DFS).

```

1  def construct_path(u, v, discovered):
2      path = [ ]                                # empty path by default
3      if v in discovered:
4          # we build list from v to u and then reverse it at the end
5          path.append(v)
6          walk = v
7          while walk is not u:
8              e = discovered[walk]                # find edge leading to walk
9              parent = e.opposite(walk)
10             path.append(parent)
11             walk = parent
12         path.reverse( )                        # reorient path from u to v
13     return path

```

Code Fragment 14.6: Function to reconstruct a directed path from u to v , given the trace of discovery from a DFS started at u . The function returns an ordered list of vertices on the path.

Testing for Connectivity

We can use the basic DFS function to determine whether a graph is connected. In the case of an undirected graph, we simply start a depth-first search at an arbitrary vertex and then test whether `len(discovered)` equals n at the conclusion. If the graph is connected, then by Proposition 14.12, all vertices will have been discovered; conversely, if the graph is not connected, there must be at least one vertex v that is not reachable from u , and that will not be discovered.

For directed graph, $\vec{G}$, we may wish to test whether it is **strongly connected**, that is, whether for every pair of vertices u and v , both u reaches v and v reaches u . If we start an independent call to DFS from each vertex, we could determine whether this was the case, but those n calls when combined would run in $O(n(n+m))$. However, we can determine if $\vec{G}$ is strongly connected much faster than this, requiring only two depth-first searches.

We begin by performing a depth-first search of our directed graph $\vec{G}$ starting at an arbitrary vertex s . If there is any vertex of $\vec{G}$ that is not visited by this traversal, and is not reachable from s , then the graph is not strongly connected. If this first depth-first search visits each vertex of $\vec{G}$, we need to then check whether s is reachable from all other vertices. Conceptually, we can accomplish this by making a copy of graph $\vec{G}$, but with the orientation of all edges reversed. A depth-first search starting at s in the reversed graph will reach every vertex that could reach s in the original. In practice, a better approach than making a new graph is to reimplement a version of the DFS method that loops through all **incoming** edges to the current vertex, rather than all **outgoing** edges. Since this algorithm makes just two DFS traversals of $\vec{G}$, it runs in $O(n+m)$ time.

Computing all Connected Components

When a graph is not connected, the next goal we may have is to identify all of the **connected components** of an undirected graph, or the **strongly connected components** of a directed graph. We begin by discussing the undirected case.

If an initial call to DFS fails to reach all vertices of a graph, we can restart a new call to DFS at one of those unvisited vertices. An implementation of such a comprehensive DFS_all method is given in Code Fragment 14.7.

```

1 def DFS_complete(g):
2     """ Perform DFS for entire graph and return forest as a dictionary.
3
4     Result maps each vertex v to the edge that was used to discover it.
5     (Vertices that are roots of a DFS tree are mapped to None.)
6     """
7     forest = { }
8     for u in g.vertices():
9         if u not in forest:
10             forest[u] = None           # u will be the root of a tree
11             DFS(g, u, forest)
12     return forest

```

Code Fragment 14.7: Top-level function that returns a DFS forest for an entire graph.

Although the `DFS_complete` function makes multiple calls to the original DFS function, the total time spent by a call to `DFS_complete` is $O(n + m)$. For an undirected graph, recall from our original analysis on page 643 that a single call to DFS starting at vertex s runs in time $O(n_s + m_s)$ where n_s is the number of vertices reachable from s , and m_s is the number of incident edges to those vertices. Because each call to DFS explores a different component, the sum of $n_s + m_s$ terms is $n + m$. The $O(n + m)$ total bound applies to the directed case as well, even though the sets of reachable vertices are not necessarily disjoint. However, because the same discovery dictionary is passed as a parameter to all DFS calls, we know that the DFS subroutine is called once on each vertex, and then each outgoing edge is explored only once during the process.

The `DFS_complete` function can be used to analyze the connected components of an undirected graph. The discovery dictionary it returns represents a **DFS forest** for the entire graph. We say this is a forest rather than a tree, because the graph may not be connected. The number of connected components can be determined by the number of vertices in the discovery dictionary that have `None` as their discovery edge (those are roots of DFS trees). A minor modification to the core DFS method could be used to tag each vertex with a component number when it is discovered. (See Exercise C-14.44.)

The situation is more complex for finding strongly connected components of a directed graph. There exists an approach for computing those components in $O(n + m)$ time, making use of two separate depth-first search traversals, but the details are beyond the scope of this book.

Detecting Cycles with DFS

For both undirected and directed graphs, a cycle exists if and only if a **back edge** exists relative to the DFS traversal of that graph. It is easy to see that if a back edge exists, a cycle exists by taking the back edge from the descendant to its ancestor and then following the tree edges back to the descendant. Conversely, if a cycle exists in the graph, there must be a back edge relative to a DFS (although we do not prove this fact here).

Algorithmically, detecting a back edge in the undirected case is easy, because all edges are either tree edges or back edges. In the case of a directed graph, additional modifications to the core DFS implementation are needed to properly categorize a nontree edge as a back edge. When a directed edge is explored leading to a previously visited vertex, we must recognize whether that vertex is an ancestor of the current vertex. This requires some additional bookkeeping, for example, by tagging vertices upon which a recursive call to DFS is still active. We leave details as an exercise (C-14.43).

14.3.3 Breadth-First Search

The advancing and backtracking of a depth-first search, as described in the previous section, defines a traversal that could be physically traced by a single person exploring a graph. In this section, we consider another algorithm for traversing a connected component of a graph, known as a *breadth-first search* (BFS). The BFS algorithm is more akin to sending out, in all directions, many explorers who collectively traverse a graph in coordinated fashion.

A BFS proceeds in rounds and subdivides the vertices into *levels*. BFS starts at vertex s , which is at level 0. In the first round, we paint as “visited,” all vertices adjacent to the start vertex s —these vertices are one step away from the beginning and are placed into level 1. In the second round, we allow all explorers to go two steps (i.e., edges) away from the starting vertex. These new vertices, which are adjacent to level 1 vertices and not previously assigned to a level, are placed into level 2 and marked as “visited.” This process continues in similar fashion, terminating when no new vertices are found in a level.

A Python implementation of BFS is given in Code Fragment 14.8. We follow a convention similar to that of DFS (Code Fragment 14.5), using a discovered dictionary both to recognize visited vertices, and to record the discovery edges of the BFS tree. We illustrate a BFS traversal in Figure 14.10.

```

1 def BFS(g, s, discovered):
2     """ Perform BFS of the undiscovered portion of Graph g starting at Vertex s.
3
4     discovered is a dictionary mapping each vertex to the edge that was used to
5     discover it during the BFS (s should be mapped to None prior to the call).
6     Newly discovered vertices will be added to the dictionary as a result.
7     """
8     level = [s]                                # first level includes only s
9     while len(level) > 0:
10         next_level = [ ]                        # prepare to gather newly found vertices
11         for u in level:
12             for e in g.incident_edges(u):      # for every outgoing edge from u
13                 v = e.opposite(u)
14                 if v not in discovered:        # v is an unvisited vertex
15                     discovered[v] = e         # e is the tree edge that discovered v
16                     next_level.append(v)      # v will be further considered in next pass
17         level = next_level                    # relabel 'next' level to become current

```

Code Fragment 14.8: Implementation of breadth-first search on a graph, starting at a designated vertex s .

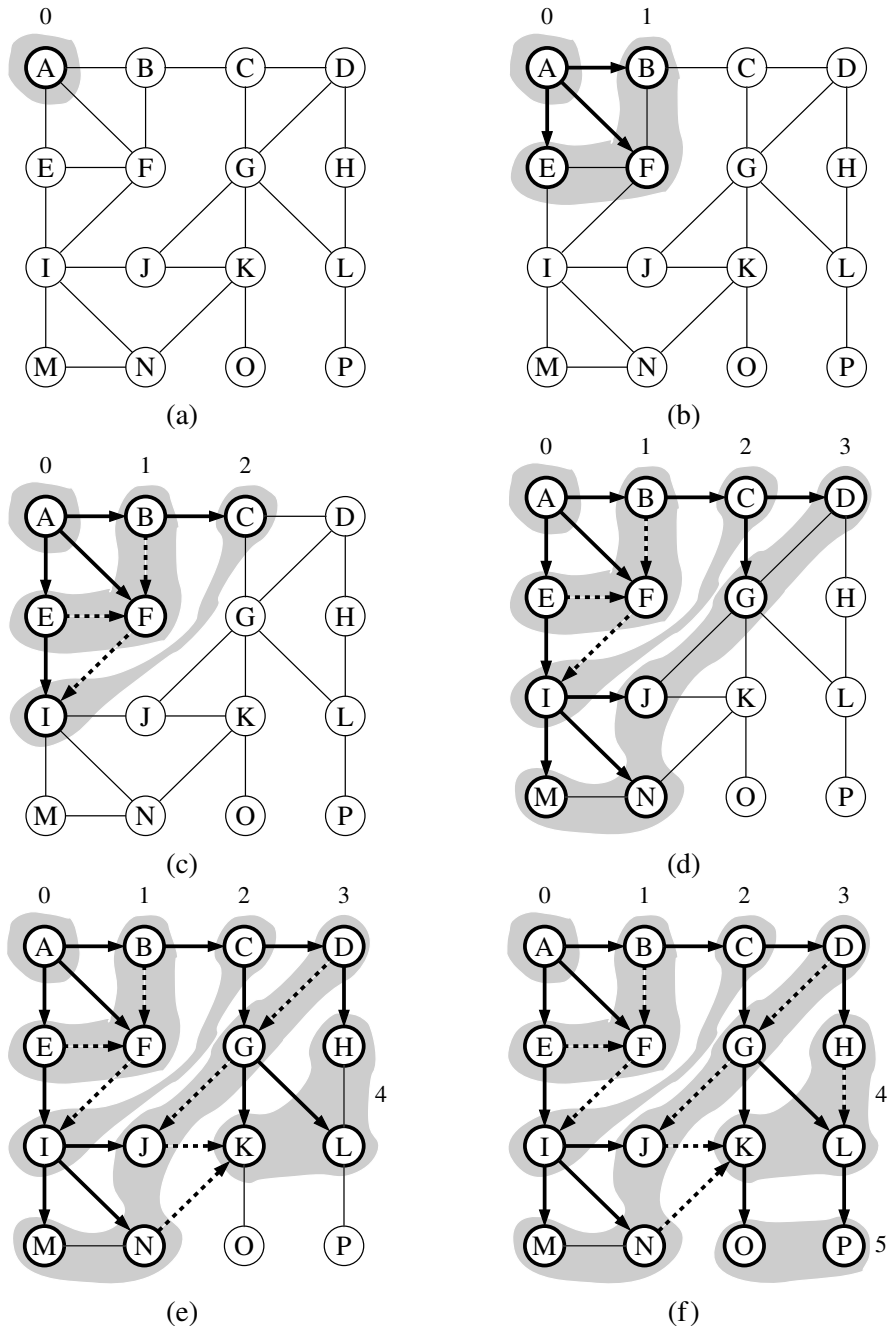


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.

When discussing DFS, we described a classification of nontree edges being either *back edges*, which connect a vertex to one of its ancestors, *forward edges*, which connect a vertex to one of its descendants, or *cross edges*, which connect a vertex to another vertex that is neither its ancestor nor its descendant. For BFS on an undirected graph, all nontree edges are cross edges (see Exercise C-14.47), and for BFS on a directed graph, all nontree edges are either back edges or cross edges (see Exercise C-14.48).

The BFS traversal algorithm has a number of interesting properties, some of which we explore in the proposition that follows. Most notably, a path in a breadth-first search tree rooted at vertex s to any other vertex v is guaranteed to be the shortest such path from s to v in terms of the number of edges.

Proposition 14.16: *Let G be an undirected or directed graph on which a BFS traversal starting at vertex s has been performed. Then*

- *The traversal visits all vertices of G that are reachable from s .*
- *For each vertex v at level i , the path of the BFS tree T between s and v has i edges, and any other path of G from s to v has at least i edges.*
- *If (u, v) is an edge that is not in the BFS tree, then the level number of v can be at most 1 greater than the level number of u .*

We leave the justification of this proposition as an exercise (C-14.50).

The analysis of the running time of BFS is similar to the one of DFS, with the algorithm running in $O(n + m)$ time, or more specifically, in $O(n_s + m_s)$ time if n_s is the number of vertices reachable from vertex s , and $m_s \leq m$ is the number of incident edges to those vertices. To explore the entire graph, the process can be restarted at another vertex, akin to the DFS_complete function of Code Fragment 14.7. Also, the actual path from vertex s to vertex v can be reconstructed using the construct_path function of Code Fragment 14.6

Proposition 14.17: *Let G be a graph with n vertices and m edges represented with the adjacency list structure. A BFS traversal of G takes $O(n + m)$ time.*

Although our implementation of BFS in Code Fragment 14.8 progresses level by level, the BFS algorithm can also be implemented using a single FIFO queue to represent the current fringe of the search. Starting with the source vertex in the queue, we repeatedly remove the vertex from the front of the queue and insert any of its unvisited neighbors to the back of the queue. (See Exercise C-14.51.)

In comparing the capabilities of DFS and BFS, both can be used to efficiently find the set of vertices that are reachable from a given source, and to determine paths to those vertices. However, BFS guarantees that those paths use as few edges as possible. For an undirected graph, both algorithms can be used to test connectivity, to identify connected components, or to locate a cycle. For directed graphs, the DFS algorithm is better suited for certain tasks, such as finding a directed cycle in the graph, or in identifying the strongly connected components.

14.4 Transitive Closure

We have seen that graph traversals can be used to answer basic questions of reachability in a directed graph. In particular, if we are interested in knowing whether there is a path from vertex u to vertex v in a graph, we can perform a DFS or BFS traversal starting at u and observe whether v is discovered. If representing a graph with an adjacency list or adjacency map, we can answer the question of reachability for u and v in $O(n + m)$ time (see Propositions 14.15 and 14.17).

In certain applications, we may wish to answer many reachability queries more efficiently, in which case it may be worthwhile to precompute a more convenient representation of a graph. For example, the first step for a service that computes driving directions from an origin to a destination might be to assess whether the destination is reachable. Similarly, in an electricity network, we may wish to be able to quickly determine whether current flows from one particular vertex to another. Motivated by such applications, we introduce the following definition. The **transitive closure** of a directed graph $\vec{G}$ is itself a directed graph $\vec{G}^*$ such that the vertices of $\vec{G}^*$ are the same as the vertices of $\vec{G}$, and $\vec{G}^*$ has an edge (u, v) , whenever $\vec{G}$ has a directed path from u to v (including the case where (u, v) is an edge of the original $\vec{G}$).

If a graph is represented as an adjacency list or adjacency map, we can compute its transitive closure in $O(n(n + m))$ time by making use of n graph traversals, one from each starting vertex. For example, a DFS starting at vertex u can be used to determine all vertices reachable from u , and thus a collection of edges originating with u in the transitive closure.

In the remainder of this section, we explore an alternative technique for computing the transitive closure of a directed graph that is particularly well suited for when a directed graph is represented by a data structure that supports $O(1)$ -time lookup for the `get_edge(u, v)` method (for example, the adjacency-matrix structure). Let $\vec{G}$ be a directed graph with n vertices and m edges. We compute the transitive closure of $\vec{G}$ in a series of rounds. We initialize $\vec{G}_0 = \vec{G}$. We also arbitrarily number the vertices of $\vec{G}$ as $v_1, v_2, \dots, v_n$. We then begin the computation of the rounds, beginning with round 1. In a generic round k , we construct directed graph $\vec{G}_k$ starting with $\vec{G}_k = \vec{G}_{k-1}$ and adding to $\vec{G}_k$ the directed edge (v_i, v_j) if directed graph $\vec{G}_{k-1}$ contains both the edges (v_i, v_k) and (v_k, v_j) . In this way, we will enforce a simple rule embodied in the proposition that follows.

Proposition 14.18: *For $i = 1, \dots, n$, directed graph $\vec{G}_k$ has an edge (v_i, v_j) if and only if directed graph $\vec{G}$ has a directed path from v_i to v_j , whose intermediate vertices (if any) are in the set $\{v_1, \dots, v_k\}$. In particular, $\vec{G}_n$ is equal to $\vec{G}^*$, the transitive closure of $\vec{G}$.*

Proposition 14.18 suggests a simple algorithm for computing the transitive closure of $\vec{G}$ that is based on the series of rounds to compute each $\vec{G}_k$. This algorithm is known as the **Floyd-Warshall algorithm**, and its pseudo-code is given in Code Fragment 14.9. We illustrate an example run of the Floyd-Warshall algorithm in Figure 14.11.

Algorithm FloydWarshall($\vec{G}$):

Input: A directed graph $\vec{G}$ with n vertices

Output: The transitive closure $\vec{G}^*$ of $\vec{G}$

let $v_1, v_2, \dots, v_n$ be an arbitrary numbering of the vertices of $\vec{G}$

$\vec{G}_0 = \vec{G}$

for $k = 1$ to n **do**

$\vec{G}_k = \vec{G}_{k-1}$

for all i, j in $\{1, \dots, n\}$ with $i \neq j$ and $i, j \neq k$ **do**

if both edges (v_i, v_k) and (v_k, v_j) are in $\vec{G}_{k-1}$ **then**

add edge (v_i, v_j) to $\vec{G}_k$ (if it is not already present)

return $\vec{G}_n$

Code Fragment 14.9: Pseudo-code for the Floyd-Warshall algorithm. This algorithm computes the transitive closure $\vec{G}^*$ of G by incrementally computing a series of directed graphs $\vec{G}_0, \vec{G}_1, \dots, \vec{G}_n$, for $k = 1, \dots, n$.

From this pseudo-code, we can easily analyze the running time of the Floyd-Warshall algorithm assuming that the data structure representing G supports methods `get_edge` and `insert_edge` in $O(1)$ time. The main loop is executed n times and the inner loop considers each of $O(n^2)$ pairs of vertices, performing a constant-time computation for each one. Thus, the total running time of the Floyd-Warshall algorithm is $O(n^3)$. From the description and analysis above we may immediately derive the following proposition.

Proposition 14.19: Let $\vec{G}$ be a directed graph with n vertices, and let $\vec{G}$ be represented by a data structure that supports lookup and update of adjacency information in $O(1)$ time. Then the Floyd-Warshall algorithm computes the transitive closure $\vec{G}^*$ of $\vec{G}$ in $O(n^3)$ time.

Performance of the Floyd-Warshall Algorithm

Asymptotically, the $O(n^3)$ running time of the Floyd-Warshall algorithm is no better than that achieved by repeatedly running DFS, once from each vertex, to compute the reachability. However, the Floyd-Warshall algorithm matches the asymptotic bounds of the repeated DFS when a graph is dense, or when a graph is sparse but represented as an adjacency matrix. (See Exercise R-14.12.)

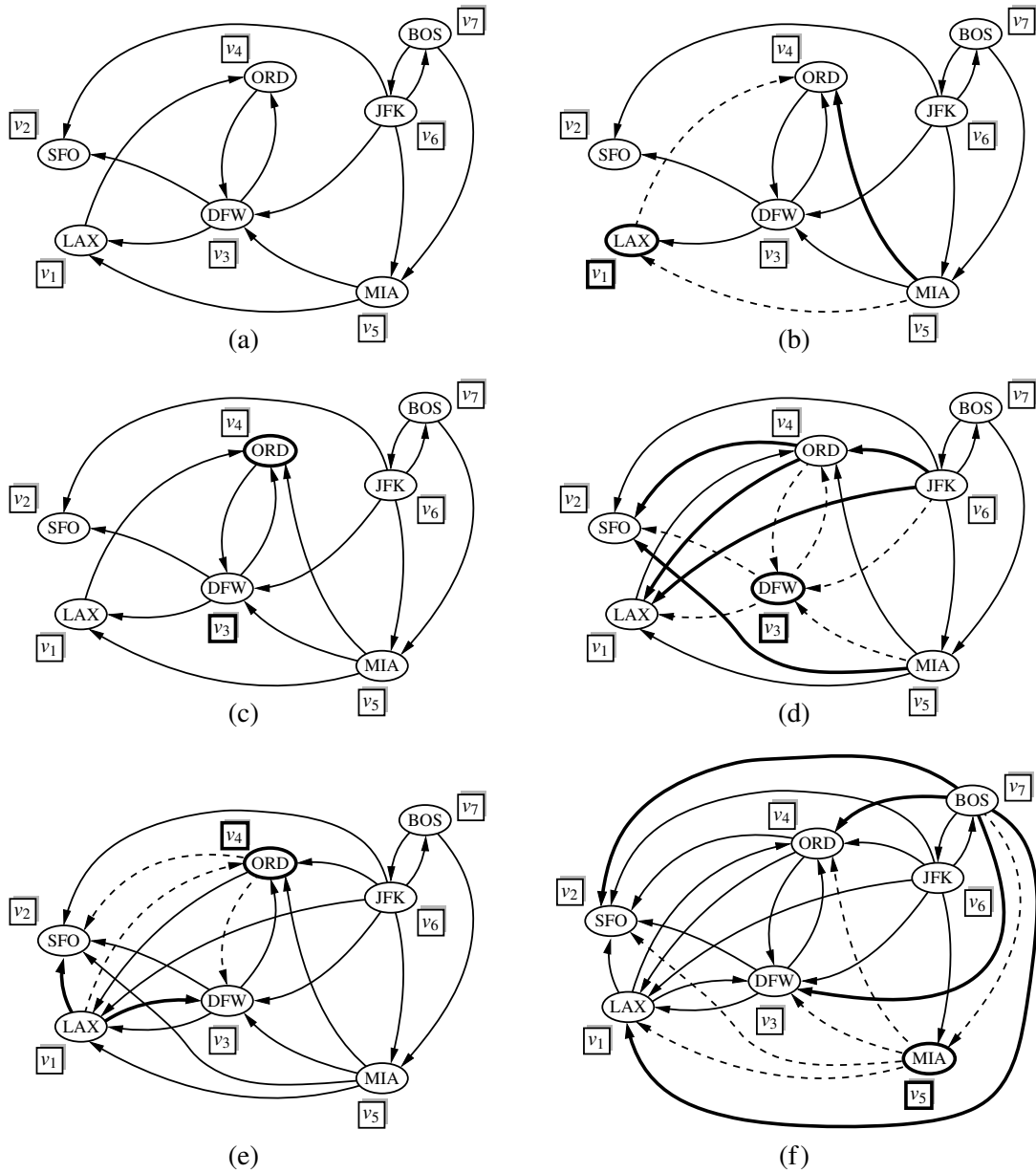


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j) , but not the edge (v_i, v_j) , in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA, LAX) and (LAX, ORD) result in new edge (MIA, ORD) .

The importance of the Floyd-Warshall algorithm is that it is much easier to implement than DFS, and much faster in practice because there are relatively few low-level operations hidden within the asymptotic notation. The algorithm is particularly well suited for the use of an adjacency matrix, as a single bit can be used to designate the reachability modeled as an edge (u, v) in the transitive closure.

However, note that repeated calls to DFS results in better asymptotic performance when the graph is sparse and represented using an adjacency list or adjacency map. In that case, a single DFS runs in $O(n + m)$ time, and so the transitive closure can be computed in $O(n^2 + nm)$ time, which is preferable to $O(n^3)$.

Python Implementation

We conclude with a Python implementation of the Floyd-Warshall algorithm, as presented in Code Fragment 14.10. Although the original algorithm is described using a series of directed graphs $\vec{G}_0, \vec{G}_1, \dots, \vec{G}_n$, we create a single copy of the original graph (using the `deepcopy` method of Python's `copy` module) and then repeatedly add new edges to the closure as we progress through rounds of the Floyd-Warshall algorithm.

The algorithm requires a canonical numbering of the graph's vertices; therefore, we create a list of the vertices in the closure graph, and subsequently index that list for our order. Within the outermost loop, we must consider all pairs i and j . Finally, we optimize by only iterating through all values of j after we have verified that i has been chosen such that (v_i, v_k) exists in the current version of our closure.

```

1 def floyd_warshall(g):
2     """ Return a new graph that is the transitive closure of g. """
3     closure = deepcopy(g)                # imported from copy module
4     verts = list(closure.vertices())      # make indexable list
5     n = len(verts)
6     for k in range(n):
7         for i in range(n):
8             # verify that edge (i,k) exists in the partial closure
9             if i != k and closure.get_edge(verts[i],verts[k]) is not None:
10                for j in range(n):
11                    # verify that edge (k,j) exists in the partial closure
12                    if i != j != k and closure.get_edge(verts[k],verts[j]) is not None:
13                        # if (i,j) not yet included, add it to the closure
14                        if closure.get_edge(verts[i],verts[j]) is None:
15                            closure.insert_edge(verts[i],verts[j])
16    return closure

```

Code Fragment 14.10: Python implementation of the Floyd-Warshall algorithm.

14.5 Directed Acyclic Graphs

Directed graphs without directed cycles are encountered in many applications. Such a directed graph is often referred to as a **directed acyclic graph**, or **DAG**, for short. Applications of such graphs include the following:

- Prerequisites between courses of a degree program.
- Inheritance between classes of an object-oriented program.
- Scheduling constraints between the tasks of a project.

We explore this latter application further in the following example:

Example 14.20: *In order to manage a large project, it is convenient to break it up into a collection of smaller tasks. The tasks, however, are rarely independent, because scheduling constraints exist between them. (For example, in a house building project, the task of ordering nails obviously precedes the task of nailing shingles to the roof deck.) Clearly, scheduling constraints cannot have circularities, because they would make the project impossible. (For example, in order to get a job you need to have work experience, but in order to get work experience you need to have a job.) The scheduling constraints impose restrictions on the order in which the tasks can be executed. Namely, if a constraint says that task a must be completed before task b is started, then a must precede b in the order of execution of the tasks. Thus, if we model a feasible set of tasks as vertices of a directed graph, and we place a directed edge from u to v whenever the task for u must be executed before the task for v , then we define a directed acyclic graph.*

14.5.1 Topological Ordering

The example above motivates the following definition. Let $\vec{G}$ be a directed graph with n vertices. A **topological ordering** of $\vec{G}$ is an ordering $v_1, \dots, v_n$ of the vertices of $\vec{G}$ such that for every edge (v_i, v_j) of $\vec{G}$, it is the case that $i < j$. That is, a topological ordering is an ordering such that any directed path in $\vec{G}$ traverses vertices in increasing order. Note that a directed graph may have more than one topological ordering. (See Figure 14.12.)

Proposition 14.21: $\vec{G}$ has a topological ordering if and only if it is acyclic.

Justification: The necessity (the “only if” part of the statement) is easy to demonstrate. Suppose $\vec{G}$ is topologically ordered. Assume, for the sake of a contradiction, that $\vec{G}$ has a cycle consisting of edges $(v_{i_0}, v_{i_1}), (v_{i_1}, v_{i_2}), \dots, (v_{i_{k-1}}, v_{i_0})$. Because of the topological ordering, we must have $i_0 < i_1 < \dots < i_{k-1} < i_0$, which is clearly impossible. Thus, $\vec{G}$ must be acyclic.

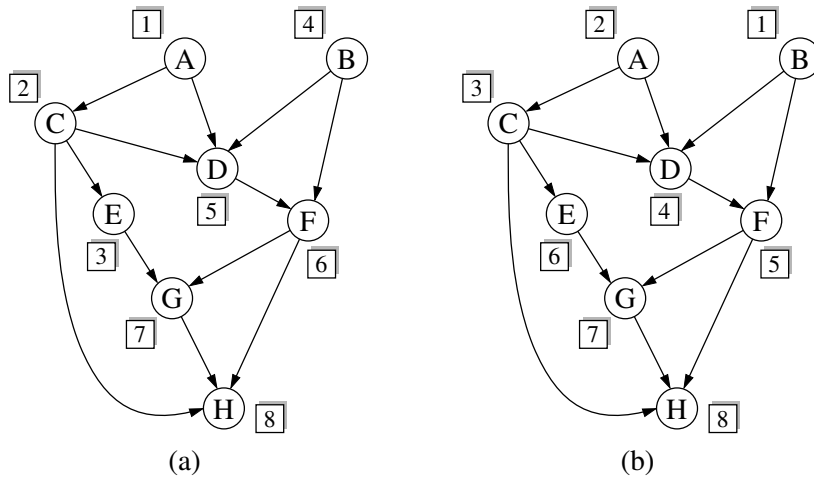


Figure 14.12: Two topological orderings of the same acyclic directed graph.

We now argue the sufficiency of the condition (the “if” part). Suppose $\vec{G}$ is acyclic. We will give an algorithmic description of how to build a topological ordering for $\vec{G}$. Since $\vec{G}$ is acyclic, $\vec{G}$ must have a vertex with no incoming edges (that is, with in-degree 0). Let v_1 be such a vertex. Indeed, if v_1 did not exist, then in tracing a directed path from an arbitrary start vertex, we would eventually encounter a previously visited vertex, thus contradicting the acyclicity of $\vec{G}$. If we remove v_1 from $\vec{G}$, together with its outgoing edges, the resulting directed graph is still acyclic. Hence, the resulting directed graph also has a vertex with no incoming edges, and we let v_2 be such a vertex. By repeating this process until the directed graph becomes empty, we obtain an ordering $v_1, \dots, v_n$ of the vertices of $\vec{G}$. Because of the construction above, if (v_i, v_j) is an edge of $\vec{G}$, then v_i must be deleted before v_j can be deleted, and thus, $i < j$. Therefore, $v_1, \dots, v_n$ is a topological ordering. ■

Proposition 14.21’s justification suggests an algorithm for computing a topological ordering of a directed graph, which we call **topological sorting**. We present a Python implementation of the technique in Code Fragment 14.11, and an example execution of the algorithm in Figure 14.13. Our implementation uses a dictionary, named `incount`, to map each vertex v to a counter that represents the current number of incoming edges to v , excluding those coming from vertices that have previously been added to the topological order. Technically, a Python dictionary provides $O(1)$ expected time access to entries, rather than worst-case time; as was the case with our graph traversals, this could be converted to worst-case time if vertices could be indexed from 0 to $n - 1$, or if we store the counter as an element of a vertex.

As a side effect, the topological sorting algorithm of Code Fragment 14.11 also tests whether the given directed graph $\vec{G}$ is acyclic. Indeed, if the algorithm terminates without ordering all the vertices, then the subgraph of the vertices that have not been ordered must contain a directed cycle.

```

1  def topological_sort(g):
2      """ Return a list of vertices of directed acyclic graph g in topological order.
3
4      If graph g has a cycle, the result will be incomplete.
5      """
6      topo = [ ]           # a list of vertices placed in topological order
7      ready = [ ]          # list of vertices that have no remaining constraints
8      incount = { }        # keep track of in-degree for each vertex
9      for u in g.vertices():
10         incount[u] = g.degree(u, False)    # parameter requests incoming degree
11         if incount[u] == 0:                 # if u has no incoming edges,
12             ready.append(u)                 # it is free of constraints
13         while len(ready) > 0:
14             u = ready.pop( )                 # u is free of constraints
15             topo.append(u)                   # add u to the topological order
16             for e in g.incident_edges(u):    # consider all outgoing neighbors of u
17                 v = e.opposite(u)
18                 incount[v] -= 1              # v has one less constraint without u
19                 if incount[v] == 0:
20                     ready.append(v)
21     return topo

```

Code Fragment 14.11: Python implementation for the topological sorting algorithm. (We show an example execution of this algorithm in Figure 14.13.)

Performance of Topological Sorting

Proposition 14.22: Let $\vec{G}$ be a directed graph with n vertices and m edges, using an adjacency list representation. The topological sorting algorithm runs in $O(n+m)$ time using $O(n)$ auxiliary space, and either computes a topological ordering of $\vec{G}$ or fails to include some vertices, which indicates that $\vec{G}$ has a directed cycle.

Justification: The initial recording of the n in-degrees uses $O(n)$ time based on the degree method. Say that a vertex u is **visited** by the topological sorting algorithm when u is removed from the ready list. A vertex u can be visited only when $\text{incount}(u)$ is 0, which implies that all its predecessors (vertices with outgoing edges into u) were previously visited. As a consequence, any vertex that is on a directed cycle will never be visited, and any other vertex will be visited exactly once. The algorithm traverses all the outgoing edges of each visited vertex once, so its running time is proportional to the number of outgoing edges of the visited vertices. In accordance with Proposition 14.9, the running time is $(n+m)$. Regarding the space usage, observe that containers `topo`, `ready`, and `incount` have at most one entry per vertex, and therefore use $O(n)$ space. ■

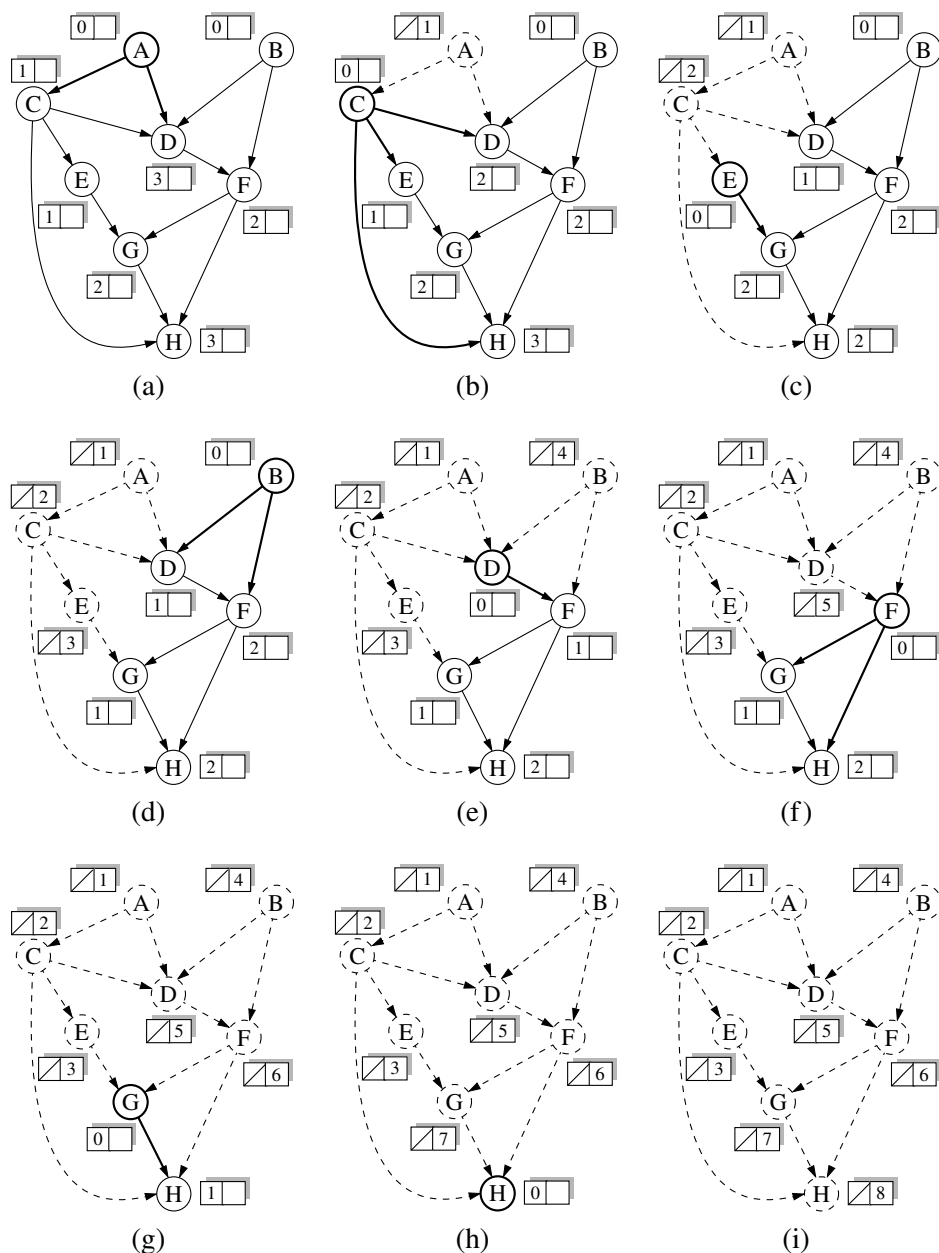


Figure 14.13: Example of a run of algorithm `topological_sort` (Code Fragment 14.11). The label near a vertex shows its current incount value, and its eventual rank in the resulting topological order. The highlighted vertex is one with incount equal to zero that will become the next vertex in the topological order. Dashed lines denote edges that have already been examined and which are no longer reflected in the incount values.

14.6 Shortest Paths

As we saw in Section 14.3.3, the breadth-first search strategy can be used to find a shortest path from some starting vertex to every other vertex in a connected graph. This approach makes sense in cases where each edge is as good as any other, but there are many situations where this approach is not appropriate.

For example, we might want to use a graph to represent the roads between cities, and we might be interested in finding the fastest way to travel cross-country. In this case, it is probably not appropriate for all the edges to be equal to each other, for some inter-city distances will likely be much larger than others. Likewise, we might be using a graph to represent a computer network (such as the Internet), and we might be interested in finding the fastest way to route a data packet between two computers. In this case, it again may not be appropriate for all the edges to be equal to each other, for some connections in a computer network are typically much faster than others (for example, some edges might represent low-bandwidth connections, while others might represent high-speed, fiber-optic connections). It is natural, therefore, to consider graphs whose edges are not weighted equally.

14.6.1 Weighted Graphs

A **weighted graph** is a graph that has a numeric (for example, integer) label $w(e)$ associated with each edge e , called the **weight** of edge e . For $e = (u, v)$, we let notation $w(u, v) = w(e)$. We show an example of a weighted graph in Figure 14.14.

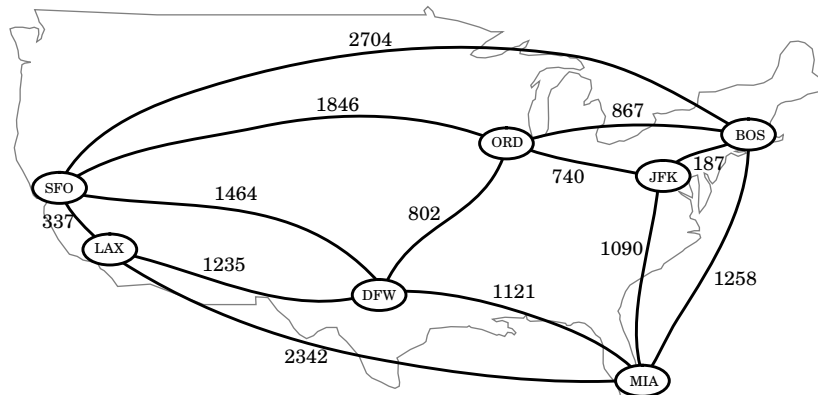


Figure 14.14: A weighted graph whose vertices represent major U.S. airports and whose edge weights represent distances in miles. This graph has a path from JFK to LAX of total weight 2,777 (going through ORD and DFW). This is the minimum-weight path in the graph from JFK to LAX.

Defining Shortest Paths in a Weighted Graph

Let G be a weighted graph. The **length** (or weight) of a path is the sum of the weights of the edges of P . That is, if $P = ((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$, then the length of P , denoted $w(P)$, is defined as

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

The **distance** from a vertex u to a vertex v in G , denoted $d(u, v)$, is the length of a minimum-length path (also called **shortest path**) from u to v , if such a path exists.

People often use the convention that $d(u, v) = \infty$ if there is no path at all from u to v in G . Even if there is a path from u to v in G , however, if there is a cycle in G whose total weight is negative, the distance from u to v may not be defined. For example, suppose vertices in G represent cities, and the weights of edges in G represent how much money it costs to go from one city to another. If someone were willing to actually pay us to go from say JFK to ORD, then the “cost” of the edge (JFK,ORD) would be negative. If someone else were willing to pay us to go from ORD to JFK, then there would be a negative-weight cycle in G and distances would no longer be defined. That is, anyone could now build a path (with cycles) in G from any city A to another city B that first goes to JFK and then cycles as many times as he or she likes from JFK to ORD and back, before going on to B . The existence of such paths would allow us to build arbitrarily low negative-cost paths (and, in this case, make a fortune in the process). But distances cannot be arbitrarily low negative numbers. Thus, any time we use edge weights to represent distances, we must be careful not to introduce any negative-weight cycles.

Suppose we are given a weighted graph G , and we are asked to find a shortest path from some vertex s to each other vertex in G , viewing the weights on the edges as distances. In this section, we explore efficient ways of finding all such shortest paths, if they exist. The first algorithm we discuss is for the simple, yet common, case when all the edge weights in G are nonnegative (that is, $w(e) \geq 0$ for each edge e of G); hence, we know in advance that there are no negative-weight cycles in G . Recall that the special case of computing a shortest path when all weights are equal to one was solved with the BFS traversal algorithm presented in Section 14.3.3.

There is an interesting approach for solving this **single-source** problem based on the **greedy method** design pattern (Section 13.4.2). Recall that in this pattern we solve the problem at hand by repeatedly selecting the best choice from among those available in each iteration. This paradigm can often be used in situations where we are trying to optimize some cost function over a collection of objects. We can add objects to our collection, one at a time, always picking the next one that optimizes the function from among those yet to be chosen.

14.6.2 Dijkstra's Algorithm

The main idea in applying the greedy method pattern to the single-source shortest-path problem is to perform a “weighted” breadth-first search starting at the source vertex s . In particular, we can use the greedy method to develop an algorithm that iteratively grows a “cloud” of vertices out of s , with the vertices entering the cloud in order of their distances from s . Thus, in each iteration, the next vertex chosen is the vertex outside the cloud that is closest to s . The algorithm terminates when no more vertices are outside the cloud (or when those outside the cloud are not connected to those within the cloud), at which point we have a shortest path from s to every vertex of G that is reachable from s . This approach is a simple, but nevertheless powerful, example of the greedy method design pattern. Applying the greedy method to the single-source, shortest-path problem, results in an algorithm known as *Dijkstra's algorithm*.

Edge Relaxation

Let us define a label $D[v]$ for each vertex v in V , which we use to approximate the distance in G from s to v . The meaning of these labels is that $D[v]$ will always store the length of the best path we have found *so far* from s to v . Initially, $D[s] = 0$ and $D[v] = \infty$ for each $v \neq s$, and we define the set C , which is our “**cloud**” of vertices, to initially be the empty set. At each iteration of the algorithm, we select a vertex u not in C with smallest $D[u]$ label, and we pull u into C . (In general, we will use a priority queue to select among the vertices outside the cloud.) In the very first iteration we will, of course, pull s into C . Once a new vertex u is pulled into C , we then update the label $D[v]$ of each vertex v that is adjacent to u and is outside of C , to reflect the fact that there may be a new and better way to get to v via u . This update operation is known as a *relaxation* procedure, for it takes an old estimate and checks if it can be improved to get closer to its true value. The specific edge relaxation operation is as follows:

Edge Relaxation:

```

if  $D[u] + w(u, v) < D[v]$  then
     $D[v] = D[u] + w(u, v)$ 

```

Algorithm Description and Example

We give the pseudo-code for Dijkstra's algorithm in Code Fragment 14.12, and illustrate several iterations of Dijkstra's algorithm in Figures 14.15 through 14.17.

Algorithm ShortestPath(G, s):

Input: A weighted graph G with nonnegative edge weights, and a distinguished vertex s of G .

Output: The length of a shortest path from s to v for each vertex v of G .

Initialize $D[s] = 0$ and $D[v] = \infty$ for each vertex $v \neq s$.

Let a priority queue Q contain all the vertices of G using the D labels as keys.

while Q is not empty **do**

 {pull a new vertex u into the cloud}

$u =$ value returned by $Q.remove_min()$

for each vertex v adjacent to u such that v is in Q **do**

 {perform the *relaxation* procedure on edge (u, v) }

if $D[u] + w(u, v) < D[v]$ **then**

$D[v] = D[u] + w(u, v)$

 Change to $D[v]$ the key of vertex v in Q .

return the label $D[v]$ of each vertex v

Code Fragment 14.12: Pseudo-code for Dijkstra's algorithm, solving the single-source shortest-path problem.

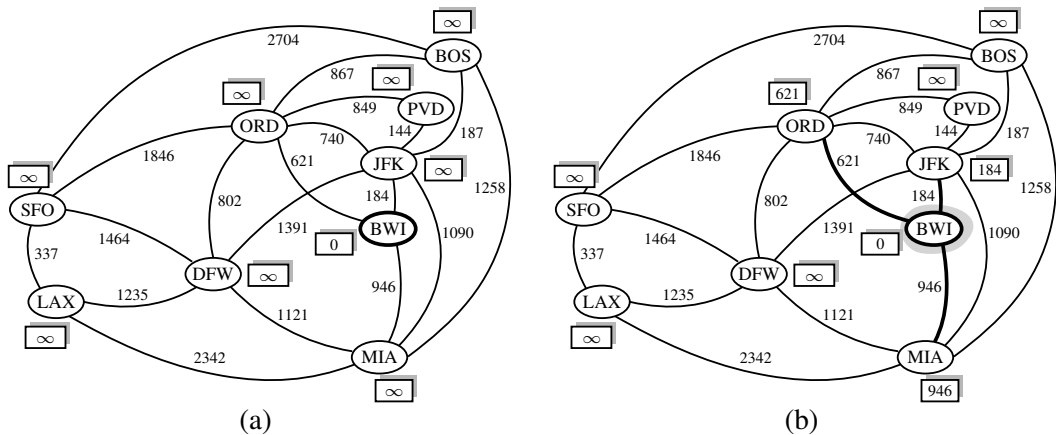


Figure 14.15: An execution of Dijkstra's algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

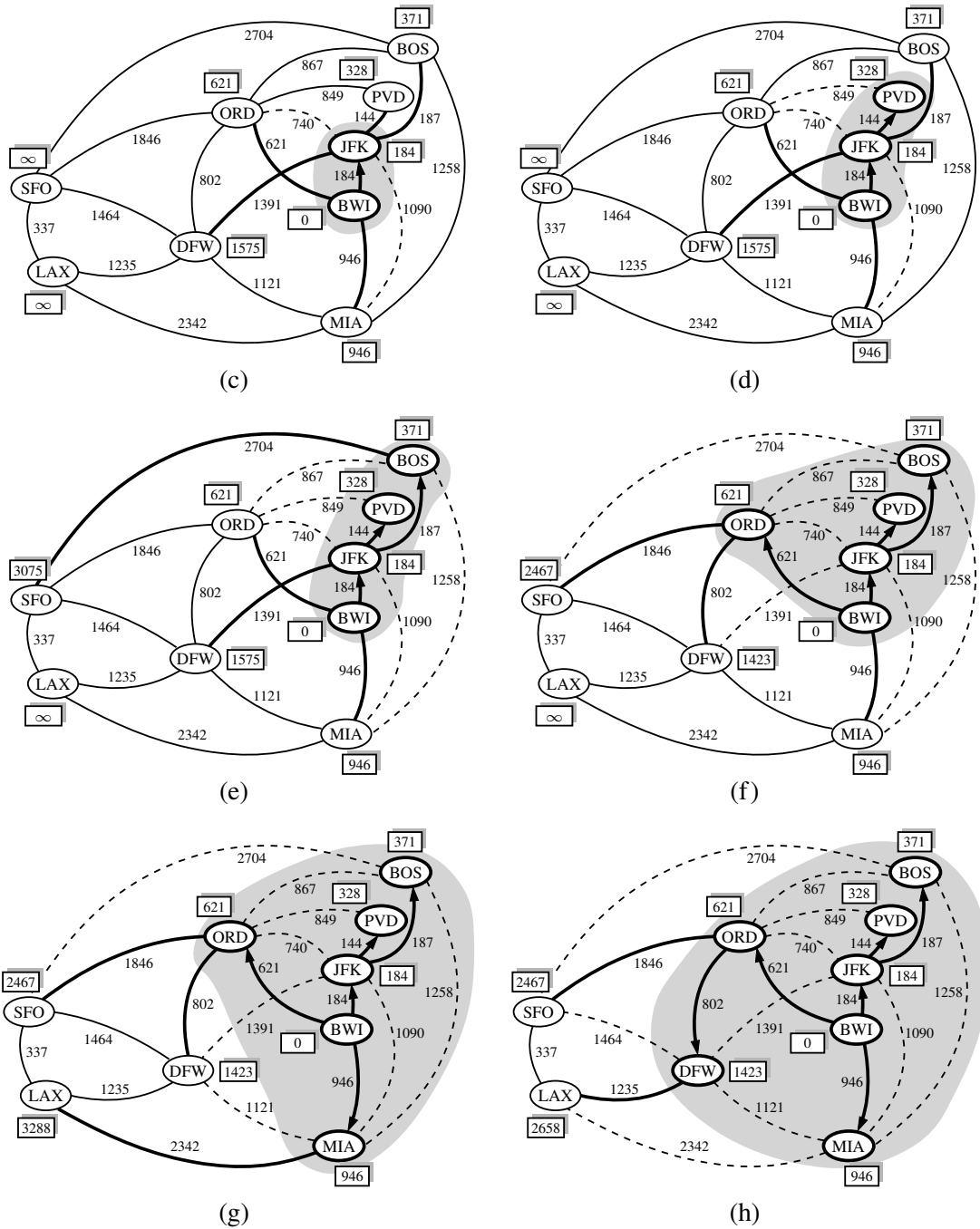


Figure 14.16: An example execution of Dijkstra's algorithm. (Continued from Figure 14.15; continued in Figure 14.17.)

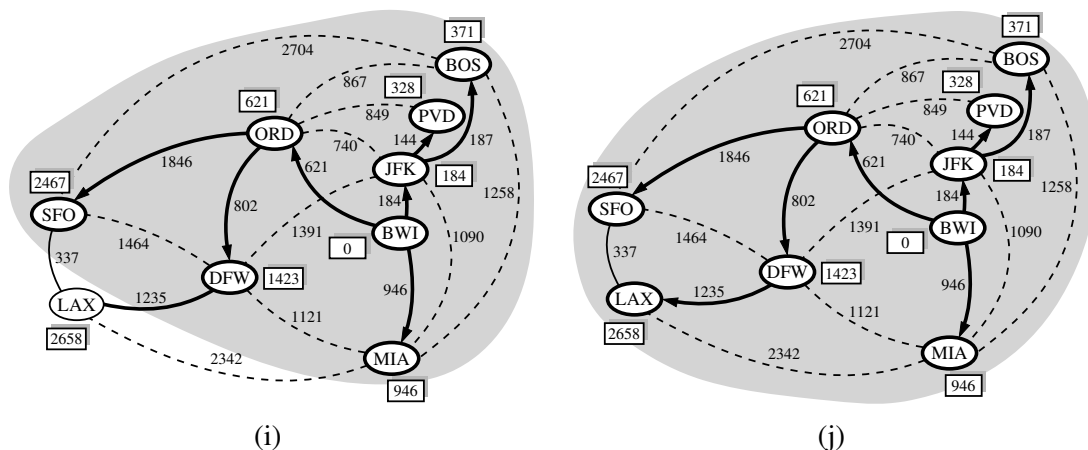


Figure 14.17: An example execution of Dijkstra's algorithm. (Continued from Figure 14.16.)

Why It Works

The interesting aspect of the Dijkstra algorithm is that, at the moment a vertex u is pulled into C , its label $D[u]$ stores the correct length of a shortest path from v to u . Thus, when the algorithm terminates, it will have computed the shortest-path distance from s to every vertex of G . That is, it will have solved the single-source shortest-path problem.

It is probably not immediately clear why Dijkstra's algorithm correctly finds the shortest path from the start vertex s to each other vertex u in the graph. Why is it that the distance from s to u is equal to the value of the label $D[u]$ at the time vertex u is removed from the priority queue Q and added to the cloud C ? The answer to this question depends on there being no negative-weight edges in the graph, for it allows the greedy method to work correctly, as we show in the proposition that follows.

Proposition 14.23: *In Dijkstra's algorithm, whenever a vertex v is pulled into the cloud, the label $D[v]$ is equal to $d(s, v)$, the length of a shortest path from s to v .*

Justification: Suppose that $D[v] > d(s, v)$ for some vertex v in V , and let z be the *first* vertex the algorithm pulled into the cloud C (that is, removed from Q) such that $D[z] > d(s, z)$. There is a shortest path P from s to z (for otherwise $d(s, z) = \infty = D[z]$). Let us therefore consider the moment when z is pulled into C , and let y be the first vertex of P (when going from s to z) that is not in C at this moment. Let x be the predecessor of y in path P (note that we could have $x = s$). (See Figure 14.18.) We know, by our choice of y , that x is already in C at this point.

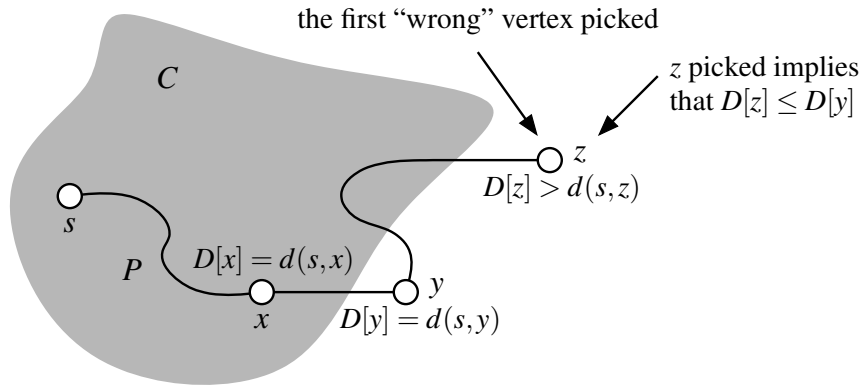


Figure 14.18: A schematic illustration for the justification of Proposition 14.23.

Moreover, $D[x] = d(s, x)$, since z is the **first** incorrect vertex. When x was pulled into C , we tested (and possibly updated) $D[y]$ so that we had at that point

$$D[y] \leq D[x] + w(x, y) = d(s, x) + w(x, y).$$

But since y is the next vertex on the shortest path from s to z , this implies that

$$D[y] = d(s, y).$$

But we are now at the moment when we are picking z , not y , to join C ; hence,

$$D[z] \leq D[y].$$

It should be clear that a subpath of a shortest path is itself a shortest path. Hence, since y is on the shortest path from s to z ,

$$d(s, y) + d(y, z) = d(s, z).$$

Moreover, $d(y, z) \geq 0$ because there are no negative-weight edges. Therefore,

$$D[z] \leq D[y] = d(s, y) \leq d(s, y) + d(y, z) = d(s, z).$$

But this contradicts the definition of z ; hence, there can be no such vertex z . ■

The Running Time of Dijkstra's Algorithm

In this section, we analyze the time complexity of Dijkstra's algorithm. We denote with n and m the number of vertices and edges of the input graph G , respectively. We assume that the edge weights can be added and compared in constant time. Because of the high level of the description we gave for Dijkstra's algorithm in Code Fragment 14.12, analyzing its running time requires that we give more details on its implementation. Specifically, we should indicate the data structures used and how they are implemented.

Let us first assume that we are representing the graph G using an adjacency list or adjacency map structure. This data structure allows us to step through the vertices adjacent to u during the relaxation step in time proportional to their number. Therefore, the time spent in the management of the nested **for** loop, and the number of iterations of that loop, is

$$\sum_{u \text{ in } V_G} \text{outdeg}(u),$$

which is $O(m)$ by Proposition 14.9. The outer **while** loop executes $O(n)$ times, since a new vertex is added to the cloud during each iteration. This still does not settle all the details for the algorithm analysis, however, for we must say more about how to implement the other principal data structure in the algorithm—the priority queue Q .

Referring back to Code Fragment 14.12 in search of priority queue operations, we find that n vertices are originally inserted into the priority queue; since these are the only insertions, the maximum size of the queue is n . In each of n iterations of the **while** loop, a call to `remove_min` is made to extract the vertex u with smallest D label from Q . Then, for each neighbor v of u , we perform an edge relaxation, and may potentially update the key of v in the queue. Thus, we actually need an implementation of an *adaptable priority queue* (Section 9.5), in which case the key of a vertex v is changed using the method `update( $\ell, k$ )`, where ℓ is the locator for the priority queue entry associated with vertex v . In the worst case, there could be one such update for each edge of the graph. Overall, the running time of Dijkstra's algorithm is bounded by the sum of the following:

- n insertions into Q .
- n calls to the `remove_min` method on Q .
- m calls to the `update` method on Q .

If Q is an adaptable priority queue implemented as a heap, then each of the above operations run in $O(\log n)$, and so the overall running time for Dijkstra's algorithm is $O((n + m) \log n)$. Note that if we wish to express the running time as a function of n only, then it is $O(n^2 \log n)$ in the worst case.

Let us now consider an alternative implementation for the adaptable priority queue Q using an unsorted sequence. (See Exercise P-9.58.) This, of course, requires that we spend $O(n)$ time to extract the minimum element, but it affords very fast key updates, provided Q supports location-aware entries (Section 9.5.1). Specifically, we can implement each key update done in a relaxation step in $O(1)$ time—we simply change the key value once we locate the entry in Q to update. Hence, this implementation results in a running time that is $O(n^2 + m)$, which can be simplified to $O(n^2)$ since G is simple.

Comparing the Two Implementations

We have two choices for implementing the adaptable priority queue with location-aware entries in Dijkstra's algorithm: a heap implementation, which yields a running time of $O((n + m) \log n)$, and an unsorted sequence implementation, which yields a running time of $O(n^2)$. Since both implementations would be fairly simple to code, they are about equal in terms of the programming sophistication needed. These two implementations are also about equal in terms of the constant factors in their worst-case running times. Looking only at these worst-case times, we prefer the heap implementation when the number of edges in the graph is small (that is, when $m < n^2 / \log n$), and we prefer the sequence implementation when the number of edges is large (that is, when $m > n^2 / \log n$).

Proposition 14.24: *Given a weighted graph G with n vertices and m edges, such that the weight of each edge is nonnegative, and a vertex s of G , Dijkstra's algorithm can compute the distance from s to all other vertices of G in the better of $O(n^2)$ or $O((n + m) \log n)$ time.*

We note that an advanced priority queue implementation, known as a *Fibonacci heap*, can be used to implement Dijkstra's algorithm in $O(m + n \log n)$ time.

Programming Dijkstra's Algorithm in Python

Having given a pseudo-code description of Dijkstra's algorithm, let us now present Python code for performing Dijkstra's algorithm, assuming we are given a graph whose edge elements are nonnegative integer weights. Our implementation of the algorithm is in the form of a function, `shortest_path_lengths`, that takes a graph and a designated source vertex as parameters. (See Code Fragment 14.13.) It returns a dictionary, named `cloud`, mapping each vertex v that is reachable from the source to its shortest-path distance $d(s, v)$. We rely on our `AdaptableHeapPriorityQueue` developed in Section 9.5.2 as an adaptable priority queue.

As we have done with other algorithms in this chapter, we rely on dictionaries to map vertices to associated data (in this case, mapping v to its distance bound $D[v]$ and its adaptable priority queue locator). The expected $O(1)$ -time access to elements of these dictionaries could be converted to worst-case bounds, either by numbering vertices from 0 to $n - 1$ to use as indices into a list, or by storing the information within each vertex's element.

The pseudo-code for Dijkstra's algorithm begins by assigning $d[v] = \infty$ for each v other than the source. We rely on the special value `float('inf')` in Python to provide a numeric value that represents positive infinity. However, we avoid including vertices with this "infinite" distance in the resulting `cloud` that is returned by the function. The use of this numeric limit could be avoided altogether by waiting to add a vertex to the priority queue until after an edge that reaches it is relaxed. (See Exercise C-14.64.)

```

1  def shortest_path_lengths(g, src):
2      """ Compute shortest-path distances from src to reachable vertices of g.
3
4      Graph g can be undirected or directed, but must be weighted such that
5      e.element() returns a numeric weight for each edge e.
6
7      Return dictionary mapping each reachable vertex to its distance from src.
8      """
9      d = { }                                # d[v] is upper bound from s to v
10     cloud = { }                            # map reachable v to its d[v] value
11     pq = AdaptableHeapPriorityQueue( )     # vertex v will have key d[v]
12     pqlocator = { }                       # map from vertex to its pq locator
13
14     # for each vertex v of the graph, add an entry to the priority queue, with
15     # the source having distance 0 and all others having infinite distance
16     for v in g.vertices():
17         if v is src:
18             d[v] = 0
19         else:
20             d[v] = float('inf')           # syntax for positive infinity
21             pqlocator[v] = pq.add(d[v], v) # save locator for future updates
22
23     while not pq.is_empty():
24         key, u = pq.remove_min()
25         cloud[u] = key                    # its correct d[u] value
26         del pqlocator[u]                 # u is no longer in pq
27         for e in g.incident_edges(u):    # outgoing edges (u,v)
28             v = e.opposite(u)
29             if v not in cloud:
30                 # perform relaxation step on edge (u,v)
31                 wgt = e.element()
32                 if d[u] + wgt < d[v]:      # better path to v?
33                     d[v] = d[u] + wgt    # update the distance
34                     pq.update(pqlocator[v], d[v], v) # update the pq entry
35
36     return cloud                          # only includes reachable vertices

```

Code Fragment 14.13: Python implementation of Dijkstra’s algorithm for computing the shortest-path distances from a single source. We assume that `e.element()` for edge `e` represents the weight of that edge.

Reconstructing the Shortest-Path Tree

Our pseudo-code description of Dijkstra's algorithm in Code Fragment 14.12, and our implementation in Code Fragment 14.13, computes the value $d[v]$, for each vertex v , that is the length of the shortest path from the source vertex s to v . However, those forms of the algorithm do not explicitly compute the actual paths that achieve those distances. The collection of all shortest paths emanating from source s can be compactly represented by what is known as the *shortest-path tree*. The paths form a rooted tree because if a shortest path from s to v passes through an intermediate vertex u , it must begin with a shortest path from s to u .

In this section, we demonstrate that the shortest-path tree rooted at source s can be reconstructed in $O(n + m)$ time, given the set of $d[v]$ values produced by Dijkstra's algorithm using s as the source. As we did when representing the DFS and BFS trees, we will map each vertex $v \neq s$ to a parent u (possibly, $u = s$), such that u is the vertex immediately before v on a shortest path from s to v . If u is the vertex just before v on the shortest path from s to v , it must be that

$$d[u] + w(u, v) = d[v].$$

Conversely, if the above equation is satisfied, then the shortest path from s to u , followed by the edge (u, v) is a shortest path to v .

Our implementation in Code Fragment 14.14 reconstructs the tree based on this logic, testing all *incoming* edges to each vertex v , looking for a (u, v) that satisfies the key equation. The running time is $O(n + m)$, as we consider each vertex and all incoming edges to those vertices. (See Proposition 14.9.)

```

1  def shortest_path_tree(g, s, d):
2      """Reconstruct shortest-path tree rooted at vertex s, given distance map d.
3
4      Return tree as a map from each reachable vertex v (other than s) to the
5      edge e=(u,v) that is used to reach v from its parent u in the tree.
6      """
7      tree = { }
8      for v in d:
9          if v is not s:
10             for e in g.incident_edges(v, False):          # consider INCOMING edges
11                 u = e.opposite(v)
12                 wgt = e.element()
13                 if d[v] == d[u] + wgt:
14                     tree[v] = e                            # edge e is used to reach v
15     return tree

```

Code Fragment 14.14: Python function that reconstructs the shortest paths, based on knowledge of the single-source distances.

14.7 Minimum Spanning Trees

Suppose we wish to connect all the computers in a new office building using the least amount of cable. We can model this problem using an undirected, weighted graph G whose vertices represent the computers, and whose edges represent all the possible pairs (u, v) of computers, where the weight $w(u, v)$ of edge (u, v) is equal to the amount of cable needed to connect computer u to computer v . Rather than computing a shortest-path tree from some particular vertex v , we are interested instead in finding a tree T that contains all the vertices of G and has the minimum total weight over all such trees. Algorithms for finding such a tree are the focus of this section.

Problem Definition

Given an undirected, weighted graph G , we are interested in finding a tree T that contains all the vertices in G and minimizes the sum

$$w(T) = \sum_{(u,v) \text{ in } T} w(u,v).$$

A tree, such as this, that contains every vertex of a connected graph G is said to be a **spanning tree**, and the problem of computing a spanning tree T with smallest total weight is known as the **minimum spanning tree** (or **MST**) problem.

The development of efficient algorithms for the minimum spanning tree problem predates the modern notion of computer science itself. In this section, we discuss two classic algorithms for solving the MST problem. These algorithms are both applications of the **greedy method**, which, as was discussed briefly in the previous section, is based on choosing objects to join a growing collection by iteratively picking an object that minimizes some cost function. The first algorithm we discuss is the Prim-Jarník algorithm, which grows the MST from a single root vertex, much in the same way as Dijkstra's shortest-path algorithm. The second algorithm we discuss is Kruskal's algorithm, which "grows" the MST in clusters by considering edges in nondecreasing order of their weights.

In order to simplify the description of the algorithms, we assume, in the following, that the input graph G is undirected (that is, all its edges are undirected) and simple (that is, it has no self-loops and no parallel edges). Hence, we denote the edges of G as unordered vertex pairs (u, v) .

Before we discuss the details of these algorithms, however, let us give a crucial fact about minimum spanning trees that forms the basis of the algorithms.

A Crucial Fact about Minimum Spanning Trees

The two MST algorithms we discuss are based on the greedy method, which in this case depends crucially on the following fact. (See Figure 14.19.)

e Belongs to a Minimum Spanning Tree

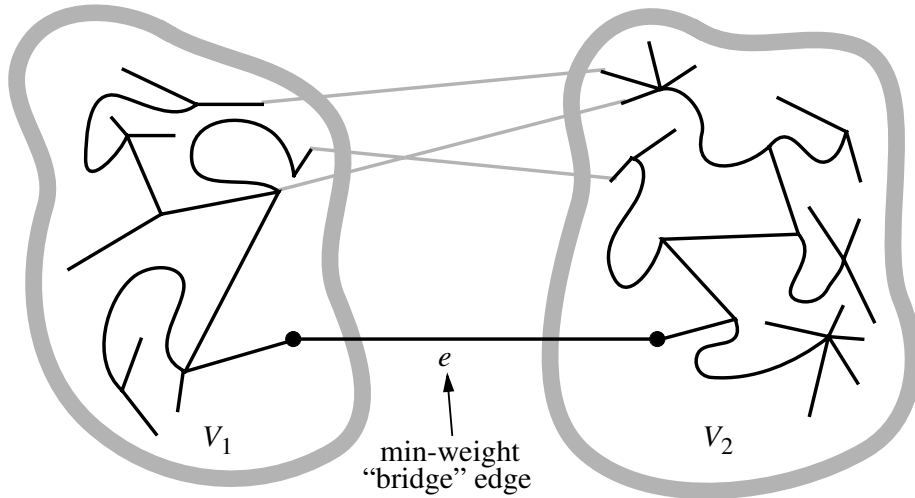


Figure 14.19: An illustration of the crucial fact about minimum spanning trees.

Proposition 14.25: Let G be a weighted connected graph, and let V_1 and V_2 be a partition of the vertices of G into two disjoint nonempty sets. Furthermore, let e be an edge in G with minimum weight from among those with one endpoint in V_1 and the other in V_2 . There is a minimum spanning tree T that has e as one of its edges.

Justification: Let T be a minimum spanning tree of G . If T does not contain edge e , the addition of e to T must create a cycle. Therefore, there is some edge $f \neq e$ of this cycle that has one endpoint in V_1 and the other in V_2 . Moreover, by the choice of e , $w(e) \leq w(f)$. If we remove f from $T \cup \{e\}$, we obtain a spanning tree whose total weight is no more than before. Since T was a minimum spanning tree, this new tree must also be a minimum spanning tree. ■

In fact, if the weights in G are distinct, then the minimum spanning tree is unique; we leave the justification of this less crucial fact as an exercise (C-14.65). In addition, note that Proposition 14.25 remains valid even if the graph G contains negative-weight edges or negative-weight cycles, unlike the algorithms we presented for shortest paths.

14.7.1 Prim-Jarník Algorithm

In the Prim-Jarník algorithm, we grow a minimum spanning tree from a single cluster starting from some “root” vertex s . The main idea is similar to that of Dijkstra’s algorithm. We begin with some vertex s , defining the initial “cloud” of vertices C . Then, in each iteration, we choose a minimum-weight edge $e = (u, v)$, connecting a vertex u in the cloud C to a vertex v outside of C . The vertex v is then brought into the cloud C and the process is repeated until a spanning tree is formed. Again, the crucial fact about minimum spanning trees comes into play, for by always choosing the smallest-weight edge joining a vertex inside C to one outside C , we are assured of always adding a valid edge to the MST.

To efficiently implement this approach, we can take another cue from Dijkstra’s algorithm. We maintain a label $D[v]$ for each vertex v outside the cloud C , so that $D[v]$ stores the weight of the minimum observed edge for joining v to the cloud C . (In Dijkstra’s algorithm, this label measured the full path length from starting vertex s to v , including an edge (u, v) .) These labels serve as keys in a priority queue used to decide which vertex is next in line to join the cloud. We give the pseudo-code in Code Fragment 14.15.

Algorithm PrimJarnik(G):

Input: An undirected, weighted, connected graph G with n vertices and m edges

Output: A minimum spanning tree T for G

Pick any vertex s of G

$D[s] = 0$

for each vertex $v \neq s$ **do**

$D[v] = \infty$

Initialize $T = \emptyset$.

Initialize a priority queue Q with an entry $(D[v], (v, \text{None}))$ for each vertex v , where $D[v]$ is the key in the priority queue, and (v, None) is the associated value.

while Q is not empty **do**

$(u, e) = \text{value returned by } Q.\text{remove_min}()$

 Connect vertex u to T using edge e .

for each edge $e' = (u, v)$ such that v is in Q **do**

 {check if edge (u, v) better connects v to T }

if $w(u, v) < D[v]$ **then**

$D[v] = w(u, v)$

 Change the key of vertex v in Q to $D[v]$.

 Change the value of vertex v in Q to (v, e') .

return the tree T

Code Fragment 14.15: The Prim-Jarník algorithm for the MST problem.

Analyzing the Prim-Jarník Algorithm

The implementation issues for the Prim-Jarník algorithm are similar to those for Dijkstra's algorithm, relying on an adaptable priority queue Q (Section 9.5.1). We initially perform n insertions into Q , later perform n extract-min operations, and may update a total of m priorities as part of the algorithm. Those steps are the primary contributions to the overall running time. With a heap-based priority queue, each operation runs in $O(\log n)$ time, and the overall time for the algorithm is $O((n + m) \log n)$, which is $O(m \log n)$ for a connected graph. Alternatively, we can achieve $O(n^2)$ running time by using an unsorted list as a priority queue.

Illustrating the Prim-Jarník Algorithm

We illustrate the Prim-Jarník algorithm in Figures 14.20 through 14.21.

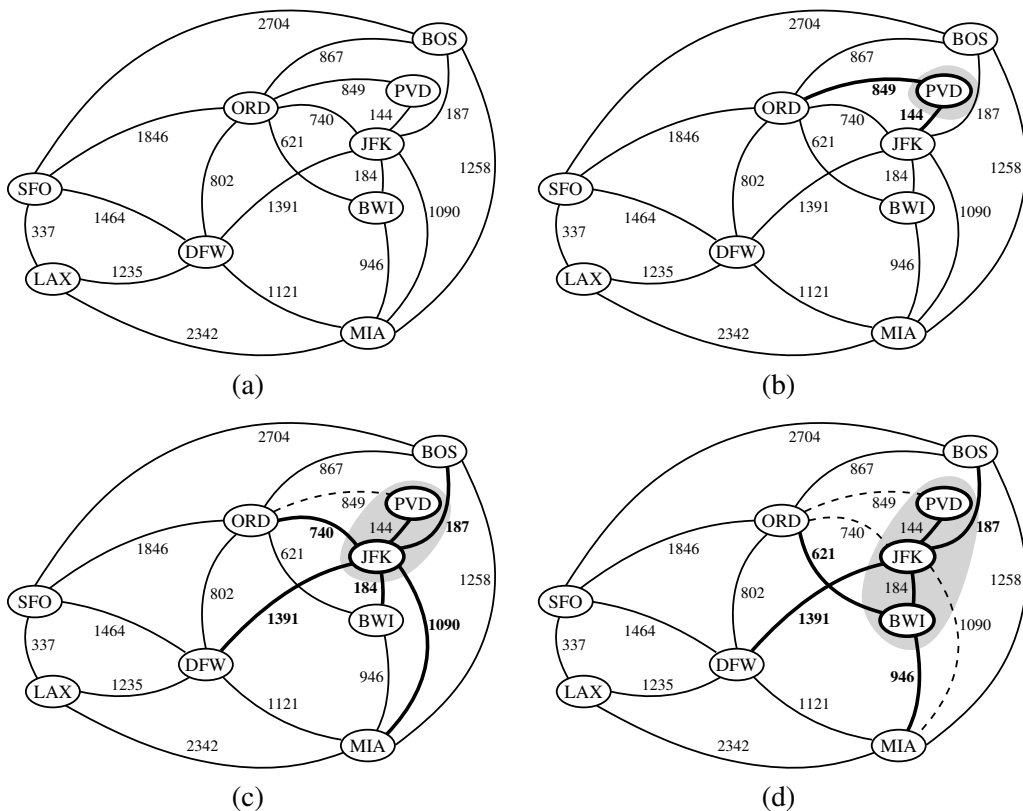


Figure 14.20: An illustration of the Prim-Jarník MST algorithm, starting with vertex PVD. (Continues in Figure 14.21.)

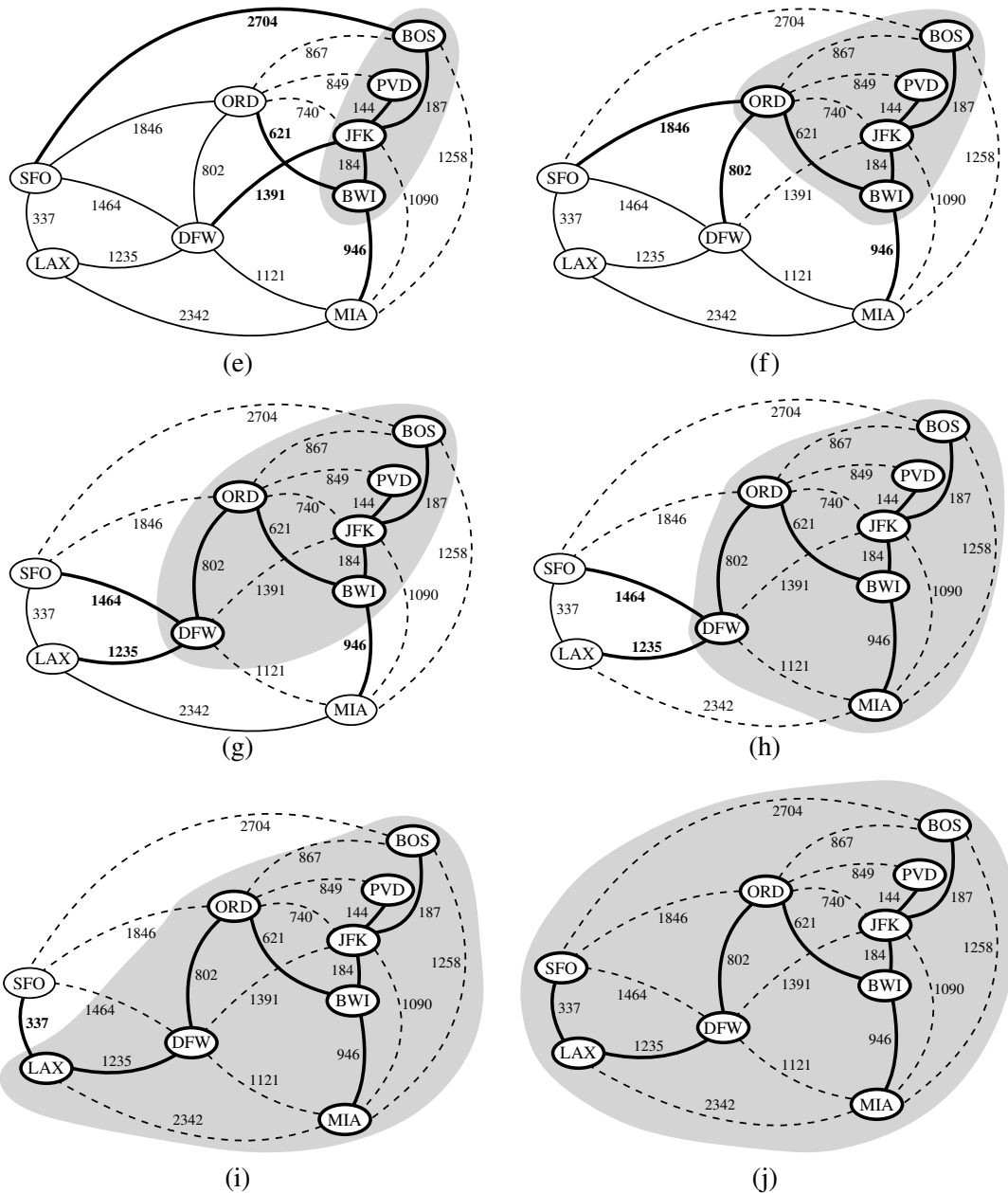


Figure 14.21: An illustration of the Prim-Jarník MST algorithm. (Continued from Figure 14.20.)

Python Implementation

Code Fragment 14.16 presents a Python implementation of the Prim-Jarník algorithm. The MST is returned as an unordered list of edges.

```

1  def MST_PrimJarnik(g):
2      """ Compute a minimum spanning tree of weighted graph g.
3
4      Return a list of edges that comprise the MST (in arbitrary order).
5      """
6      d = { }                                # d[v] is bound on distance to tree
7      tree = [ ]                             # list of edges in spanning tree
8      pq = AdaptableHeapPriorityQueue( )     # d[v] maps to value (v, e=(u,v))
9      pqlocator = { }                       # map from vertex to its pq locator
10
11     # for each vertex v of the graph, add an entry to the priority queue, with
12     # the source having distance 0 and all others having infinite distance
13     for v in g.vertices():
14         if len(d) == 0:                     # this is the first node
15             d[v] = 0                        # make it the root
16         else:
17             d[v] = float('inf')             # positive infinity
18             pqlocator[v] = pq.add(d[v], (v, None))
19
20     while not pq.is_empty():
21         key, value = pq.remove_min()
22         u, edge = value                     # unpack tuple from pq
23         del pqlocator[u]                   # u is no longer in pq
24         if edge is not None:
25             tree.append(edge)               # add edge to tree
26         for link in g.incident_edges(u):
27             v = link.opposite(u)
28             if v in pqlocator:              # thus v not yet in tree
29                 # see if edge (u,v) better connects v to the growing tree
30                 wgt = link.element()
31                 if wgt < d[v]:               # better edge to v?
32                     d[v] = wgt              # update the distance
33                     pq.update(pqlocator[v], d[v], (v, link)) # update the pq entry
34     return tree

```

Code Fragment 14.16: Python implementation of the Prim-Jarník algorithm for the minimum spanning tree problem.

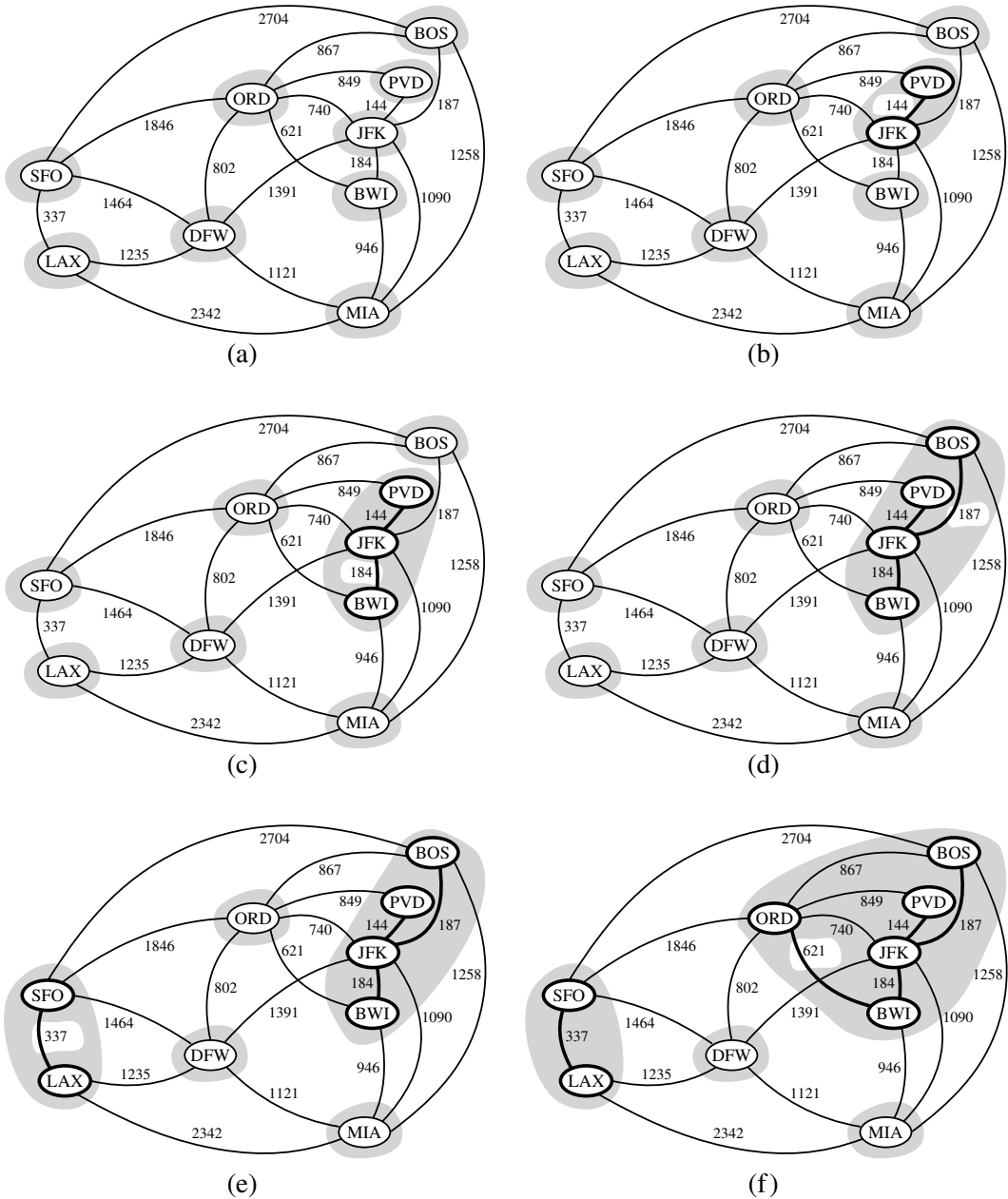


Figure 14.22: Example of an execution of Kruskal's MST algorithm on a graph with integer weights. We show the clusters as shaded regions and we highlight the edge being considered in each iteration. (Continues in Figure 14.23.)

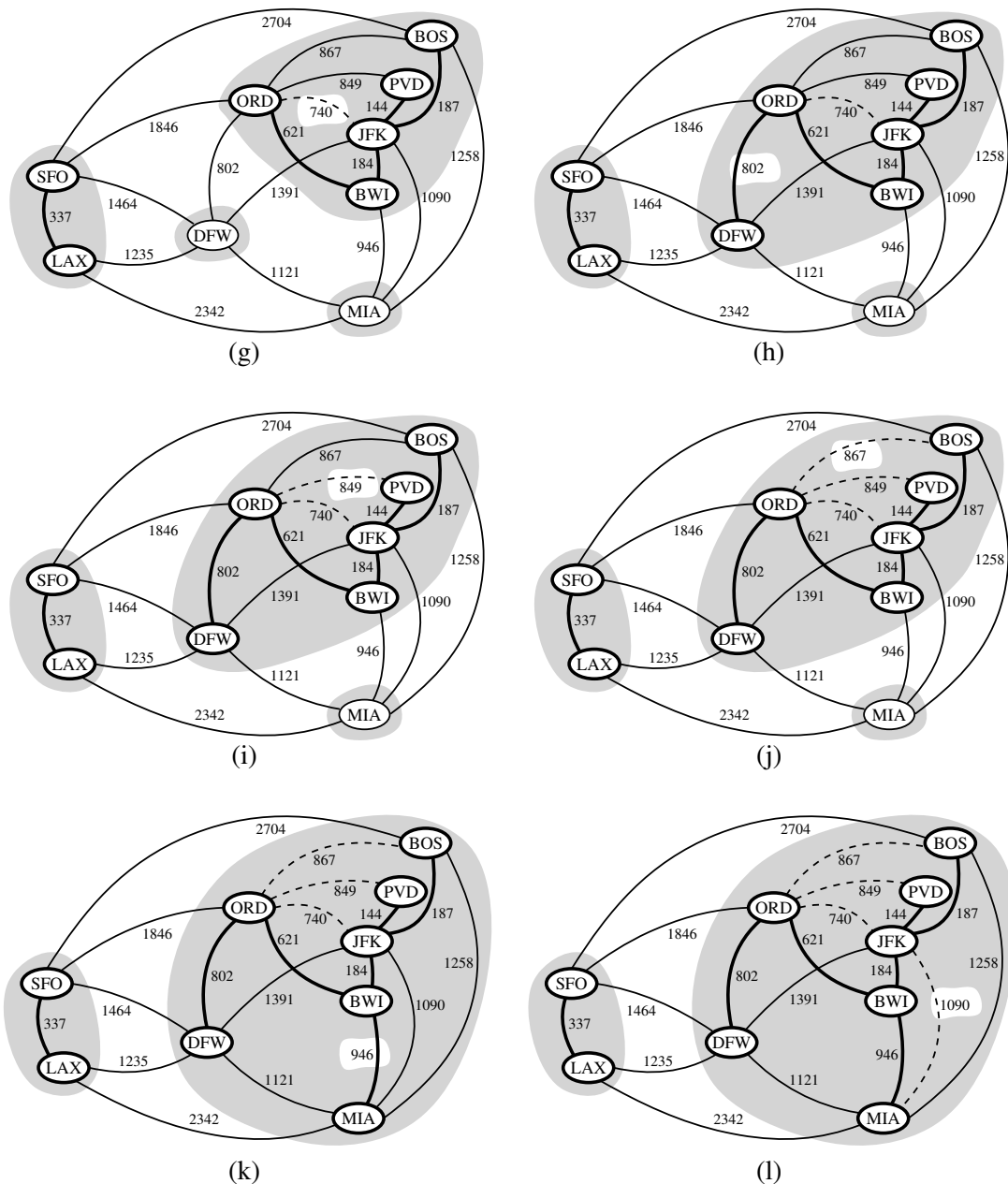


Figure 14.23: An example of an execution of Kruskal's MST algorithm. Rejected edges are shown dashed. (Continues in Figure 14.24.)

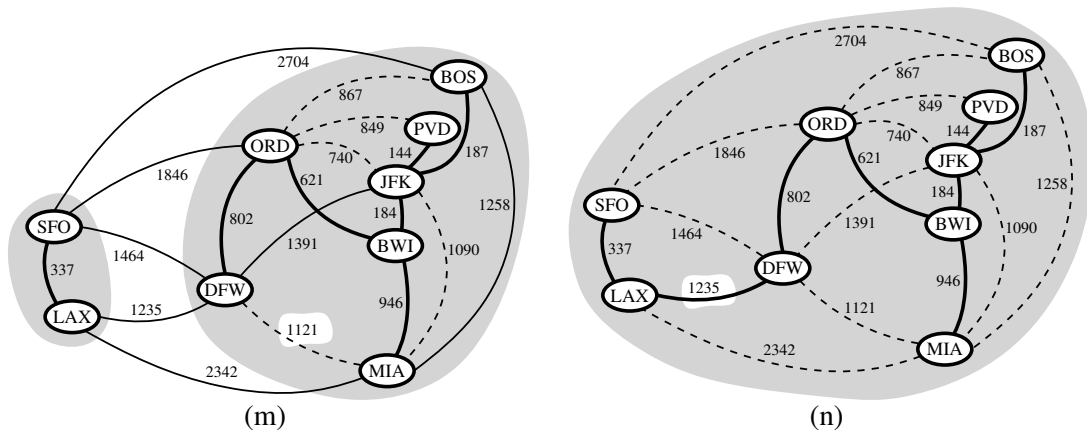


Figure 14.24: Example of an execution of Kruskal's MST algorithm (continued). The edge considered in (n) merges the last two clusters, which concludes this execution of Kruskal's algorithm. (Continued from Figure 14.23.)

The Running Time of Kruskal's Algorithm

There are two primary contributions to the running time of Kruskal's algorithm. The first is the need to consider the edges in nondecreasing order of their weights, and the second is the management of the cluster partition. Analyzing its running time requires that we give more details on its implementation.

The ordering of edges by weight can be implemented in $O(m \log m)$, either by use of a sorting algorithm or a priority queue Q . If that queue is implemented with a heap, we can initialize Q in $O(m \log m)$ time by repeated insertions, or in $O(m)$ time using bottom-up heap construction (see Section 9.3.6), and the subsequent calls to `remove_min` each run in $O(\log m)$ time, since the queue has size $O(m)$. We note that since m is $O(n^2)$ for a simple graph, $O(\log m)$ is the same as $O(\log n)$. Therefore, the running time due to the ordering of edges is $O(m \log n)$.

The remaining task is the management of clusters. To implement Kruskal's algorithm, we must be able to find the clusters for vertices u and v that are endpoints of an edge e , to test whether those two clusters are distinct, and if so, to merge those two clusters into one. None of the data structures we have studied thus far are well suited for this task. However, we conclude this chapter by formalizing the problem of managing *disjoint partitions*, and introducing efficient *union-find* data structures. In the context of Kruskal's algorithm, we perform at most $2m$ find operations and $n - 1$ union operations. We will see that a simple union-find structure can perform that combination of operations in $O(m + n \log n)$ time (see Proposition 14.26), and a more advanced structure can support an even faster time.

For a connected graph, $m \geq n - 1$, and therefore, the bound of $O(m \log n)$ time for ordering the edges dominates the time for managing the clusters. We conclude that the running time of Kruskal's algorithm is $O(m \log n)$.

Python Implementation

Code Fragment 14.18 presents a Python implementation of Kruskal's algorithm. As with our implementation of the Prim-Jarník algorithm, the minimum spanning tree is returned in the form of a list of edges. As a consequence of Kruskal's algorithm, those edges will be reported in nondecreasing order of their weights.

Our implementation assumes use of a `Partition` class for managing the cluster partition. An implementation of the `Partition` class is presented in Section 14.7.3.

```

1  def MST_Kruskal(g):
2      """ Compute a minimum spanning tree of a graph using Kruskal's algorithm.
3
4      Return a list of edges that comprise the MST.
5
6      The elements of the graph's edges are assumed to be weights.
7      """
8      tree = [ ]                # list of edges in spanning tree
9      pq = HeapPriorityQueue( ) # entries are edges in G, with weights as key
10     forest = Partition( )     # keeps track of forest clusters
11     position = { }           # map each node to its Partition entry
12
13     for v in g.vertices():
14         position[v] = forest.make_group(v)
15
16     for e in g.edges():
17         pq.add(e.element(), e) # edge's element is assumed to be its weight
18
19     size = g.vertex_count()
20     while len(tree) != size - 1 and not pq.is_empty():
21         # tree not spanning and unprocessed edges remain
22         weight, edge = pq.remove_min()
23         u, v = edge.endpoints()
24         a = forest.find(position[u])
25         b = forest.find(position[v])
26         if a != b:
27             tree.append(edge)
28             forest.union(a, b)
29
30     return tree

```

Code Fragment 14.18: Python implementation of Kruskal's algorithm for the minimum spanning tree problem.

14.7.3 Disjoint Partitions and Union-Find Structures

In this section, we consider a data structure for managing a *partition* of elements into a collection of disjoint sets. Our initial motivation is in support of Kruskal's minimum spanning tree algorithm, in which a forest of disjoint trees is maintained, with occasional merging of neighboring trees. More generally, the disjoint partition problem can be applied to various models of discrete growth.

We formalize the problem with the following model. A partition data structure manages a universe of elements that are organized into disjoint sets (that is, an element belongs to one and only one of these sets). Unlike with the Set ADT or Python's set class, we do not expect to be able to iterate through the contents of a set, nor to efficiently test whether a given set includes a given element. To avoid confusion with such notions of a set, we will refer to the clusters of our partition as *groups*. However, we will not require an explicit structure for each group, instead allowing the organization of groups to be implicit. To differentiate between one group and another, we assume that at any point in time, each group has a designated entry that we refer to as the *leader* of the group.

Formally, we define the methods of a *partition ADT* using position objects, each of which stores an element x . The partition ADT supports the following methods.

make_group(x): Create a singleton group containing new element x and return the position storing x .

union(p , q): Merge the groups containing positions p and q .

find(p): Return the position of the leader of the group containing position p .

Sequence Implementation

A simple implementation of a partition with a total of n elements uses a collection of sequences, one for each group, where the sequence for a group A stores element positions. Each position object stores a variable, *element*, which references its associated element x and allows the execution of an *element()* method in $O(1)$ time. In addition, each position stores a variable, *group*, that references the sequence storing p , since this sequence is representing the group containing p 's element. (See Figure 14.25.)

With this representation, we can easily perform the *make_group(x)* and *find(p)* operations in $O(1)$ time, allowing the first position in a sequence to serve as the "leader." Operation *union(p , q)* requires that we join two sequences into one and update the group references of the positions in one of the two. We choose to implement this operation by removing all the positions from the sequence with smaller

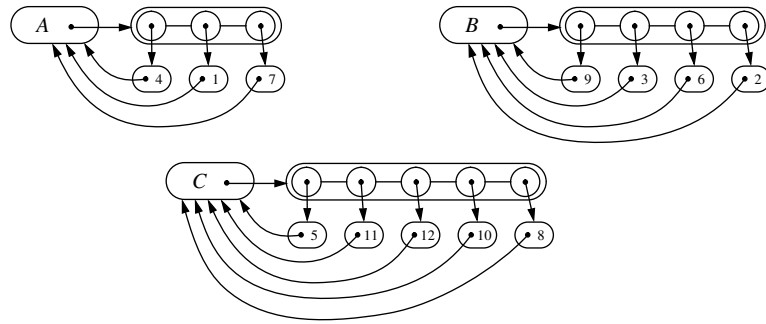


Figure 14.25: Sequence-based implementation of a partition consisting of three groups: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

size, and inserting them in the sequence with larger size. Each time we take a position from the smaller group a and insert it into the larger group b , we update the group reference for that position to now point to b . Hence, the operation $\text{union}(p, q)$ takes time $O(\min(n_p, n_q))$, where n_p (resp. n_q) is the cardinality of the group containing position p (resp. q). Clearly, this time is $O(n)$ if there are n elements in the partition universe. However, we next present an amortized analysis that shows this implementation to be much better than appears from this worst-case analysis.

Proposition 14.26: *When using the above sequence-based partition implementation, performing a series of k `make_group`, `union`, and `find` operations on an initially empty partition involving at most n elements takes $O(k + n \log n)$ time.*

Justification: We use the accounting method and assume that one cyber-dollar can pay for the time to perform a `find` operation, a `make_group` operation, or the movement of a position object from one sequence to another in a `union` operation. In the case of a `find` or `make_group` operation, we charge the operation itself 1 cyber-dollar. In the case of a `union` operation, we assume that 1 cyber-dollar pays for the constant-time work in comparing the sizes of the two sequences, and that we charge 1 cyber-dollar to each position that we move from the smaller group to the larger group. Clearly, the 1 cyber-dollar charged for each `find` and `make_group` operation, together with the first cyber-dollar collected for each `union` operation, accounts for a total of k cyber-dollars.

Consider, then, the number of charges made to positions on behalf of union operations. The important observation is that each time we move a position from one group to another, the size of that position's group at least doubles. Thus, each position is moved from one group to another at most $\log n$ times; hence, each position can be charged at most $O(\log n)$ times. Since we assume that the partition is initially empty, there are $O(n)$ different elements referenced in the given series of operations, which implies that the total time for moving elements during the union operations is $O(n \log n)$. ■

A Tree-Based Partition Implementation ★

An alternative data structure for representing a partition uses a collection of trees to store the n elements, where each tree is associated with a different group. (See Figure 14.26.) In particular, we implement each tree with a linked data structure whose nodes are themselves the group position objects. We view each position p as being a node having an instance variable, *element*, referring to its element x , and an instance variable, *parent*, referring to its parent node. By convention, if p is the **root** of its tree, we set p 's parent reference to itself.

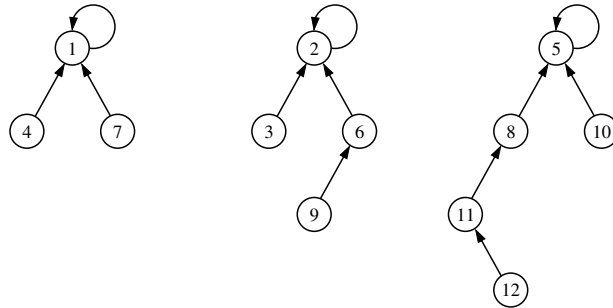


Figure 14.26: Tree-based implementation of a partition consisting of three groups: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

With this partition data structure, operation $\text{find}(p)$ is performed by walking up from position p to the root of its tree, which takes $O(n)$ time in the worst case. Operation $\text{union}(p, q)$ can be implemented by making one of the trees a subtree of the other. This can be done by first locating the two roots, and then in $O(1)$ additional time by setting the parent reference of one root to point to the other root. See Figure 14.27 for an example of both operations.

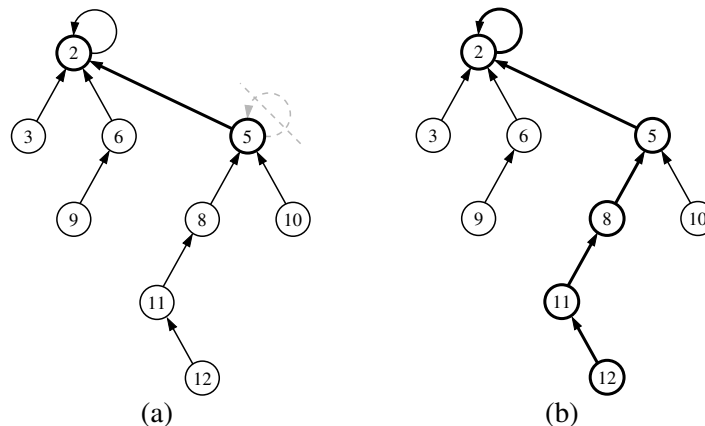


Figure 14.27: Tree-based implementation of a partition: (a) operation $\text{union}(p, q)$; (b) operation $\text{find}(p)$, where p denotes the position object for element 12.

At first, this implementation may seem to be no better than the sequence-based data structure, but we add the following two simple heuristics to make it run faster.

Union-by-Size: With each position p , store the number of elements in the subtree rooted at p . In a union operation, make the root of the smaller group become a child of the other root, and update the size field of the larger root.

Path Compression: In a find operation, for each position q that the find visits, reset the parent of q to the root. (See Figure 14.28.)

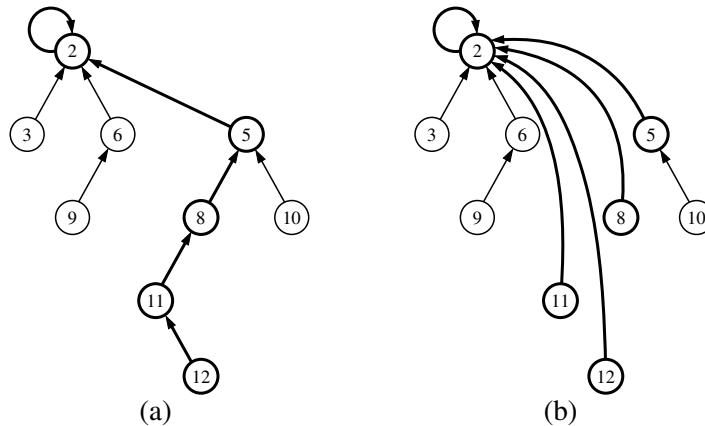


Figure 14.28: Path-compression heuristic: (a) path traversed by operation find on element 12; (b) restructured tree.

A surprising property of this data structure, when implemented using the union-by-size and path-compression heuristics, is that performing a series of k operations involving n elements takes $O(k \log^* n)$ time, where $\log^* n$ is the **log-star** function, which is the inverse of the **tower-of-twos** function. Intuitively, $\log^* n$ is the number of times that one can iteratively take the logarithm (base 2) of a number before getting a number smaller than 2. Table 14.4 shows a few sample values.

minimum n	2	$2^2 = 4$	$2^{2^2} = 16$	$2^{2^{2^2}} = 65,536$	$2^{2^{2^{2^2}}} = 2^{65,536}$
$\log^* n$	1	2	3	4	5

Table 14.4: Some values of $\log^* n$ and critical values for its inverse.

Proposition 14.27: When using the tree-based partition representation with both union-by-size and path compression, performing a series of k make_group, union, and find operations on an initially empty partition involving at most n elements takes $O(k \log^* n)$ time.

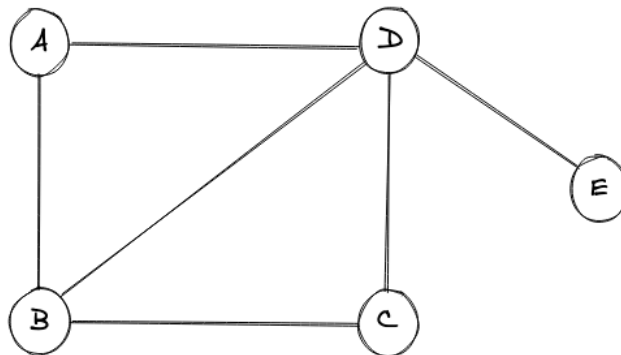
Although the analysis for this data structure is rather complex, its implementation is quite straightforward. We conclude with complete Python code for the structure, given in Code Fragment 14.19.

Implementation.

Graphs – Introduction:

A **graph** is an **advanced data structure** that is used to organize items in an **interconnected network**. Each item in a graph is known as a **node**(or **vertex**) and these nodes are connected by **edges**.

In the figure below, we have a simple graph where there are five nodes in total and six edges.



The nodes in any graph can be referred to as **entities** and the edges that connect different nodes define the **relationships between these entities**. In the above graph we have a set of nodes $\{V\} = \{A, B, C, D, E\}$ and a set of edges, $\{E\} = \{A-B, A-D, B-C, B-D, C-D, D-E\}$ respectively

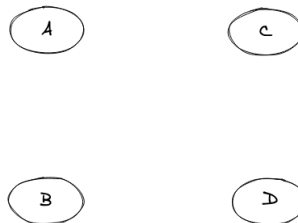
Types of Graphs

Let's cover various different types of graphs.

1. Null Graphs

A graph is said to be null if there are no edges in that graph.

A pictorial representation of the null graph is given below:

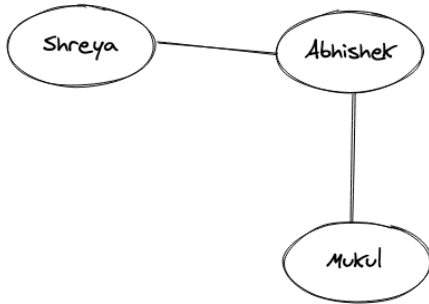


2. Undirected Graphs

If we take a look at the pictorial representation that we had in the Real-world example above, we can clearly see that different nodes are connected by a link (i.e. edge) and that edge doesn't have any kind of direction

associated with it. For example, the edge between "Anuj" and "Deepak" is bi-directional and hence the relationship between them is two ways, which turns out to be that "Anuj" knows "Deepak" and "Deepak" also knows about "Anuj". These types of graphs where the relation is bi-directional or there is not a single direction, are known as Undirected graphs.

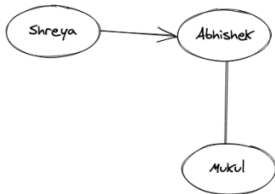
A pictorial representation of another undirected graph is given below:



3. Directed Graphs

What if the relation between the two people is something like, "Shreya" know "Abhishek" but "Abhishek" doesn't know "Shreya". This type of relationship is one-way, and it does include a direction. The edges with arrows basically denote the direction of the relationship and such graphs are known as directed graphs.

A pictorial representation of the graph is given below:

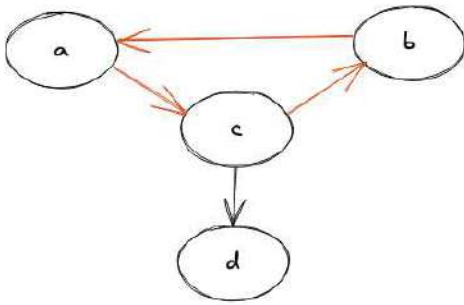


It can also be noted that the edge from "Shreya" to "Abhishek" is an outgoing edge for "Shreya" and an incoming edge for "Abhishek".

4. Cyclic Graph

A graph that contains at least one node that traverses back to itself is known as a cyclic graph. In simple words, a graph should have at least one cycle formation for it to be called a cyclic graph.

A pictorial representation of a cyclic graph is given below:

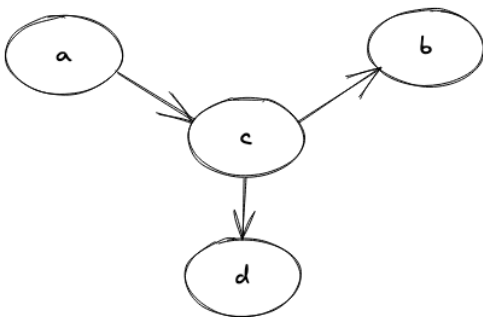


It can be easily seen that there exists a cycle between the nodes (a, b, c) and hence it is a cyclic graph.

5. Acyclic Graph

A graph where there's no way we can start from one node and can traverse back to the same one, or simply doesn't have a single cycle is known as an acyclic graph.

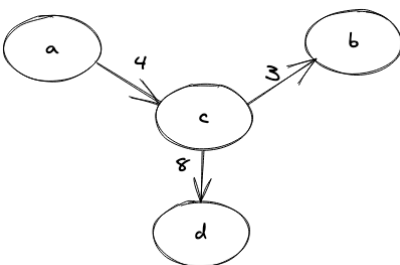
A pictorial representation of an acyclic graph is given below:



6. Weighted Graph

When the edge in a graph has some weight associated with it, we call that graph as a weighted graph. The weight is generally a number that could mean anything, totally dependent on the relationship between the nodes of that graph.

A pictorial representation of the weighted graph is given below:

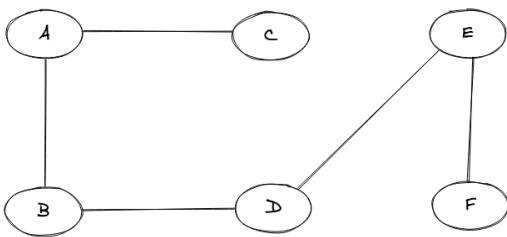


It can also be noted that if any graph doesn't have any weight associated with it, we simply call it an unweighted graph.

7. Connected Graph

A graph where we have a path between every two nodes of the graph is known as a connected graph. A path here means that we are able to traverse from a node "A" to say any node "B". In simple terms, we can say that if we start from one node of the graph we will always be able to traverse to all the other nodes of the graph from that node, hence the connectivity.

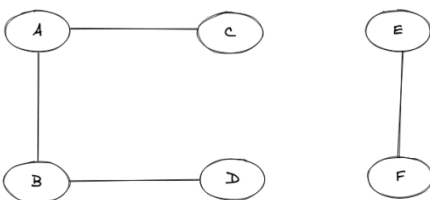
A pictorial representation of the connected graph is given below:



8. Disconnected Graph

A graph that is not connected is simply known as a disconnected graph. In a disconnected graph, we will not be able to find a path from between every two nodes of the graph.

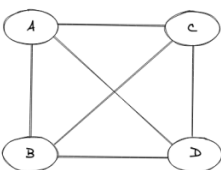
A pictorial representation of the disconnected graph is given below:



9. Complete Graph

A graph is said to be a complete graph if there exists an edge for every pair of vertices(nodes) of that graph.

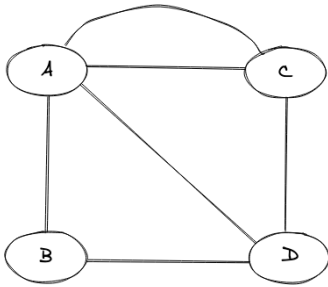
A pictorial representation of the complete graph is given below:



10. Multigraph

A graph is said to be a multigraph if there exist two or more than two edges between any pair of nodes in the graph.

A pictorial representation of the multigraph is given below:



Commonly Used Graph Terminologies

- **Path** - A sequence of alternating nodes and edges such that each of the successive nodes are connected by the edge.
- **Cycle** - A path where the starting and the ending node is the same.
- **Simple Path** - A path where we do not encounter a vertex again.
- **Bridge** - An edge whose removal will simply make the graph disconnected.
- **Forest** - A forest is a graph without cycles.
- **Tree** - A connected graph that doesn't have any cycles.
- **Degree** - The degree in a graph is the number of edges that are incident on a particular node.
- **Neighbour** - We say vertex "A" and "B" are neighbours if there exists an edge between them.

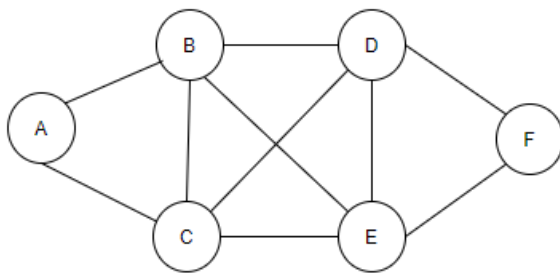
Weighted vs UnWeighted Graph

To understand difference between weighted vs unweighted graph, first we need to understand what **weight** represent ?

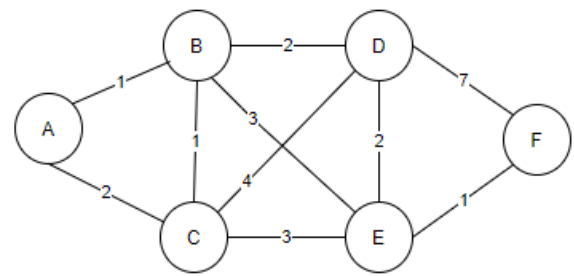
A **weight** is a numerical value attached to each individual edge in the graph.

Weighted Graph will contains weight on each edge where as **unweighted does not**.

Following is an example, where both graphs looks exactly the same but one is weighted another is not.



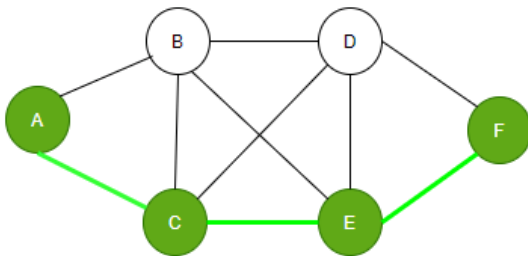
UnWeighted Graph



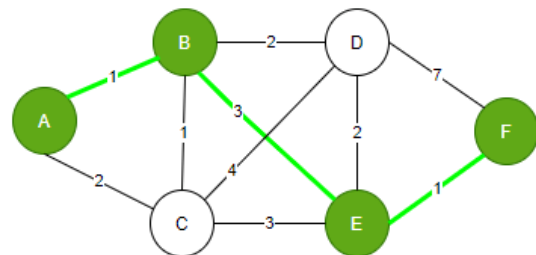
Weighted Graph

What difference does it make ?

Let's take the same example to **find shortest path from A to F**. Result is different, just because one is weighted another doesn't.



UnWeighted Graph



Weighted Graph

But how?

Because when you doesn't have weight, all edges are considered equal. Shortest distance means less number of nodes you travel.

But in case of weighted graph, calculation happens on the sum of weights of the travelled edges.

Breadth First Search:

BFS is an algorithm that is used to graph data or searching tree or traversing structures. The algorithm efficiently visits and marks all the key nodes in a graph in an accurate breadthwise fashion.

This algorithm selects a single node (initial or source point) in a graph and then visits all the nodes adjacent to the selected node. Once the algorithm visits and marks the starting node, then it moves towards the nearest unvisited nodes and analyses them.

Once visited, all nodes are marked. These iterations continue until all the nodes of the graph have been successfully visited and marked. The full form of BFS is the Breadth-first search.

Steps in Breadth First Search

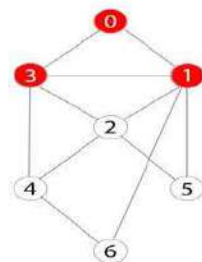
1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

Example of BFS

In the following example of DFS, we have used graph having 6 vertices.

Example of BFS

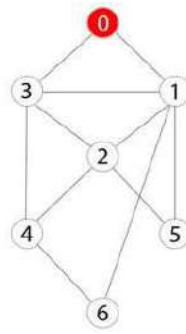
Step 1)



1. Mark 0 as visited
2. Insert 0 to the queue
3. Traverse the un-visited adjacent nodes which are 3 and 1

5.
You have a graph of seven numbers ranging from 0 – 6.

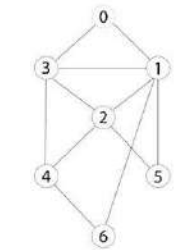
Step 2)



Root node = 0

6.
0 or zero has been marked as a root node.

Step 3)



Queue:

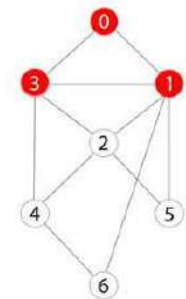
0	1	2	3	4	5	6
0	1	2	3	4	5	6

Delete values from queue
and Print as result

Result = 0, 1, 2, 3, 4, 5, 6

7.
0 is visited, marked, and inserted into the queue data structure.

Step 4)

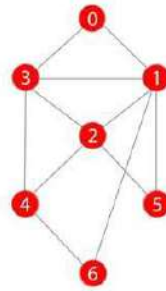


Visit 3 and 1 in any
sequence and mark them
as visited and add them
to the queue

8.

Remaining 0 adjacent and unvisited nodes are visited, marked, and inserted into the queue.

Step 5)



Visit all adjacent and un-visited nodes of the previous node and iterate until all visited

9.

Traversing iterations are repeated until all nodes are visited.

Depth First Search.

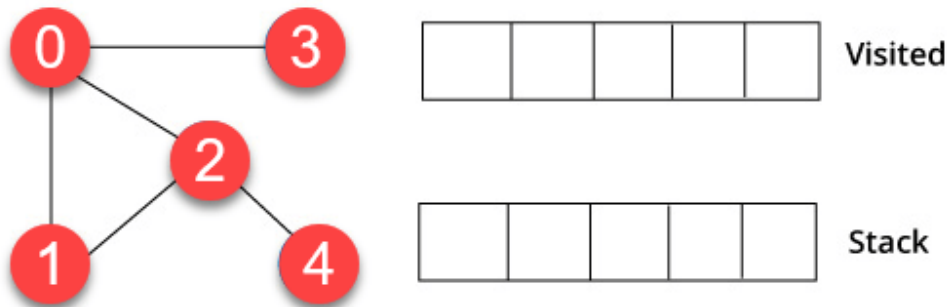
DFS is an algorithm for finding or traversing graphs or trees in depth-ward direction. The execution of the algorithm begins at the root node and explores each branch before backtracking. It uses a stack data structure to remember, to get the subsequent vertex, and to start a search, whenever a dead-end appears in any iteration. The full form of DFS is Depth-first search.

Steps in Depth First Search

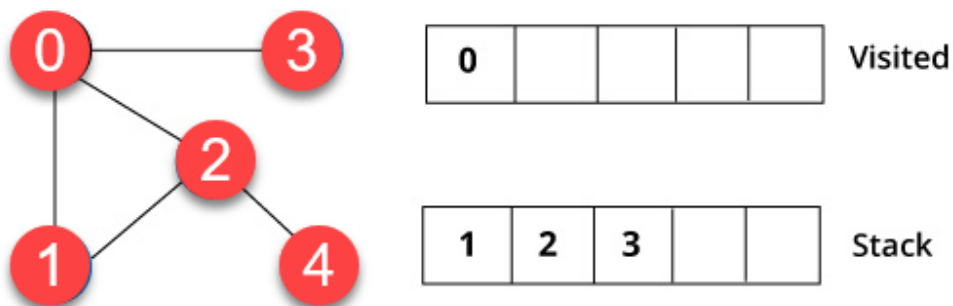
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

Example of DFS

In the following example of DFS, we have used an undirected graph having 5 vertices.

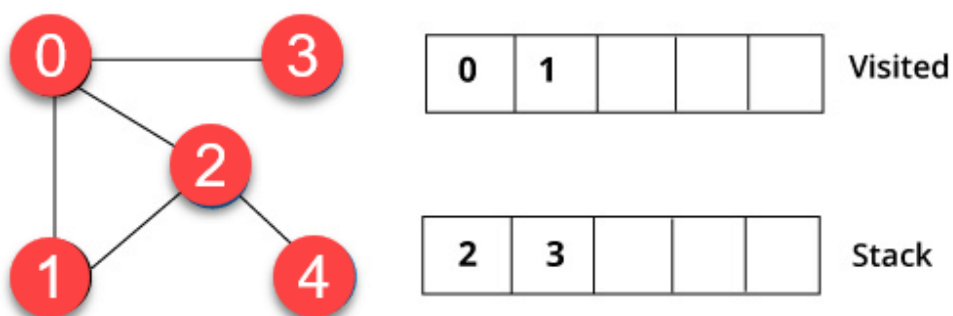


Step 1)



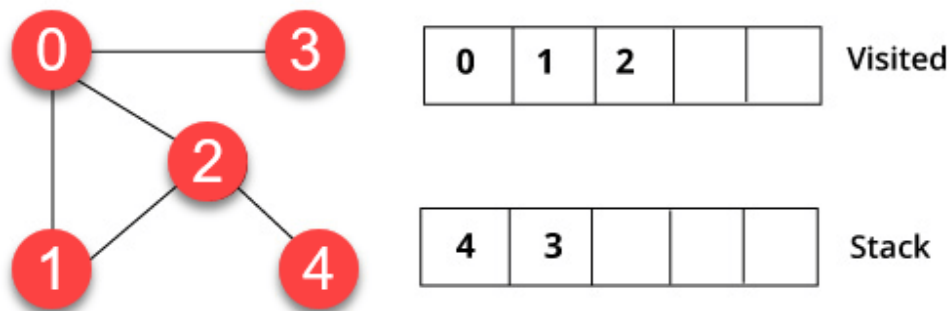
We have started from vertex 0. The algorithm begins by putting it in the visited list and simultaneously putting all its adjacent vertices in the data structure called stack.

Step 2)



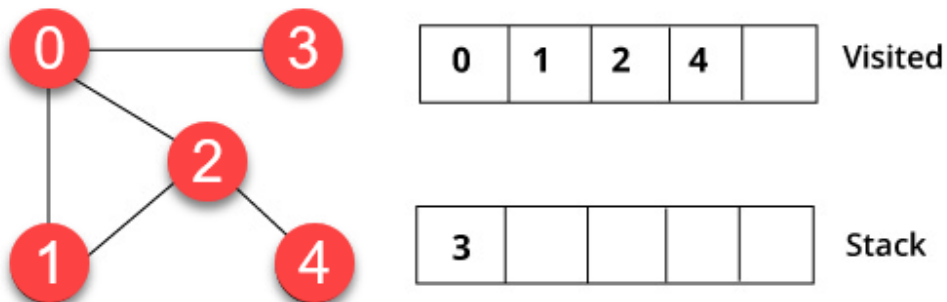
You will visit the element, which is at the top of the stack, for example, 1 and go to its adjacent nodes. It is because 0 has already been visited. Therefore, we visit vertex 2.

Step 3)

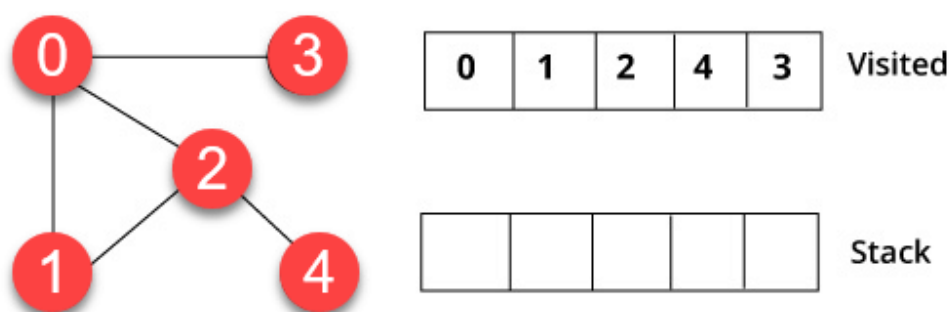


Vertex 2 has an unvisited nearby vertex in 4. Therefore, we add that in the stack and visit it.

Step 4)

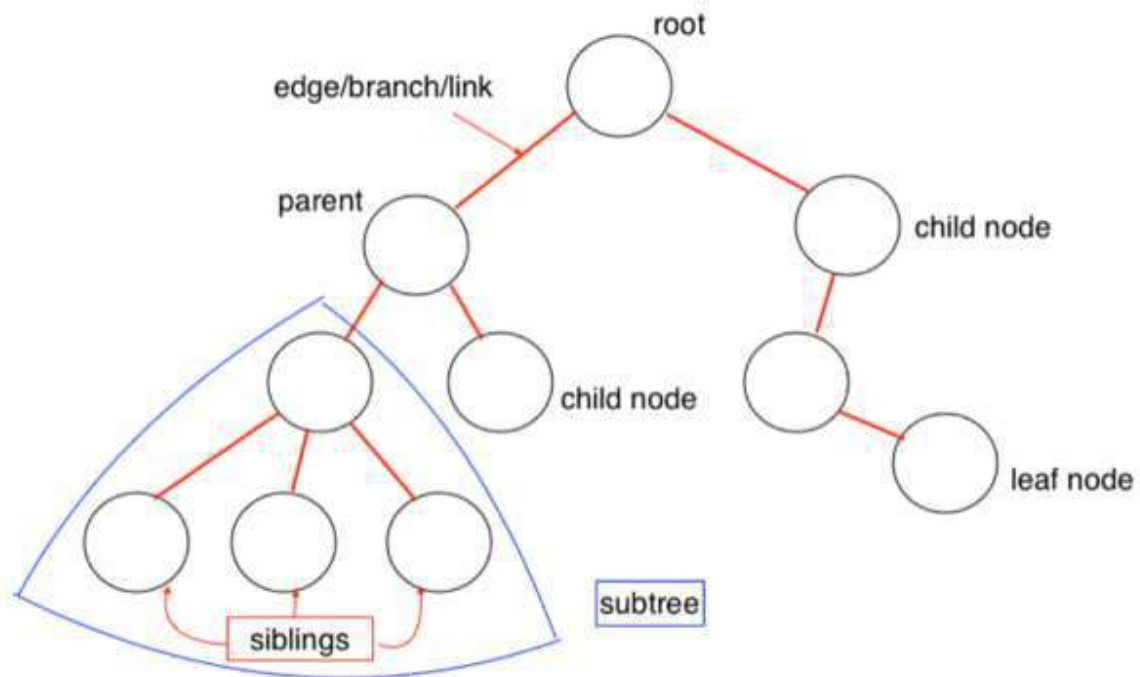


Finally, we will visit the last vertex 3, it doesn't have any unvisited adjoining nodes. We have completed the traversal of the graph using DFS algorithm.

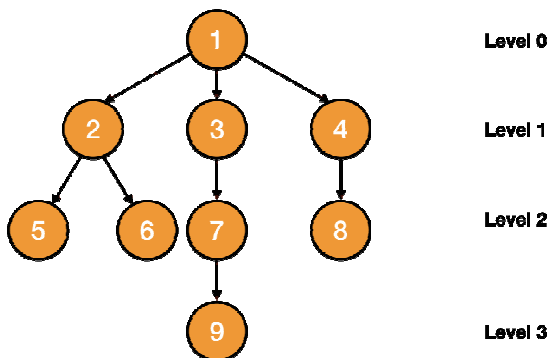


Tree Data Structures

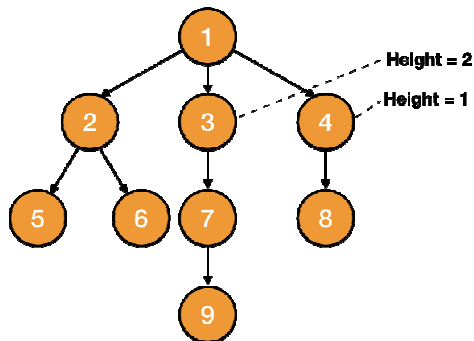
Trees are non-linear data structures that represent nodes connected by edges. Each tree consists of a root node as the Parent node, and the left node and right node as Child nodes.



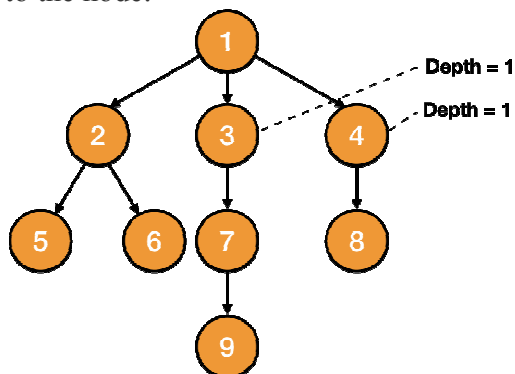
- **Root** → The topmost node of the hierarchy is called the root of the tree.
- **Child** → Nodes next in the hierarchy are the children of the previous node.
- **Parent** → The node just previous to the current node is the parent of the current node.
- **Siblings** → Nodes with the same parent are called siblings.
- **Ancestors** → Nodes which are higher in the hierarchy are ancestors of a given node.
- **Descendants** → Nodes which are lower in the hierarchy are descendants of a given node.
- **Internal Nodes** → Nodes with at least one child are internal nodes.
- **External Nodes/Leaves** → Nodes which don't have any child are called leaves of a tree.
- **Edge** → The link between two nodes is called an edge.
- **Level** → The root of a tree is at level 0 and the nodes whose parent is root are at level 1 and so on.



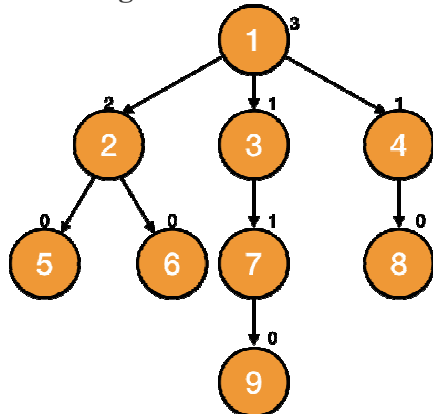
- **Height** → The height of a node is the number of nodes (excluding the node) on the longest path from the node to a leaf.



- **Height of Tree** → Height of a tree is the height of its root.
- **Depth** → The depth of a node is the number of nodes (excluding the node) on the path from the root to the node.



- **Node Degree** → It is the maximum number of children a node has.



- **Tree Degree** → Tree degree is the maximum of the node degrees. So, the tree degree in the above picture is 3.

Till now, we have an idea of what a tree is and the terminologies we use with a tree. But we don't know yet what the specific properties of a tree are or which structure should qualify as a tree. So, let's see the properties of a tree.

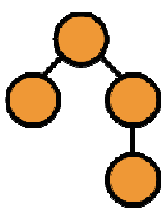
Properties of a Tree

A tree must have some properties so that we can differentiate from other data structures. So, let's look at the properties of a tree.

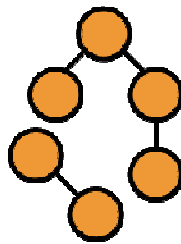
The numbers of nodes in a tree must be a finite and nonempty set.

There must exist a path to every node of a tree i.e., every node must be connected to some other node.

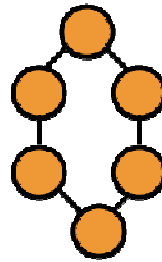
There must not be any cycles in the tree. It means that the number of edges is one less than the number of nodes.



A tree



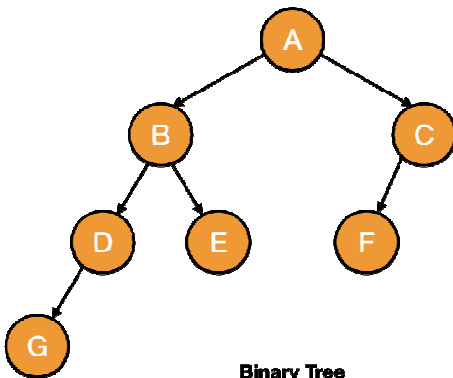
Not a tree
All nodes are not connected



Not a tree
Cycle exists

Binary Trees

A binary tree is a tree in which every node has at most two children.

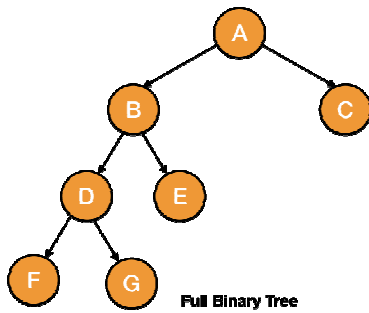


Binary Tree

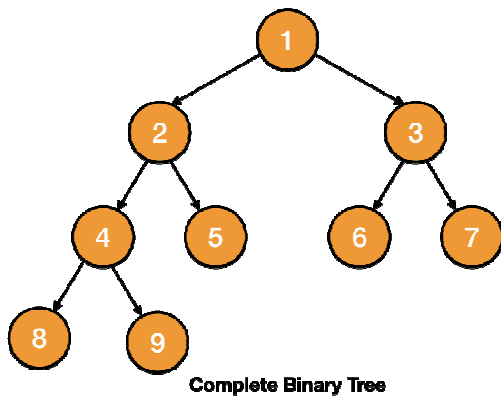
As you can see in the picture given above, a node can have less than 2 children but not more than that.

We can also classify a binary tree into different categories. Let's have a look at them:

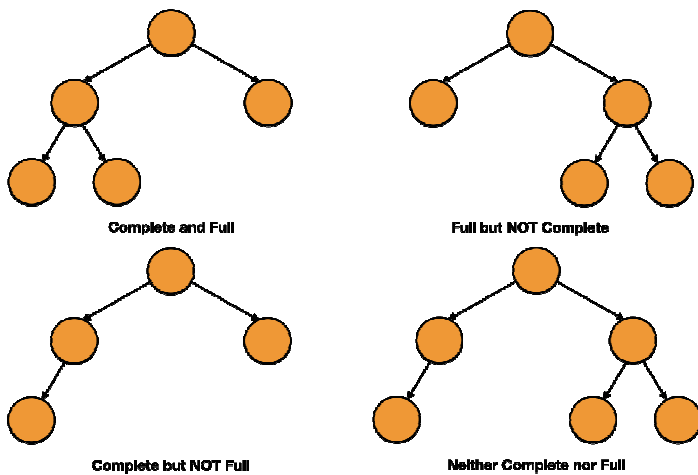
Full Binary Tree → A binary tree in which every node has 2 children except the leaves is known as a full binary tree.



Complete Binary Tree → A binary tree which is completely filled with a possible exception at the bottom level i.e., the last level may not be completely filled and the bottom level is filled from left to right.

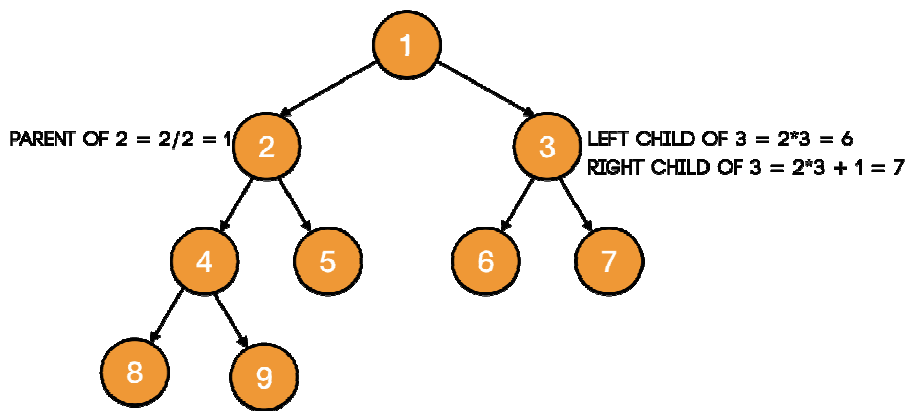


Let's look at this picture to understand the difference between a full and a complete binary tree.

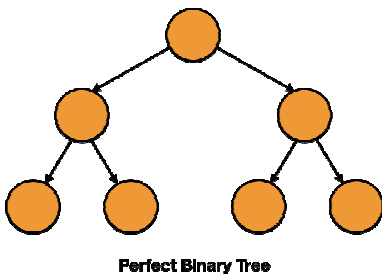


A complete binary tree also holds some important properties. So, let's look at them.

- The **parent of node i** is $\lfloor i/2 \rfloor$. For example, the parent of node 4 is 2 and the parent of node 5 is also 2.
- The **left child of node i** is $2i$.
- The **right child of node i** is $2i+1$.



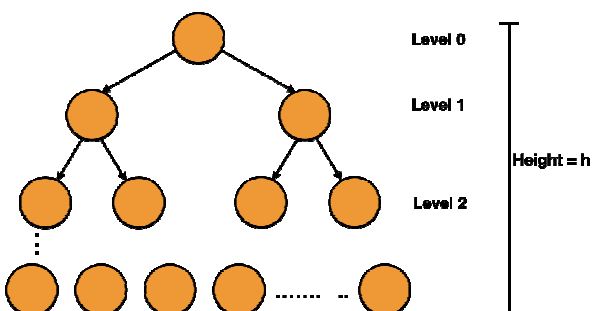
Perfect Binary Tree → In a perfect binary tree, each leaf is at the same level and all the interior nodes have two children.



Thus, a perfect binary tree will have the maximum number of nodes for all alternative binary trees of the same height and it will be $2^{h+1}-1$ which we are going to prove next.

Maximum Number of Nodes in a Binary Tree

We know that the maximum number of nodes will be in a perfect binary tree. So, let's assume that the height of a perfect binary tree is h .



Number of nodes at level 0 = $2^0 = 1$

Number of nodes at level 1 = $2^1 = 2$

Similarly, the number of nodes at level h = 2^h

Thus, the total number of nodes in the tree = $2^0 + 2^1 + \dots + 2^h$

The above sequence is a G.P. with common ratio 2 and first term 1 and total number of terms are h+1. So, the value of the summation will be $2^{h+1} - 1$.

Thus, the **total number of nodes in a perfect binary tree** = $2^{h+1} - 1$.

Height of a Perfect Binary Tree

We know that the number of nodes (n) for height (h) of a perfect binary tree = $2^{h+1} - 1$.

$$\Rightarrow n = 2^{h+1} - 1 \Rightarrow n + 1 = 2^{h+1}$$

$$\text{or, } 2^h = \frac{n+1}{2} \text{ or, } 2^h = \frac{n+1}{2}$$

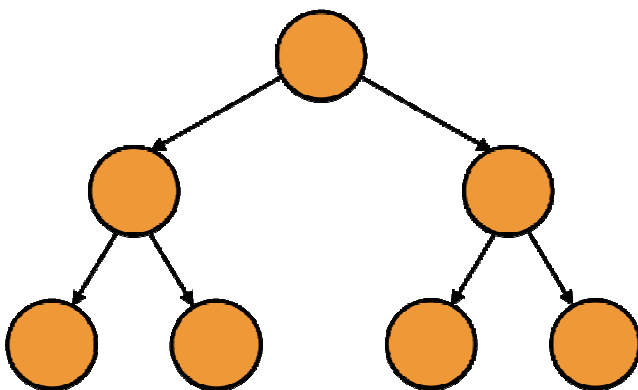
$$h = \lg \frac{n+1}{2} = \lg \frac{n+1}{2}$$

Thus, the height of a perfect binary tree with n nodes = $\lg \frac{n+1}{2}$.

We know that the number of nodes at level i in a perfect binary tree = 2^i . Thus, the **number of leaves** (nodes at level h) = 2^h .

Thus, the total **number of non-leaf nodes** = $2^{h+1} - 1 - 2^h = 2^h - 1$ i.e., number of leaf nodes - 1.

Thus, the maximum number of nodes will be in a perfect binary tree and the minimum number of nodes will be in a tree in which nodes are linked just like a linked list.



Maximum number of nodes for height 2

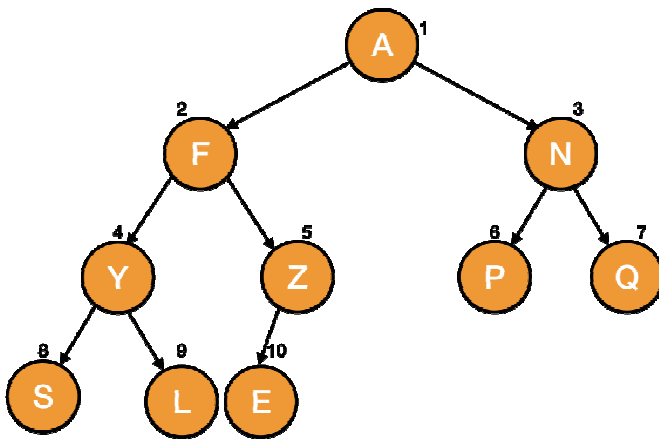


Minimum number of nodes for height 2

Array Representation of Binary Tree

In the previous chapter, we have already seen to make a node of a tree. We can easily use those nodes to make a linked representation of a binary tree. For now, let's discuss the array representation of a binary tree.

We start by numbering the nodes of the tree from 1 to n(number of nodes).

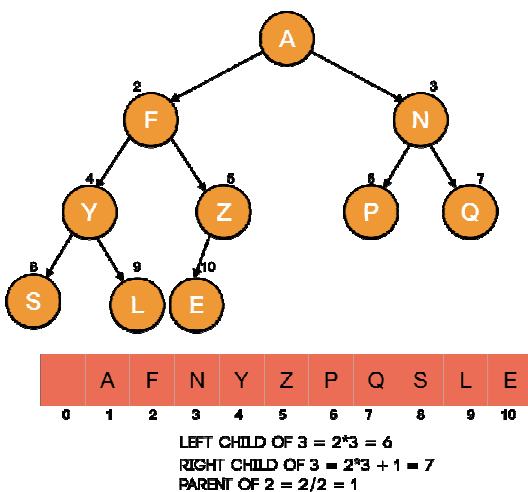


As you can see, we have numbered from top to bottom and left to right for the same level. Now, these numbers represent the indices of an array (starting from 1) as shown in the picture given below.

	A	F	N	Y	Z	P	Q	S	L	E
0	1	2	3	4	5	6	7	8	9	10

Array Representation of Binary Tree

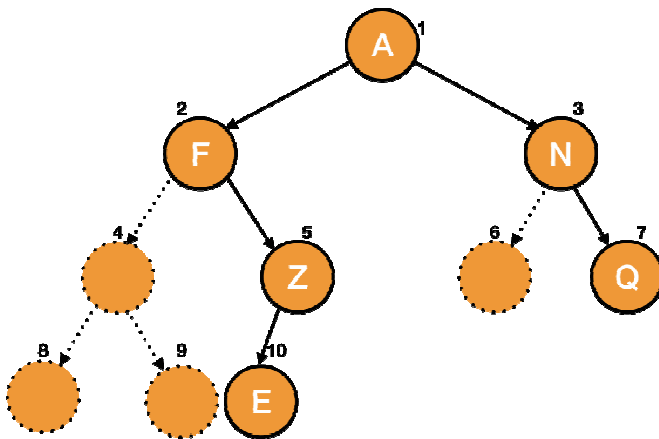
We can also get the parent, the right child and the left child using the properties of a complete binary tree we have discussed above i.e., for a node i , the parent is $\lfloor i/2 \rfloor$, the left child is $2i$ and the right child is $2i+1$.



So, we represented a complete binary tree using an array and saw how to get the parent and children of any node. Let's discuss about doing the same for an incomplete binary tree.

Array Representation of Incomplete Binary Tree

To represent an incomplete binary tree with an array, we first assume that all the nodes are present to make it a complete binary tree and then number the nodes as shown in the picture given below.



Now according to these numbering, we fill up the array.

	A	F	N		Z		Q			E
0	1	2	3	4	5	6	7	8	9	10

Coding a Binary Tree

For the linked representation, we can easily access the parent, right child and the left child with `T.parent`, `T.right` and `T.left` respectively.

So, we will first write explain the codes for the array representation and then write the full codes in C, Java and Python for both array and linked representation.

Let's start by writing the code to get the right child of a node. We will pass the index and the tree to the function - `RIGHT_CHILD(index)`.